

数 字 系 统 设 计

补 充 讲 义

王维东 刘鹏 沈继忠 徐新民 史治国 编著



浙江大学信息与电子工程学院

2019 年 5 月

0000 0001 0010 0011 0100 0101 0110 0111 1000 1001 1010 1011 1100 1101 1110 1111

目 录

第八章 控制器设计	2
8.1 概述	2
8.2 状态机设计	3
8.3 控制器的设计	17
8.4 微码控制器	27
8.5 算法与流水结构	36
8.6 系统设计实现	40
习题	41
第九章 验证和测试	44
9.1 引言.....	44
9.2 测试过程.....	44
9.3 可测性设计.....	45
9.4 测试图形的生成.....	54
习题	56
第十章 微处理器设计	57
10.1 微处理器结构	57
10.2 微处理器指令集	59
10.3 硬连线型微处理器设计	66
10.4 微码型微处理器设计.....	81
10.5 微处理器的 Verilog 实现	98
习题	112
附录	
A. 74HC/HCT181 参考手册	116
B. MIPS 指令集	122
C. Four-bit CPU 电路图	124

第八章 控制器设计

8.1 概述

数字系统中的二进制信息存储在寄存器或者存储器中，这些二进制信息主要分为数据信息和控制信息。数据是需要通过算术、逻辑、移位等微操作加以处理的离散量信息，这些操作由如加法器、译码器、乘法器、计数器和移位寄存器等数字部件执行。为了完成要实现的数据处理任务，控制信息则是提供在数据部分协调和实施各种微操作序列的命令信号。

数字系统或者数字计算机的逻辑设计，就是确定怎样执行数据处理和提供相应的控制信号，并且用数字功能部件构造成电路的过程。逻辑设计十分复杂，涉及的因素也很多。目前，人们研究和多种计算机自动设计(Computer automated design)技术来进行辅助逻辑设计。尽管如此，确定系统的技术条件和拟定达到技术所需要的算法，还是依赖于设计者的经验和理论，不能完全借助于自动设计技术。“**算法**”是指一个问题的求解过程，可以通过有限个定义的程序性步骤来说明。根据给定的设备寻求解决问题的过程，一般称之为“设计算法”。编制设计算法时，要求设计者必须对要解决的问题有深刻的理解，并且预先假设一种所采用设备的初始方案，然后才能对给定的问题和设备进行详细的求解和编制算法。

同步数字系统中，所有寄存器的时序是由一个主时钟发生器控制的。时钟脉冲作用于系统中所有的触发器和寄存器，只有当寄存器由控制信号选通时，时钟脉冲才能改变其状态。各个选择变量和各个寄存器的选通输入是由控制器产生的，控制器的输出信号不仅仅选择和选通系统中数据处理器，而且也确定控制器本身的下一个状态(次态)。数字系统中控制器和数据处理器的关系，如图 8.1 所示。

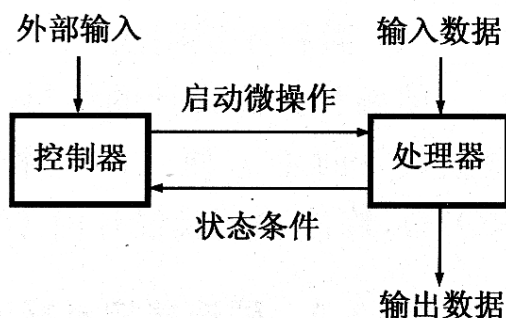


图 8.1 控制器与处理器的关系

数据处理器所有**微操作**都是由控制器启动的，产生微操作控制序列信号的控制器是一个时序电路，它的各种状态(指内部状态)表示系统的各个控制功能。任意时刻，控制器都处于一定的状态而启动一组预定的微操作，同时根据有关状态条件和输入条件进入下一个状态，进而启动其他微操作。也就是说，控制器是为启动数字系统中数据处理器微操作提供控制信号时间序列的。

设计数字系统时，该系统要求一个控制序列。将每个定时变量用一个状态表示，然后根据状态间的变化画出状态图或者其他等效的表达形式。拟定控制序列的同时，

决定每个控制状态所启动的微操作。复杂的系统用状态图无法描述时，完全可以用寄存器传送方法通过控制函数和微操作语句来描述。控制序列和寄存器传送关系可以直接从问题的文字说明中得出，但是通过时间图或者流程图来描述系统的操作序列比较直观、清楚。

时间图(Timing Diagram)可以说明各种控制信号的时间和其他关系。同步系统中，时钟脉冲同步包括控制变量改变在内的所有操作。异步系统中，一个控制变量信号的改变可能直接引起另一个控制变量的变化。时间图形象地描述了所要求的状态变化和所有控制变量的改变，所以对异步控制是非常有用的。

流程图(Flowchart)是用来描述解题过程步骤的顺序和决定算法途径的有效方法。设计算法的流程图一般采用初始设备方案中定义的寄存器名称，它将一种算法从文字说明解释成一张信息流程图，在图中列出寄存器传送操作序列和执行操作所需要的条件。算法流程图在数字系统设计过程中，表现为不同状态之间的转换描述形式，通常从表示算法的流程图可以直接推导出状态图。

控制器的逻辑设计不能脱离求解给定问题的算法拟订，还直接与由其控制的数据处理器有关。本章通过四种控制器组成的方案，说明控制器逻辑设计的各种方法。所举的例子都是从拟订算法开始，然后根据拟订的算法导出系统的数据处理器，进而说明设计按算法中给定的步骤对数据处理器确定控制顺序的控制器。

系统的控制序列确定后，便可以设计实现这些控制操作的控制器。控制器的逻辑设计不仅使控制器能实现控制序列要求的逻辑功能，而且应该使控制器的结构是个简单且非常有规则的网络。

控制器是一个时序电路，完全可以按时序电路的设计方法进行设计。通常，控制器的状态数很多，运用状态表和激励表进行设计在理论上是可行的，但是由于太麻烦，实践上是不可行的。假若按这种方法设计控制器，往往需要大量的触发器和门电路。从使用 IC 组件数量和连线数量的角度分析，这种设计方式也是不经济的。若是尽量简化以减少 IC 组件，其结果又使控制器的结构形成一个无规则的网络，给设备的维护带来很大的困难。由于上述原因，控制器的逻辑设计往往采用一些特殊的方法，这些方法是建立在经典时序逻辑方法和寄存器传送方法基础上的。一般有下列四种方法：

- (1) 每个状态一个触发器法；
- (2) 序列寄存器—译码器法；
- (3) 可编程逻辑阵列设计法；
- (4) 微程序控制法；

8.2 状态机设计

状态机是很重要的一种时序电路，是构成许多数字电路的核心部件。特别是寄存器型 PAL 器件应用大多是时序控制设计，其中经常采用状态机设计技术。随着集成电路工艺技术的发展，新型高速、高性能器件不断投放市场，它们简化了状态机的设计，使状态机设计具有各种不同层次的性能方案可供选择。

8.2.1 状态机的基本结构和功能

状态机是指按有序方式遍历预先确定的状态序列的数字逻辑功能电路。这里所说

的状态是二进制数 0 和 1 的特定组合，一般它们存储在寄存器、锁存器、PROM 器件、计数器或双稳态器件中。状态机的一般形式如图 8.2 所示。

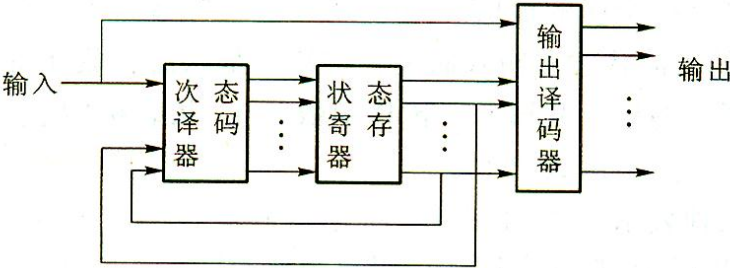


图 8.2 状态机方框图

除了输入输出端外，状态机中的状态寄存器还记忆了状态机的内部状态。状态机的下一个状态及输出，不仅同输入信号有关，而且还与当前状态有关。状态机是组合逻辑和寄存器逻辑的特殊组合，它包括两个主要部分：即组合逻辑部分和寄存器部分。寄存器用于存储状态机内部状态；组合逻辑部分又可分为状态译码器和输出译码器，状态译码器确定状态机的下一个状态，即确定状态机的激励方程，输出译码器确定状态机的输出，即确定状态机的输出方程。

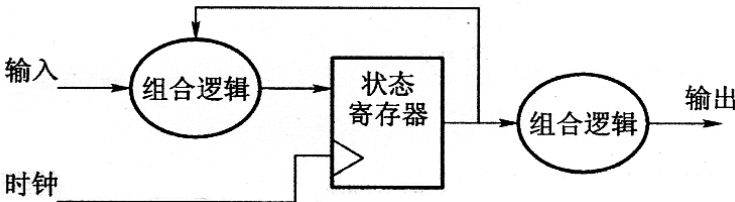
状态机主要完成两种基本操作：

1. 状态机内部状态转换。遍历某一确定的状态序列，其中次态由次态译码器根据现态和输入条件来确定。
2. 根据状态变化(称为状态转移)产生输出信号。输出译码器根据现态和输入条件可确定输出信号。

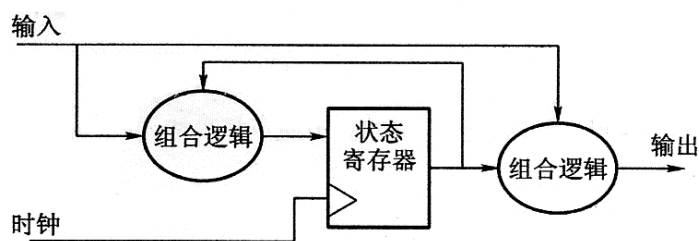
使用输入信号决定次态也称为转移。除了转移之外，复杂的时序机还具有分支、重复(循环)序列和编程等功能。从一个状态到另一个状态的转移为控制时序，而决定次态所需的逻辑称为转移函数。

在产生的输出中，是否使用输入信号决定了状态机的类型。状态机的两种基本类型是米里(Mealy)状态机和莫尔(Moore)状态机。莫尔状态机的输出只是当前状态的函数，而一般的米里状态机输出是当前状态和输入信号的函数，其中所需的逻辑称为输出函数。不论哪种类型控制时序都要取决于状态和输入信号。

有限状态机(FSM)是一种强大的时序逻辑实现方式，通过在设计中使用有限状态机，可以产生极其复杂的电路行为，满足一个复杂系统的要求。目前流行的有限状态机主要有两种实现模型，即 Moore 机和 Mealy 机，如图 8.3 (a)、(b)所示。



(b) Moore 机



(b)Mealy 机

图 8.3 Moore 机和 Mealy 机

在 Moore 机中,时序逻辑电路的输出仅仅与时序逻辑电路状态机的当前状态有关。而在 Mealy 机中,时序逻辑电路的输出不仅与时序逻辑电状态机的当前状态有关,而且与输入有关。

在图 8.3 中,输出是通过组合逻辑译码状态机的状态(在 Mealy 机中还包括输入信号)而获得的。理论上这种时序逻辑电路不会有任何问题。但是在实际的时序逻辑电路中,由于信号传输需要时间,因此极有可能导致输出译码器的所有输入信号不能同时到达,从而导致在状态机的输出上出现毛刺。如果对毛刺处理不好,就容易导致电路的误动作。因此,消除毛刺成为时序逻辑电路状态机设计的一项重点内容。一般有如下一些技术手段消除毛刺:

- 对状态编码使用 Gray 编码。如果所有的状态变化是一个顺序的序列,这可以部分消除时序逻辑电路状态切换过程中的毛刺。但是,许多时序逻辑电路状态机包含有极其复杂的分支,往往容易使得 Gray 编码消除毛刺技术途径失去效用;
- 对输出加入寄存器是一个行之有效的方法。通过在时钟边沿取样输出信号,可以极大地消除毛刺带来的影响(如图 8.4 所示)。

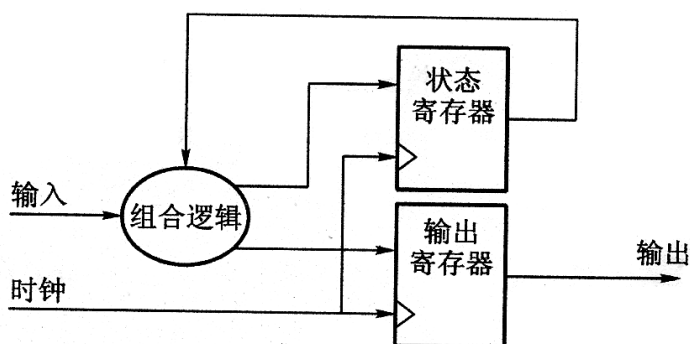


图 8.4 在输出加入寄存器消除毛刺的影响

在设计时序逻辑电路有限状态机时,还需要注意一个问题,那就是必须确保相应的状态机是安全的,也即在任何情况下,状态机不会进入一个“一去不复返”的状态。如果在状态机的运行中,出现了一个未定义的状态,而状态机又没有对这个未定义的状态进行任何处理,那么状态机就会“陷”在这个未定义的状态中,永远也不会再转移到其他的工作状态。

为了避免这种情况的出现,需要对状态机的所有可能状态进行完全编码。例如,

如果时序逻辑电路状态机的状态用 4 位表示,那么需要对所有的 16 种可能状态进行编码,并进行相应的处理。在使用 VerilogHDL 语言时由 case 语句的 default 语句,使得所有未定义的状态都会得到 default 语句中所定义操作的处理。

常用的状态机一般都采用同步时序电路,由时钟信号触发状态的转换。该类电路的主要特点是:有一个时钟信号连接到所有的状态计数器单元和输出电路触发器,在时钟的上升(或下降)沿到来时,状态计数值和输出信号发生转移和变化。此外,还有利用组合逻辑中的传播延迟以实现状态机的存储功能的异步状态机。这样的状态机容易发生组合冒险,且设计难度大,因而极少采用。下面将集中讨论同步时序状态机。

在数字系统中,那些输出取决于过去状态和当前状态的状态机称为**有限状态机**(Finite State Machine),简称 FSM,即简单状态机。有限状态机的“历史”都反映在当前状态上,当给 FSM 一个新的输入,就根据该输入及当前状态会产生一个新的输出。同时 FSM 也会转移到下一个新状态上,该新状态也由 FSM 的当前状态和输入而定。FSM 内部状态存放在寄存器中,次态的值由次态译码器中的组合逻辑—转移函数产生,状态机的输出由输出译码器中的组合逻辑—输出函数产生。

状态机有三种表示方法:状态图、状态表和流程图。三种表示方法是等价的,相互之间可以转换,但状态图法应用更为普遍。

8.2.2 算法状态机图 ASM

算法流程图是描述数字系统功能最普通的常用工具,它用约定的几种几何图形、箭头线和简练文字说明来描述系统的基本工作过程。但是,用算法流程图描述的系统仅规定了操作顺序,并未严格规定各操作的时间及操作之间的时序关系。采用同步时序结构的控制器,它在时钟脉冲的驱动下完成各种操作,为此应该对各操作之间的时间关系做出严格的描述。此外,算法流程图也没有对控制器的输出信号做具体的规定。在设计控制器时,重要的是控制器的状态转换和相应的输出,这里介绍描述时钟驱动的控制器的算法状态机图(Algorithmic State Machine Chart, ASM 图)方法。

ASM 图的图形符号以及符号的意义与算法流程图相似,但 ASM 图用来描述控制器不同时间内应完成的一系列操作,指出控制器状态转换、转换条件以及控制器的输出。它与控制器硬件实现有很好的对应关系。

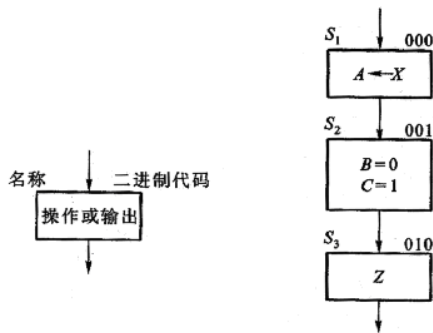
从上至下设计的基本方法是把规模较大的系统从逻辑上划分为控制电路和受控电路两大部分,在得到系统的流程图或算法并设计子系统后,接着就要设计系统的控制器。可以直接利用已有的流程图或算法状态图去设计控制器,常采用 ASM (Alogrithmic State Machine)图来描述系统控制器的控制过程,可很容易地将文字描述的设计问题变成时序流程图的描述,并求得状态函数和输出函数,从而导出相应的硬件电路。

ASM 图又称为算法状态机图,它用符合来表示系统的时序操作,类似于流程图的形式,但又不同于流程图。在流程图中表述的是系统的工作过程,即数据流在数据子系统中的流动。而在 ASM 图中不仅反映了工作顺序,而且还表明了控制器的状态转换顺序。用 ASM 图描述数字系统的工作过程,首先需要研究 ASM 图中采用的符号和规则。

一、ASM 图符号

ASM 图有三种符号：状态框、判断框和条件框。

数字系统控制序列中的状态用状态框表示，状态框的形状是一个矩形，框内标出在此状态下实现的寄存器传输操作或输出，状态的名称置于状态框的左上角，分配给状态的二进制代码置于状态框的右上角，如图 8.5 (a)所示。图 8.5 (b)是用三个状态框构成 ASM 图的实例，该状态框的名称是 S1, S2 和 S3，状态代码分别是 000、001 和 010，框内规定寄存器操作分别是把 x 的值赋给 A，B 清零，C 置 1，S3 状态时输出 Z。

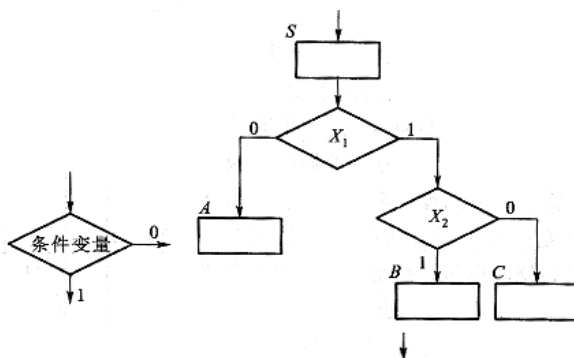


(a)状态框符号 (b)状态框组成的 ASM 图实例

图 8.5 ASM 图状态框

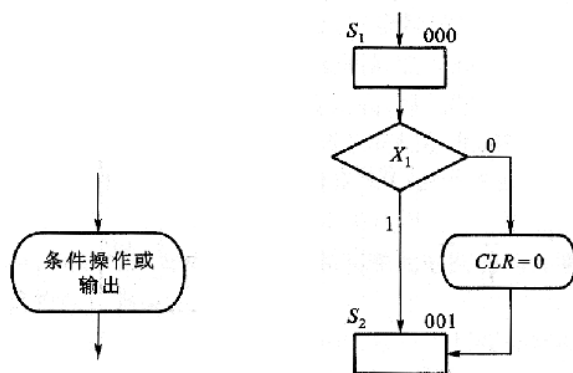
判断框表示状态变量或输入信号对控制器工作的影响，即次态不仅决定于现态，还与现态的外输入有关。菱形框内填写条件变量的判断条件，经判断框后状态转移出现两个或多个分支，如图 8.6 中(a)所示。若条件是真，选定一个分支，若条件是假，选定另一个分支。图 8.6 (b)是由两个判断框构成 ASM 图的实例。

条件框的形状为椭圆形，框内填写数据子系统进行的条件操作，框外填写必需的条件输出，条件框的输入通道必定来自判断框的分支，即条件框的操作或输出必须是在同时满足状态与条件的情况下才进行，如图 8.7 (b)所示。当系统处于状态 S1 时，如果条件 X1=0，那么 CLR 被清“0”，否则 CLR 保持不变，同时不论 X1 为何值，系统的下一状态都是 S2。



(a)判断框符号 (b)判断框构成的 ASM 图实例

图 8.6 ASM 图判断框



(a)条件框符号 (b)条件框构成的 ASM 图实例

图 8.7 ASM 图条件框

开始块与结束块的符号如图 8.8 所示，用于标注算法流程图的首、尾。当流程图的首、尾比较明确时，也可省略开始块和结束块。



图 8.8 开始块和结束块

二、ASM 图的时间划分

在 ASM 图中，一般把一个状态框和它下属的几个判断框和条件框，称为一个 ASM 块。如图 8.9 所示虚线内即为三个 ASM 块，中间最大的一个 ASM 块有一个入口和三个由判断框及条件框构成的出口，头尾两个仅有一个状态框的 ASM 块称为简单块。

ASM 块描述了一个时钟周期内系统的工作情况，它包括数据子系统和控制器两个方面，即在当前状态及条件下，数据子系统所完成的各种操作以及控制器转换的后续状态。所有的操作和状态转换都发生在时钟的同一个跳变边沿。所以 ASM 图是按时钟的节拍描述整个数字系统的操作。系统的主时钟不仅作用到数据子系统的寄存器上，而且也作用到控制器的触发器上。ASM 图与流程图的区别也就在于此，流程图中表明事件的顺序，而 ASM 图还进一步表明了状态的顺序。

下面以图 8.9 为例，说明 ASM 图的时间划分。

图中 A 是一个四位移位寄存器，同步清零和移位置数，其中 A3 为 A 的最高位，RUN 为外部输入的异步变量，LODA 为移位置数变量，它为条件输出即 $LODA = S_1 \cdot A_3 \cdot RUN$ 。根据同步时序电路的特点，可把现态变量、次态变量、测试条件及输出在 CP 有效边沿时刻的值列表，如表 8.1 所示。

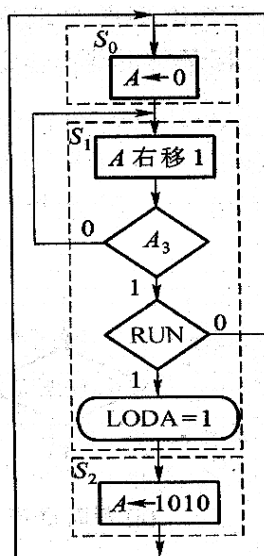


图 8.9 4 位移位寄存器的 ASM

表 8.1 图 8.8 的系统状态转换表

现态	条件		现时寄存器的状态				次态
	A ₃	RUN	A ₀	A ₁	A ₂	A ₃	
S ₀	∅	∅	0	0	0	0	S ₁
S ₁	0	∅	1	0	0	0	S ₁
S ₁	0	∅	1	1	0	0	S ₁
S ₁	0	∅	1	1	1	0	S ₁
S ₁	0	∅	1	1	1	1	S ₁
S ₁	1	0	1	1	1	1	S ₀
S ₀	∅	∅	0	0	0	0	S ₁
...			...				
S ₁	1	1	1	1	1	1	S ₂
S ₂	∅	∅	1	0	1	0	S ₀

根据列表还是无法看出异步信号的输入带来的时序影响,如 RUN 是一个异步输入,它的持续时间不一定是一个 CP 周期,因而 LODA 就可能有两种情况,如图 8.10 所示。在图 8.10 (a)中, RUN 持续的时间比较长,在下一个 CP 的上升沿处 LODA=1,则状态从 S1 转换到 S2;而在图 8.10 (b)中,由于在 CP 上升沿处 RUN 为零,导致状态跳回 S0。

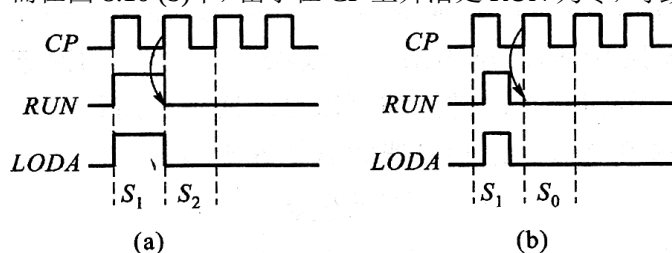


图 8.10 异步信号导致输出的变化

三、ASM 图的建立

根据系统流程图或算法等已有模型建立系统的 ASM 图要注意的原则为：

1. 流程图中的工作块基本上对应了 ASM 图中的状态框。但如果工作块的操作不能在一个 CP 内同时进行，在 ASM 图中就必须将其分为几个状态框，在这几个状态之间实现无条件转移。

2. 流程图中的判断块基本上对应了 ASM 图中的判断框。但如果判断条件是上个操作的结果，那么在 ASM 图中应在此判断框前增加一个状态框，因为在 ASM 图中，各个 ASM 块的执行时间是同一个 CP 的有效边沿，如果不增加一个状态框，则判断条件对应于前一个 CP 的工作块的操作结果。

3. 一般在 ASM 图的最上层加一个起始状态。

下面就以例题来说明如何建立 ASM 图。

【例 8.1】串行数据序列是每个时钟周期传送一个数据 0 或 1 的数据流。设 x 为输入的串行数据序列。当检测到数据流中出现所需的 010 数据时，使检测器的输出 Z 为 1。试画出其 ASM 图。

解 这是一个米里型时序电路，其输出 Z 与输入信号有关。

设 S_0 为起始态， S_1 为收到 X 序列里的 0 后的状态， S_2 为收到 X 序列里 0-1 后的状态，若接着后面 X 输入的是 0 的话，则输出 $Z=1$ ，并且状态返回 S_0 。依此可以得到状态转换图。如图 8.11 (a) 所示。

根据 ASM 图建立的原则，由状态转换图即可得到对应的 ASM 图，见图 8.11 (b) 所示。

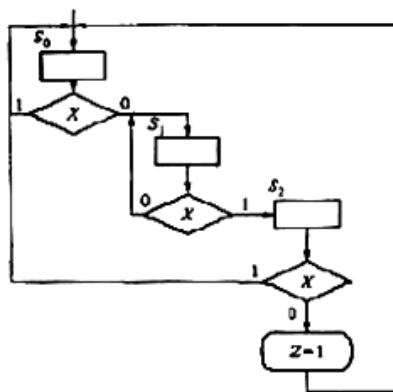
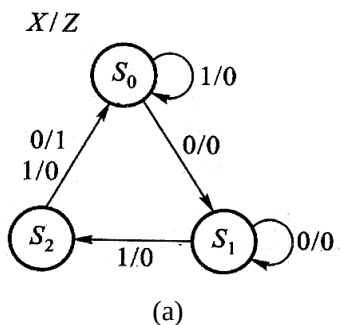


图 8.11 例 8.1 的状态转换图及 ASM

【例 8.2】试设计一个米里型同步模 4 可逆计数器，推导出该计数器的算法流程图。

解 计数器功能分析：

(1)当控制信号 $x=0$ 时，计数器对输入脉冲 CP 进行累加计数，并在累加 4 个脉冲时，输出进位 $z=1$ 。

(2)当控制信号 $x=1$ 时，计数器对输入脉冲 CP 进行累减计数，并在累减 4 个脉冲时，输出借位 $z=1$ 。图 8.12(a)给出了模 4 可逆计数器的示意图。

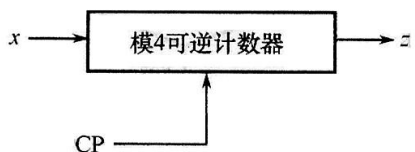


图 8.12(a) 模 4 可逆计数器示意图

根据题意，模 4 计数器有两个二进制寄存器， $R=R_1R_2$ 。

模 4 计数器算法的具体做法为：

$$x = \begin{cases} 0 & R \leftarrow R_i + 1 & \text{当 } i = 3 \text{ 时, } Z = 1 (i = 0 \sim 3) \\ 1 & R \leftarrow R_i - 1 & \text{当 } i = 0 \text{ 时, } Z = 1 (i = 0 \sim 3) \end{cases}$$

其状态图如图 8.12(b)所示。

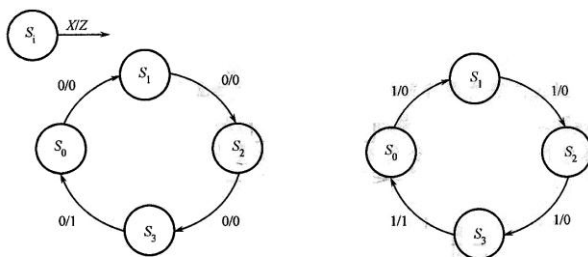


图 8.12(b) 模 4 可逆计数器状态图

由此可得出图 8.12(c)所示的算法流程图。实现图 8.12(c)所示算法流程图的完整逻辑电路如图 8.12(d)所示。

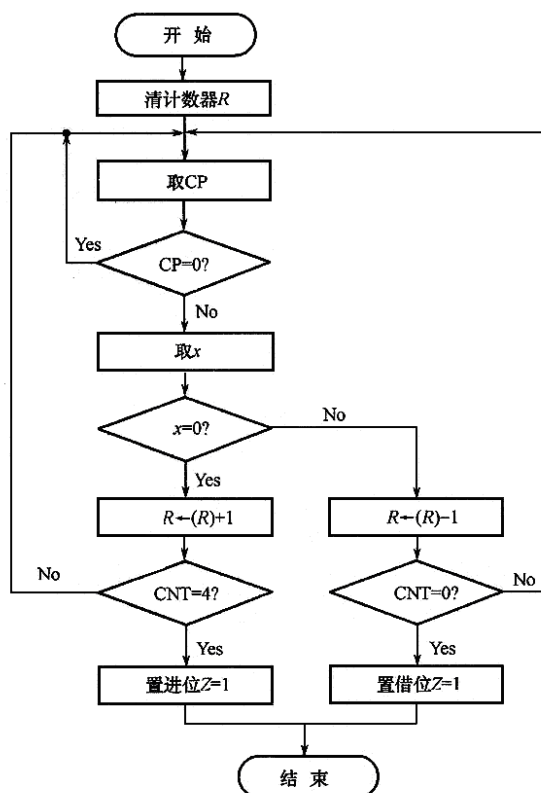


图 8.12(c) 模 4 可逆计数器的算法流程图

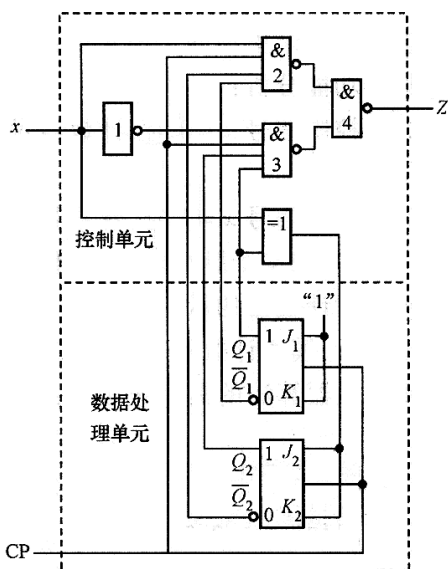


图 8.12(d) 模 4 可逆计数器逻辑图

从图 8.12(d)可以清楚地看到，这个简单的数字系统由控制单元和数据处理单元两部分组成。控制单元的基本功能在于对指令和数据流实施时间和空间上的正确控制。而数据处理单元是在控制信号的控制下，有序地实现数据处理操作。

8.2.3 状态机设计

状态机的设计可分为以下四个步骤：

- 1. 确定任务。首先要对实际问题进行分析和归纳，以确定控制系统的任务以及要实现的功能。
 - 2. 把设计任务转化为状态图。即列出采用的状态机全部可能的状态，并对每一个状态进行状态编码及定义相应的状态转换条件。
 - 3. 根据状态图(或把状态图转化为状态表，并对状态图和状态表进行必要的简化处理)和输出函数，画出状态转移图。
 - 4. 建立激励函数和输出函数，画出逻辑电路。
- 下面以例题来说明上述状态机的设计方法。

【例 8.3】已知某数字系统的控制器 ASM 图如图 8.13 所示，试设计对应的逻辑电路图。

解 1. 对 ASM 图进行状态分配

参照时序电路状态分配的原则，用三位状态变量 Q_2 、 Q_1 、 Q_0 的二进制编码赋给 ASM 图中的 S_0 、 S_1 、 S_2 、 S_3 和 S_4 五个状态

$S_0=000 \quad S_1=001 \quad S_2=011$
 $S_3=010 \quad S_4=110$

从而可得编码的 ASM 图如图 8.14 (a)所示，状态分配图如图 8.14 (b)所示。

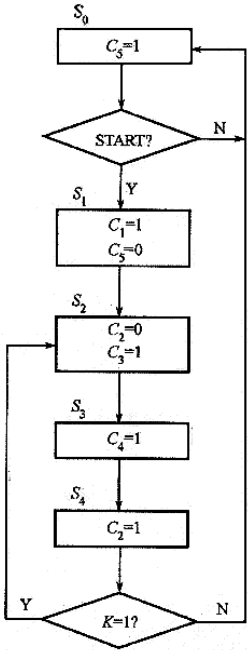


图 8.13 某数字系统的控制器 ASM 图

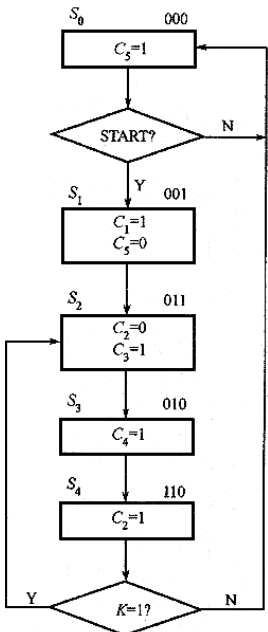


图 8.14 (a) 对应图 8.13 编码 ASM 图

2. 建立激励表

若选择 D 触发器作为控制器的状态寄存器，则由编码 ASM 图建立三个 D 触发器激励函数，卡诺图是设计中的重要步骤，下面简要介绍建立的方法。

在图 8.14 (a)给定的 ASM 图中，状态 $S_0(000)$ 有状态分支；当 START 到来时，状

态 S_0 转换到状态 $S_1(001)$ ，否则状态 S_0 保持不变。由此看出，无论 $START$ 来到与否， Q_2 和 Q_1 总保持为 0，因此 D_2 和 D_1 激励函数卡诺图上与状态 S_0 对应的小方格中均填写 0，而 S_0 在满足 $START=1$ 时， D_0 由 $0 \rightarrow 1$ ，否则保持 0，因此 D_0 卡诺图上状态 S_0 对应的小方格中应填写 D_0 置 1 的条件 $START$ 。显然，当 $START=0$ 时，由于 $D_2=D_1=D_0=0$ ，在下一个时钟作用下， $Q_2=Q_1=Q_0=0$ ，控制器保持在状态 S_0 ，只有当 $START=1$ 时，下一时钟检查到 $D_2=D_1=0$ ，而 $D_0=1$ ，则使 $Q_2Q_1Q_0=001$ ，控制器由状态 S_0 转换为状态 S_1 。 D_2 、 D_1 、 D_0 的激励函数卡诺图如图 8.15 所示。

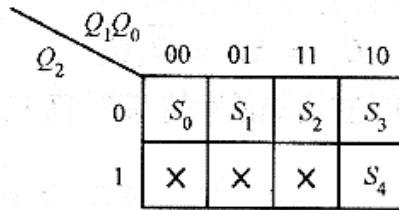


图 8.14 (b) 对应图 8.13 的状态分配图

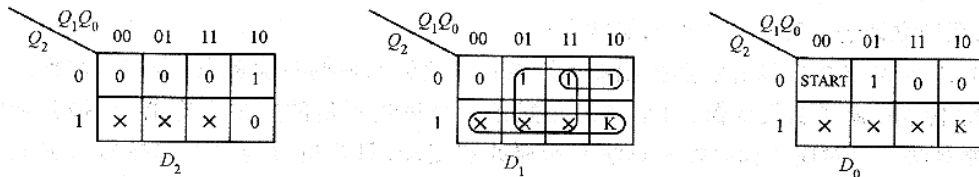


图 8.15 激励函数卡诺图

ASM 图中 $S_1 \rightarrow S_2$ ， $S_2 \rightarrow S_3$ ， $S_3 \rightarrow S_4$ 均为无判别条件的状态转换，只要时钟脉冲到来，自动按照激励信号进行变换，因此，与这些状态(S_1 、 S_2 、 S_3)相应的卡诺图小方格中，直接填入各自次态的二进制编码即可。

考察 ASM 图中状态 $S_4(110)$ 的分支情况，当满足条件 $K=1$ 时，转换为状态 $S_2(011)$ ，即 Q_2 由 $1 \rightarrow 0$ ， Q_1 保持， Q_0 由 $0 \rightarrow 1$ ；当满足条件 $K=0$ 时，转换为状态 $S_0(000)$ ，即 Q_2 由 $1 \rightarrow 0$ ， Q_1 由 $1 \rightarrow 0$ ， Q_0 保持为 0，因此在 D_2 卡诺图上与状态 S_4 对应的小方格中填写 0。因为无论 K 为什么逻辑值，次态 S_2 和 S_0 的 Q_2 均为 0， D_1 卡诺图上对应方格中填写 K ， D_0 卡诺图上对应小方格中也填写 K 。这些均已在图 8.15 中标明。

由卡诺图求得激励函数为

$$D_2 = \bar{Q}_2 \bar{Q}_1 \bar{Q}_0$$

$$D_1 = Q_2 K + \bar{Q}_2 Q_1 + Q_0$$

$$D_0 = \bar{Q}_1 START + Q_2 K + \bar{Q}_1 Q_0$$

3. 输出函数方程

在 ASM 图中，每个状态块中表明了该状态的输出，条件输出由椭圆块表示；因此从 ASM 图得到输出函数方程十分简便。这里应注意的是输出信号的极性问题，也就是输出信号是以高电平有效，还是以低电平有效。例如，进入 S_1 状态， C_1 信号以高电平有效，进入 S_2 状态， $C_4=0$ ，表示 C_4 信号在此状态持续期间以低电平为有效等。

输出信号方程为

$$C_5 = S_0 = \bar{Q}_2 \bar{Q}_1 \bar{Q}_0$$

$$C_4 = S_3 = \overline{Q_2} Q_1 \overline{Q_0}$$

$$C_3 = S_2 = \overline{Q_2} Q_1 Q_0$$

$$C_2 = S_4 = Q_2 Q_1 \overline{Q_0}$$

$$C_1 = S_1 = \overline{Q_2} \overline{Q_1} Q_0$$

在本例中，所有输出均为无条件输出，信号持续时间只与状态有关。

4. 逻辑电路图

本例控制器逻辑电路图如图 8.16 (a)所示。图中 START 是系统外部的输入，已经同步化处理；K 是数据处理单元送来的工作状况反馈信号；图中 Cr 信号为异步复位信号，开机复位，系统立即进入算法流程的初始状态。

为使 ASM 图的控制算法和控制器电路结构有此较明显的对应关系，采用了数据选择器(MUX)构成激励函数发生器，每个触发器配备一只适当容量的数据选择器，这种激励电路与控制算法一一对应。如果控制算法有所修改和变动，只要更改与数据选择器有关通道的输入，再不必更改激励电路，这样就为控制器设计提供了方便。

5. 控制器的工作波形图

建立控制器逻辑电路图以后，该控制器的工作波形图如图 8.16 (b)所示，外部输入 START 信号保证 CP 脉冲的有效作用沿能够检查到它的变化。并设定状态发生器记忆元件均响应 CP 脉冲的上升沿(状态变化发生在 CP 的哪个作用沿可由设计者自定)。

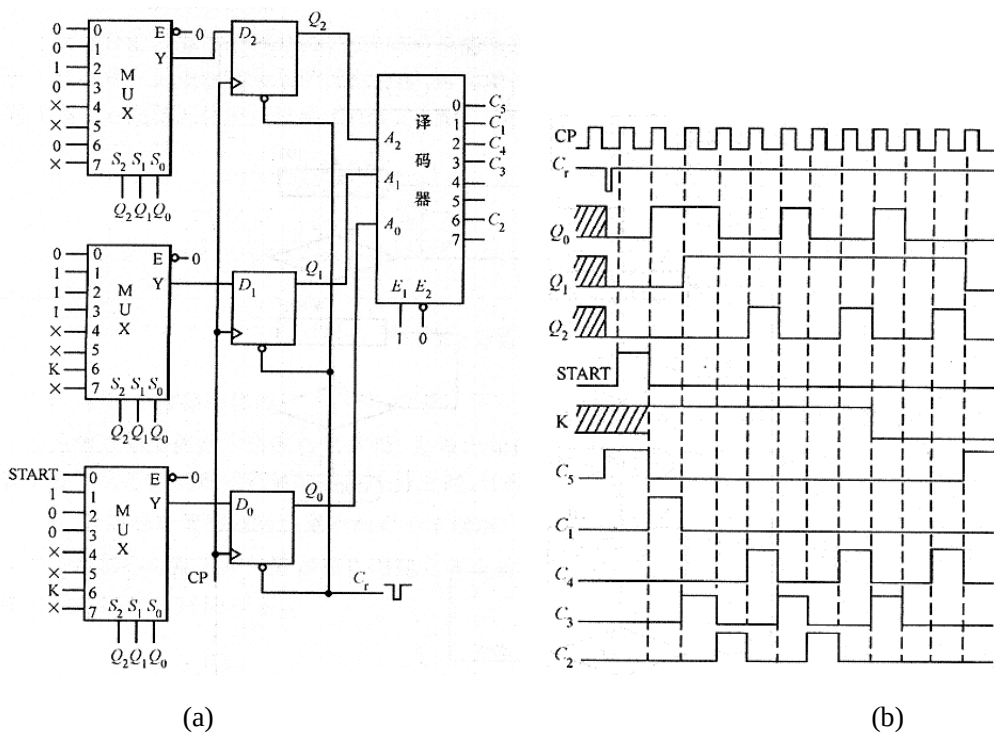


图 8.16 控制器逻辑电路图和波形图

【例 8.4】试用状态机法设计一个异步清零的十进制加/减法可逆计数器。

解 1. 确定任务。

据题意，得到计数器的设计方框图，如图 8.17 所示。

电路中 $Q_0 \cdots Q_3$ 为当前计数值，其中 Q_0 为最低位， Q_3 为最高位。计数的加/减由 INF 信号决定：当 INF 为高电平时，计数器每来一个时钟脉冲加 1；当 INF 为低电平时，则每来一个时钟脉冲减 1。有效计数范围是 0 至 9，在加计数时，计数从 0 到 9，再返回到 0；在减计数时，计数反过来，从 9 到 0，再回到 9。

由于 PLD 的输出要经过一级缓冲器，所以这里定义使能端 OE 为输出允许端，当 OE 为低电平时，输出允许；当 OE 为高电平时，输出呈高阻状态。不过即使在输出被禁止时，计数器仍在时钟到来时刻进行加/减计数，使能端 OE 控制的只是输出缓冲器，而非寄存器。因而，寄存器数值的变化与实际输出值无关。

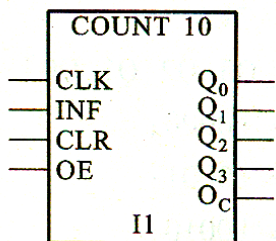


图 8.17 计数器框图

2. 状态转移图。

由于存在十个可能的计数值，所以对应要十个状态分别表示，如图 8.18 中的 $s_0, \dots, s_9$ 。输入信号 CLR 和 INF 决定了计数器的工作方式，计数器有加、减、清零三种工作方式，分别用 11, 10 和 00 表示，其中清零工作方式中 X 的状态为无关项。

3. 数字电路实现。

10 个状态根据编码原则可以用 4 位二进制数来表示。对应的状态机使用 4 个寄存器。(具体详细电路图留给读者练习)

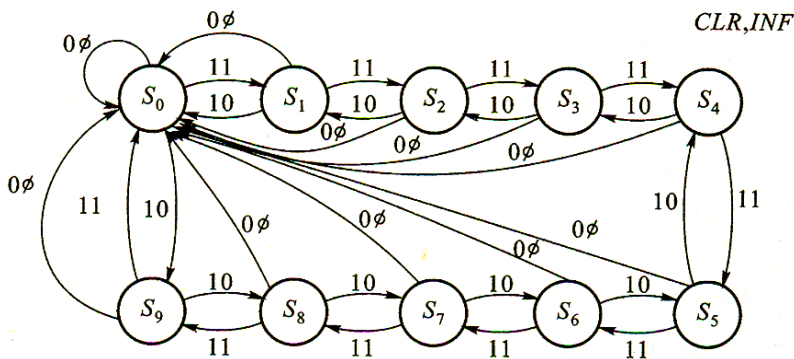


图 8.18 状态转移图

8.3 控制器的设计

8.3.1 控制器的基本概念

数字系统中，各个子系统通过数据总线联接形成数据通路，在一系列控制信号的控制下，才能实现数据在各子系统之间的传输、处理或存储。这些控制信号就来自控制器，所以控制器本质上是一种时序网络。

控制器的设计是数字系统硬件设计的中心子系统，它的作用是解释所接收到的各个输入信号，根据输入信号和预定的算法流程图程序使整个系统按指定的方式工作。控制器的控制程序可以直接由硬件实现，也可以由固件(固化的控制软件)实现。例如，微码程序控制器就是把控制命令固化到 ROM 中，然后读出并执行各种控制操作的一种灵活的控制装置。

由硬件直接实现的控制器设计与一般时序电路并无区别，仅仅是设计着眼点不同。控制器设计的主要特点是：控制器的成本仅占系统总成本的很小一部分，其性能却对整个系统的工作有举足轻重的影响，所以控制器的设计不必追求状态最简，触发器的数量也不必追求最少。有时，在控制时序中增加一些多余状态，会使数字系统工作更加直观，便于监视和检查故障。在状态化简时，应首先考虑工作性能的优劣，工作是否可靠直观，不必过分追求最简状态。为了使控制状态单纯而明确，可采用“一对一”法设置触发器，即一个状态设一个触发器，避免状态分配的麻烦。

控制器的形式可以多种式样，但其基本设计方法有较强的规律性。本节介绍小型控制器的一般设计方法，下一节将介绍通用性强、结构简单、可用于大型数字系统的微程序控制器设计。

8.3.2 计数器型控制器

有了 ASM 流程图，把它转换成硬件就变成一件比较简单的工作了。只要按给定的 ASM 流程图构造一个状态发生电路，使它具有 ASM 中所需的全部状态，并能依照控制算法条件进行状态转移和条件输出就行了。

“状态发生电路”的形式多种多样，因而控制器的形式也是多种多样的。由于计数器本身有许多不同的状态，因而各种型式的计数器均可以改造为控制器，这只要按照控制条件实现状态的转移，就可以从不同状态下的输出得到所需要的控制信号。控制状态数较少的控制器可采用环形计数器作为状态发生电路，它的一个触发器对应一种控制状态。当控制状态数较多时，为了节省触发器数目，宜采用编码方式组成状态。对 n 个触发器进行编码最多可代表 2^n 个状态，也就是可以构成 2^n 个状态编码。将所要求的控制状态按一定原则进行编码分配，就可设计出一种状态计数型的控制器，通常称之为计数器型控制器。

计数器型控制器的一般框图如图 8.19 所示。图中计数器含有 n 个触发器，触发器的状态作为状态变量以二进制编码的形式赋于 ASM 流图中的每一个状态框，而条件输出框不予赋值。按照输入条件及状态流程图所规定的状态转移的要求，设计状态控制逻辑，使计数器状态按流程图所规定的转移顺序进行计数转移。计数器状态经译码后输出，作为有关的控制信号。

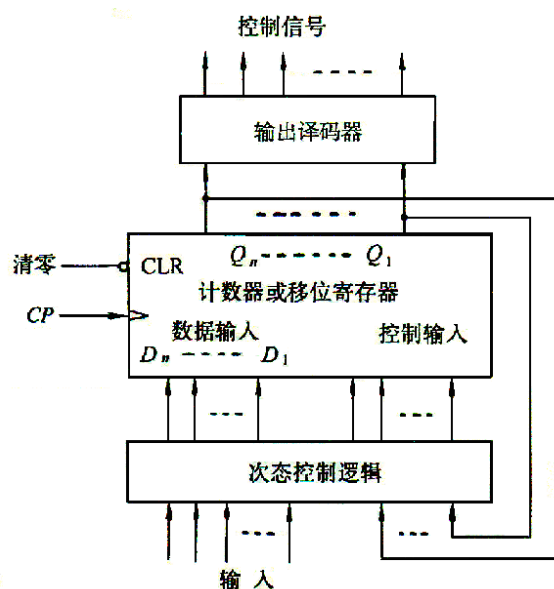


图 8.19 计数器型控制器的结构框图

【例 8.5】图 8.20 为某一控制器的算法流程图，请设计一个计数器型控制器。

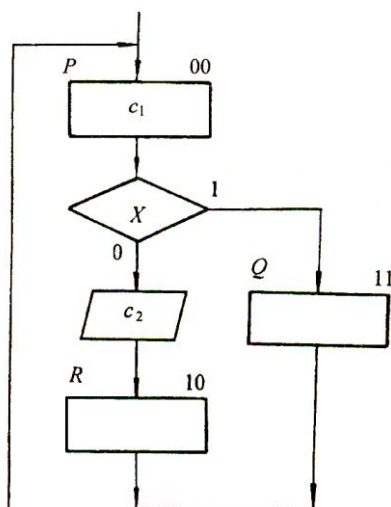


图 8.20 状态已编码的 ASM 流程图

解 根据控制算法流程图可知，该控制器需要两个触发器，应设置两个状态变量 A 和 B。触发器可采用 JK 型或 D 型。此处采用 D 型触发器。

从 ASM 流程图可看出算法状态机的输入和输出参数。本例的算法状态机的输入为 X，并有 C_1 和 C_2 两个控制命令输出。使用两个触发器 A 和 B 可以组成三种编码 00、10 和 11，故其基本电路模型如图 8.21 (a) 所示。

用来给出次态激励函数的组合逻辑应完成表 8.2 所示的逻辑运算。编码 01 状态没有用到，为了避免电源刚接通时状态被锁在 01 的无用状态，必须使 01 的次态为某一有效状态，设 01 的次态为 00。这样就需要把编码方式状态发生电路所有四种不同的

编码情况都考虑在内，同时解决了该电路的初态引导问题。

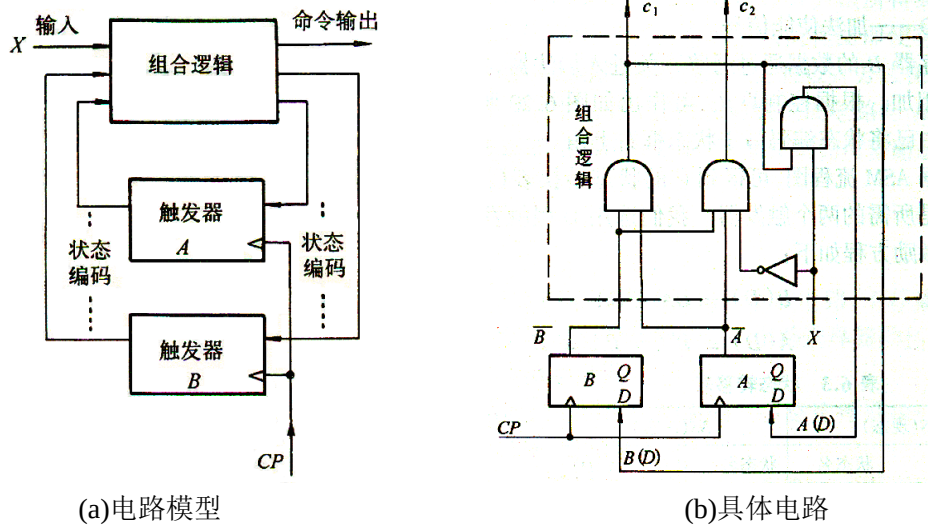


图 8.21 例 8.5 的计数器型控制器

根据表 8.2，不难写出触发器 A 和 B 的次态激励函数 $A(D)$ 和 $B(D)$ 的表达式：

$$B(D) = \overline{B} \overline{A} X + \overline{B} A \overline{X} = \overline{B} \overline{A}$$

$$A(D) = \overline{B} \overline{A} X$$

表 8.2 状态激励函数真值表

PS			NS	
B	A	X	B(D)	A(D)
0	0	0	1	0
0	0	1	1	1
0	1	×	0	0
1	0	×	0	0
1	1	×	0	0

控制器的主要目的是产生一定的控制命令。从算法流程图可得控制信号：

$$C_1 = \text{状态 P} = \overline{B} \overline{A}$$

$$C_2 = \text{状态 P 和 } \overline{X} \text{ 的“与”} = \overline{B} \overline{A} \overline{X}$$

最后就可以按上述分析构成控制器的电路，如图 8.21 (b)所示。

【例 8.6】如图 8.22 a)所示，一个简单的 8 位二进制无符号数并行加法运算器，数据从存储器通过同一个总线输入，控制算法分四步：1) 溢出标志寄存器 C 清零，先取被加数至寄存器 A；2) 将 A 中被加数送寄存器 B；3) 后取加数至寄存器 A；4) 将 A 与 B 中的数相加，结果存于 B，且 1 位进位溢出标志存放于寄存器 C。请设计该控制器，要求采用计数器型。假设状态周期 $T = T_1 + T_2$ 计数器状态变化发生在 T_1 时序，控制信号 LDA，LDB 发生在 T_2 时序。

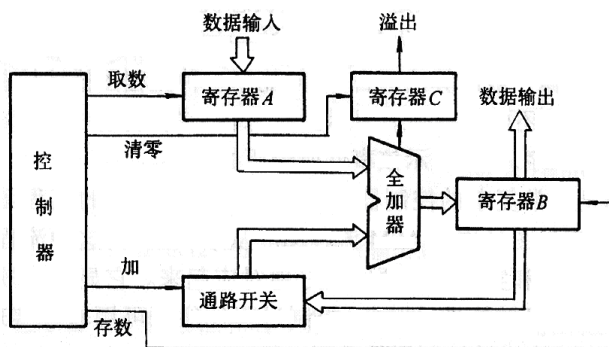


图 8.22 a) 累加运算器基本框图

解 并行加法运算器的控制器的设计中，需要四步操作来完成两个数相加，要求控制器有四个控制状态。设四个状态的名字为 a、b、c、d，并把操作命令命名为

$\overline{CLR}$ — 寄存器清零；

LDA — 寄存器 A 接受数据；

LDB — 寄存器 B 接收从加法器送来的数据；

ADD — 加法使能信号。

寄存器 B 的数据通过通路开关进入加法器，并与寄存器 A 送来的数相加。根据控制算法，可作出如图 8.22 b) 所示的 ASM 流程图。图中已将状态编码写于状态框右上角。

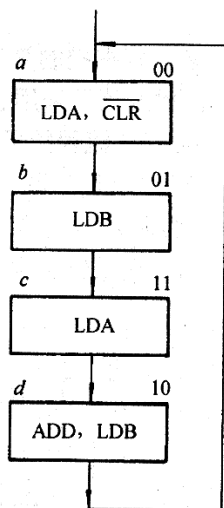


图 8.22 b) 例 8.6 的 ASM 流程图

根据 ASM 流程图，可得到它的状态转换表(见表 8.3)。表中 A 和 B 是所需的两个触发器。我们选用 D 型触发器，则求得触发器次态激励方程如下：

$$B(D) = \overline{B}A + B\overline{A} = A$$

$$A(D) = \overline{B}\overline{A} + B\overline{A} = \overline{B}$$

表 8.3 状态转移表

PS(现态)			NS(次态)		
B	A	状态名	状态名	B(D)	A(D)
0	0	a	b	0	1
0	1	b	c	1	1
1	1	c	d	1	0
1	0	d	a	0	0

从算法流程图还可得到各控制信号的逻辑函数关系为

$$LDA = (\bar{B}\bar{A} + BA)T_2$$

$$LDB = (\bar{B}\bar{A} + \bar{B}A)T_2 = (B \oplus A)T_2$$

$$\overline{CLR} = \bar{B}\bar{A}$$

$$ADD = B\bar{A}$$

其中 LDA、LDB 为脉冲控制信号， $\overline{CLR}$ 、ADD 为电位控制信号。写入控制信号 LDA、LDB 应在 T_2 上升沿有效。电位控制信号持续时间应与状态周期 T 相同。

根据上列关系，可画出如图 8.23 所示的运算器与控制器电路图。寄存器 A 和 B 选用 74LS273 八 D 触发器，寄存器 C 选用 74LS74，三态缓冲器采用 74LS244，全加器选用 2 片 74LS283。注意，图中右边的控制信号是电位信号，为了加入时间因素 T_2 ，左边电路中增加了少量的与门以产生脉冲控制信号。

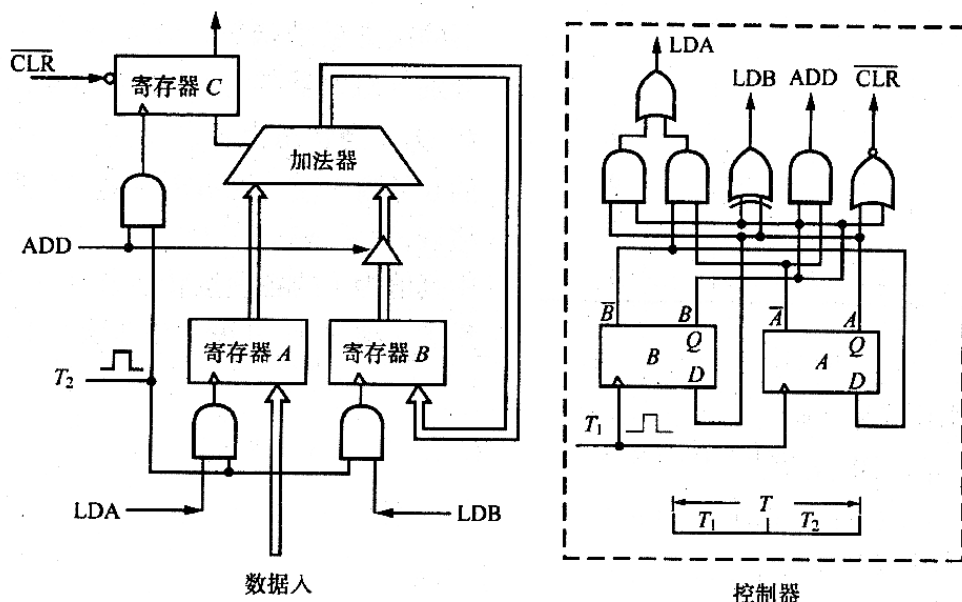


图 8.23 并行加法运算器及其计数器型控制器

上面所举例子中，控制器的设计方法是一种经典方法。这种设计方法所得到的硬件逻辑电路与它所代表的控制算法之间没有明显的对应关系。算法流程图即使有微小变动，也会牵动全局，都要重新逐一计算生成次态激励函数。显然，这种设计方法在“清晰明确”方面还存在着不足。因为我们希望设计过程标准化，次态函数生成电路

与 ASM 流程图有清晰的对应关系。下面介绍的用多路选择器构成次态函数生成电路就是其中一种设计方法。

8.3.3 多路选择器型控制器

采用多路选择器作为控制器状态触发器的次态激励函数的生成电路，不仅能使控制器的设计过程标准化，而且可以使整个控制器电路“清晰明确”。多路选择器的任务是：按控制算法要求，为其对应的触发器生成次态激励函数。

显然，所有多路选择器输出的组合就是控制器次态的编码。多路选择器应有足够的输入端，使得所有的状态变量都有其相应的输入端。

设计的任务是，按 ASM 控制算法要求，给多路选择器的每一个输入端提供适当的输入值，使之能给出次态所要求的逻辑值。为此，我们给 ASM 图中的每一个二进制编码状态赋予一个十进制数码，以便和多路选择器的输入端编码相对应。然后按 ASM 流程图建立起状态转换表，进而求得多路选择器的各个数据输入端的函数值，完成控制器的设计。

【例 8.7】设计一个多路选择器型控制器，实现图 8.20 的控制算法。

解 根据算法流程图可作出如表 8.4 所示的状态转移数据表。状态转移数据表比通常的状态转移表多了一栏—转换条件。转换条件栏的数据或表达式，也就是对应的多路选择器的数据输入值。

表 8.4 图 8.20 的状态转移数据表

PS(现态)		NS(次态)			转换条件
编码	状态名	状态名	B	A	
0 (00)	P	R	1	0	$\overline{X}$
		Q	1	1	X
2 (10)	R	P	0	0	0
3 (11)	Q	P	0	0	0
1 (01)	-	P	0	0	0

设电路选用两个 D 型触发器 FA 和 FB，相应地需要两个 4 位数据选择器。多路选择器的输出就是触发器的输入，也就是触发器的次态激励函数。多路选择器的控制端分别与触发器的 Q 输出端相连，标之以 Q_A 和 Q_B 。当状态为 $Q_A=0, Q_B=0$ 时，选择器的 0 号输入端(数据端)被选中，根据状态转移表，利用 $NS = \sum PS \cdot C$ 公式，将次态变量中真值为 1 的各项按转换条件写出(如 A 变量中 1 项为 1，B 变量中 2 项为 1)，于是得：

$$\begin{aligned} MUXA(0) &= X & MUXB(0) &= X + \overline{X} = 1 \\ MUXA(2) &= 0 & MUXB(2) &= 0 \\ MUXA(3) &= 0 & MUXB(3) &= 0 \\ MUXA(1) &= 0 & MUXB(1) &= 0 \end{aligned}$$

此时用 PS 作为 MUX 的地址输入，用转换条件 C 作为 MUX 的数据输入。例如，

PS=00 时，触发器 FB 的次态将为 1，而触发器 FA 的次态取决于输入 X。当系统处于其他 PS 状态时，NS 都应 00，故 MUX 的输入数据为 0,0。因此可得图 8.24 所示的 MUX 型控制器电路。其控制信号可从 ASM 流程图的输出条件直接得到，并利用触发器的反码端信号得以简化。

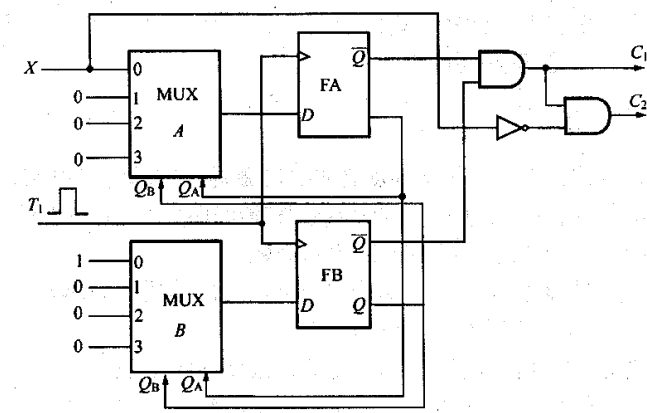


图 8.24 对应图 8.20 的 MUX 型控制器

用 MUX 来设计控制器的方法简单易行，设计过程相当规正。当 ASM 流程图有变化时，只需对电路中相应的 MUX 输入作适当改变就行了，不至于牵动全局，重新计算各状态激励函数。所使用的器件基本上不受算法流程图变化的影响。一旦熟悉了这种设计方法后，可以不必列出次态函数计算式，而直接从控制算法流程图决定多路选择器的各个输入值。

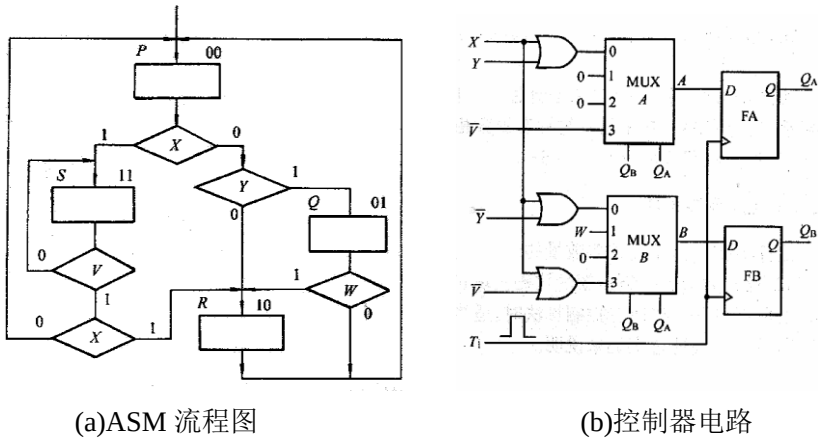


图 8.25 例 8.8 的 MUX 型控制器

【例 8.8】 图 8.25 (a)为某控制器的 ASM 流程图，试用 MUX 来实现该控制器。

解 由算法流程图可得到表 8.5 所示的状态转移数据表。该控制器共需两个触发器，两个 4 位多路转换器，表中 A 和 B 为状态变量。

根据状态转移数据表，可以求得各个状态的转换条件，也就是 MUX 的数据输入。例如，求 MUXA(0)时，只需将状态 0(即状态 P)的次态变量 A 取真值为 1 的各项相加即得，即有

表 8.5 图 8.25 ASM 的状态转移数据表

PS(现态)		NS(次态)			转换条件
编码	状态名	状态名	B	A	
0 (00)	P	S	1	1	X
		Q	0	1	$\overline{XY}$
		R	1	0	$\overline{XY}$
1 (01)	Q	P	0	0	$\overline{W}$
		R	1	0	W
2 (10)	R	P	0	0	-
3 (11)	S	P	0	0	$V\overline{X}$
		R	1	0	VX
		S	1	1	$\overline{V}$

$$MUXA(0) = X + \overline{X}Y = X + Y \quad MUXB(0) = X + \overline{X}\overline{Y} = X + \overline{Y}$$

$$MUXA(1) = 0$$

$$MUXB(1) = W$$

$$MUXA(2) = 0$$

$$MUXB(2) = 0$$

$$MUXA(3) = \overline{V}$$

$$MUXB(3) = VX + \overline{V} = X + \overline{V}$$

由上列函数关系，可画出图 8.25 (b)所示的控制器电路。(注：图中未画状态译码电路。)

由此例看到，采用 MUX 来设计控制器比用门电路逻辑设计要简单，而且有规则，控制关系明确。所需的 MUX 数量与触发器数相同，其数据输入的端子数与总状态数有关。事实上，这种方法仍采用 $NS = \sum PS \cdot C$ 公式，只是 MUX 的输入为 C 而已。

8.3.4 定序型控制器

定序型控制器需要较多数量的触发器。其基本思想是一对应一法，即触发器的数目代表了状态数，并依赖一组最新的代码实现状态转换。这种方法的优点是设计简单，而且也不需要状态译码。当改变控制算法时，适当改变序列发生器即可，不必推倒一切而重新设计控制器。下面通过实例来说明。

【例 8.9】有一个数字比较系统，它能连续对两个二进制数据进行比较，操作过程如下：先将两个数写入寄存器 R_A 和 R_B ，然后进行比较，最后将大数移入寄存器 R_A 中。其方框图和 ASM 流程图见图 8.26 所示。其中 X 为输入信号， LDR_A ， LDR_B 为写入控制信号，CAP 是三态门使能控制信号， $A > B$ 是比较器输出信号。

请设计定序型控制器。假设状态变化发生在 T_1 时序，写入寄存器操作发生在 T_2 时序，状态周期 $T = T_1 + T_2$ 。

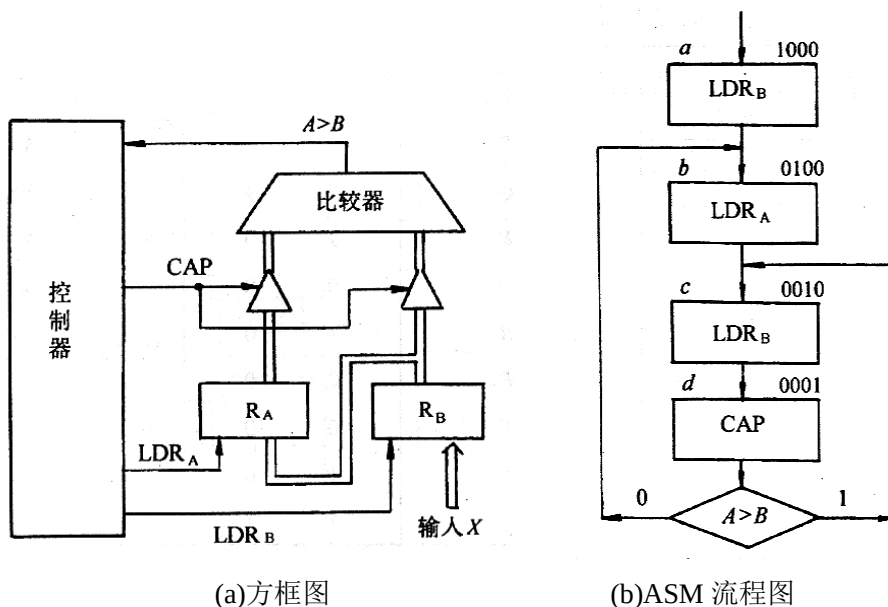


图 8.26 数字比较系统方框图及 ASM 流程图

解 定序型控制器采用“一对一法”进行设计，即一个状态使用一个 D 触发器。首先对 ASM 流程图进行编码(写于右上角)。表 8.6 列出了状态转移真值表。

表 8.6 状态转移真值表

PS(现态)				NS(次态)				转换条件 C
Q_a	Q_b	Q_c	Q_d	$Q_a(D)$	$Q_b(D)$	$Q_c(D)$	$Q_d(D)$	
1	0	0	0	0	1	0	0	初始化置"1" Q_a
0	1	0	0	0	0	1	0	-
0	0	1	0	0	0	0	1	-
0	0	0	1	0	0	1	0	$A > B$
				0	1	0	0	$\overline{A > B}$

由于状态与触发器是一对一的，所以各触发器的次态激励方程可按 $NS = \Sigma PS \cdot C$ 公式写出：

$$\begin{aligned}
 Q_a(D) &= 0 \\
 Q_b(D) &= Q_a + \overline{(A > B)} \cdot Q_d \\
 Q_c(D) &= Q_b + (A > B) \cdot Q_d \\
 Q_d(D) &= Q_c
 \end{aligned}$$

控制信号逻辑表达式如下：

$$\begin{aligned}
 LDR_B &= (Q_a + Q_c) \cdot T_2 \\
 LDR_A &= Q_b \cdot T_2 \\
 CAP &= Q_d
 \end{aligned}$$

图 8.27 画出了定序型控制器的逻辑电路图。为了直观，激励方程表达式的实现直

接采用了与或门的形式。 LDR_B 和 LDR_A 控制信号的表达式用与非门和与门实现, CAP 信号不需要译码, 直接从 Q_d 端引出。写入控制信号应和 T_2 相与, 用 T_1 上升沿改变触发器状态。

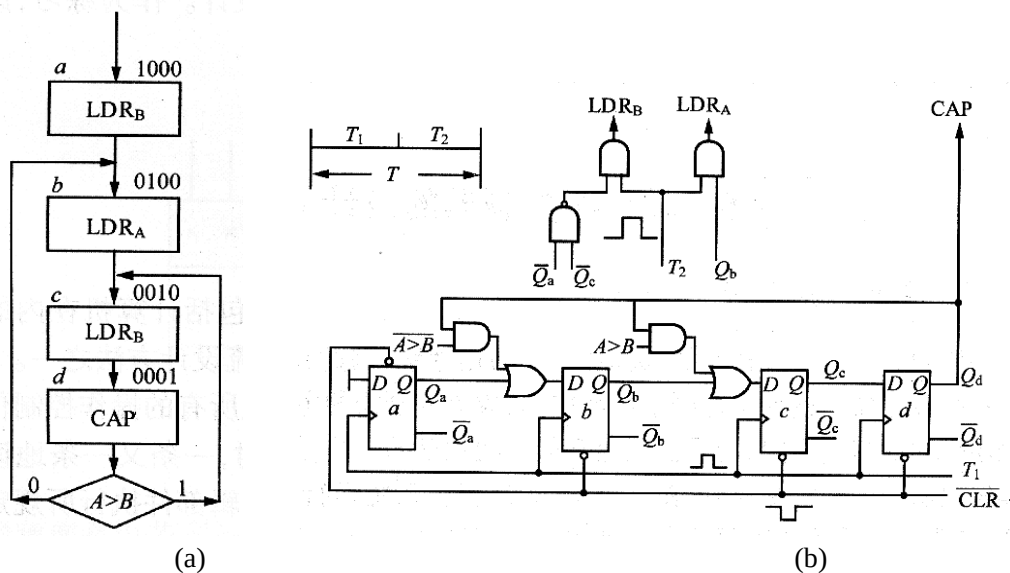
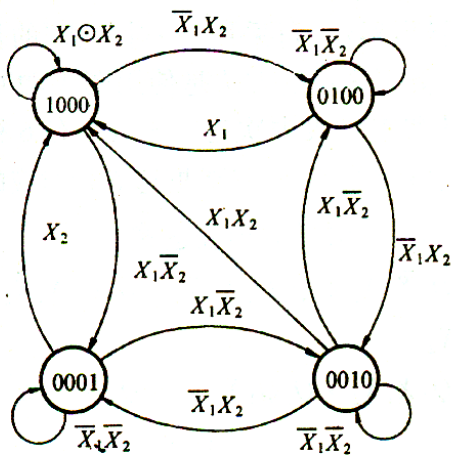


图 8.27 数字比较系统的定序型控制器

【例 8.10】图 8.28 所示某控制器的状态图及状态转移表, 请用数字器件设计一个定序型控制器(不考虑控制信号译码电路)。



状态转移表

PS(现态)				NS(次态)				转换 条件
Q_A	Q_B	Q_C	Q_D	Q_A^*	Q_B^*	Q_C^*	Q_D^*	
1	0	0	0	1	0	0	0	$X_1 \odot X_2$
				0	1	0	0	$\bar{X}_1 X_2$
				0	0	0	1	$X_1 \bar{X}_2$
0	1	0	0	1	0	0	0	X_1
				0	1	0	0	$\bar{X}_1 \bar{X}_2$
				0	0	1	0	$\bar{X}_1 X_2$
0	0	1	0	1	0	0	0	$X_1 X_2$
				0	1	0	0	$X_1 \bar{X}_2$
				0	0	1	0	$\bar{X}_1 \bar{X}_2$
				0	0	0	1	$\bar{X}_1 X_2$
0	0	0	1	1	0	0	0	X_2
				0	0	1	0	$X_1 \bar{X}_2$
				0	0	0	1	$\bar{X}_1 \bar{X}_2$

图 8.28 一对一法表示的状态图和状态表

解 根据状态转移表, 利用公式 $NS = \sum PS \cdot C$, 可以直接写出 Q_A 、 Q_B 、 Q_C 、 Q_D 四个触发器的次态激励方程表达式:

$$Q_A^* = Q_A(X_1 \odot X_2) + Q_B X_1 + Q_C X_1 X_2 + Q_D X_2$$

$$Q_B^* = Q_A \bar{X}_1 X_2 + Q_B X_1 + Q_C \bar{X}_1 \bar{X}_2$$

$$Q_C^* = Q_B \bar{X}_1 X_2 + Q_C \bar{X}_1 \bar{X}_2 + Q_D \bar{X}_1 X_2$$

$$Q_D^* = Q_C \bar{X}_1 X_2 + Q_D \bar{X}_1 \bar{X}_2 + Q_A X_1 \bar{X}_2$$

图 8.29 画出了“一对一法”实现的四状态时序机的逻辑图。

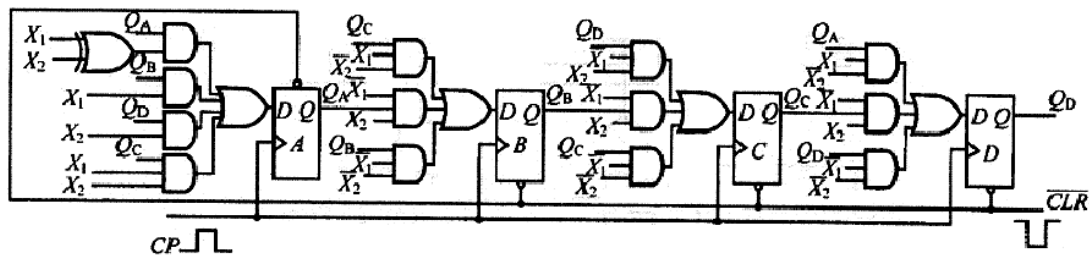


图 8.29 “一对一法”实现的四状态时序机逻辑图

这些方程采用与-或表达式, 它适合于采用 ISP(In-System Programming, 在系统可编程)器件的 GLB(Generic Logic Block, 通用逻辑块)模块实现。采用可编程器件, 还可以直接采用状态图和 Verilog 硬件描述语言等来进行设计。作为练习, 请读者自行完成。

8.4 微码控制器

8.4.1 微码控制的基本原理

微程序控制方法由于规整性好，灵活方便，通用性强，因此在包括计算机在内的各种复杂数字系统控制器的设计中得到了广泛应用，成为控制器的主流设计方法之一。

微程序控制的基本思想，就是仿照通常的解题程序的方法，把所有的操作控制信号汇集一起编码成所谓的**微指令**，存放在一个 EPROM 里。系统运行时，一条又一条地读出这些微指令，从而产生系统所需要的各种操作控制信号，以控制各逻辑部件执行所规定的操作。

一个数字系统基本上可以划分成**控制部件**和**执行部件**两大部分，如图 8.30 所示。

控制器就是控制部件。而 ALU、寄存器组、存储器 RAM 等，相对控制器来讲，就是执行部件。为使数字系统的结构尽量简单，一般在小型系统中数据通路多采用单一总线结构。在较大系统中可采用双总线或三总线结构。所有的控制信号由控制器产生，在它们的协调配合下，数据流通过 BUS 总线在各执行子系统之间进行流动，如图 8.33 所示。控制部件与执行部件可以通过控制线进行联系。控制部件通过控制线向执行部件发出各种控制命令，我们把这种控制命令称为**微命令**，而执行部件接受微命令所执行的操作叫作**微操作**。

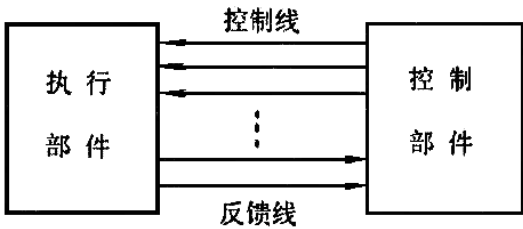


图 8.30 数字系统两大部分

控制部件与执行部件之间的另一联系是反馈信息。例如由于运算处理中正在处理的数据常因其结果特征(正、负、进位、溢出等)而影响下一个操作的执行，因此就需要规定条件测试或状态测试。执行部件通过反馈线向控制部件反映当前操作的结果情况，以便使控制部件根据执行部件的“状态”标志下达新的微命令。

在系统的一个**基本周期**(又称**机器周期**，一般由几个时钟周期组成)中，一组实现一定操作功能的微命令的组合，构成一条微指令。

这里要强调两点：第一，一条微指令的有效持续时间为一个系统基本周期，它表示从 ROM 中读出微码指令与执行这条微码指令的时间总和。当从 ROM 中读出下一条微指令后，当前的这条微指令即失效。第二，一条微指令中包含若干个微命令，它们分头并行地控制执行部件进行相应的微操作。

微指令除给出微命令信息外，还应给出测试判别信息。一旦出现此信息，执行这条微指令时要对系统的有关“状态标志”进行测试，从而实现控制算法流程图的条件分支。微指令中还包含一个下址字段，该字段将指明 ROM 中下一条微指令的地址。

图 8.31 示出了微指令的典型结构，长条框内的符号×表示一个二进制位(bit)。其中微命令字段用于操作控制；×编码为 1 时表示有微命令，×编码为 0 时表示无微命

令。测试判别字段和下址字段一起实现顺序控制：当测试判别字段无效时(×编码为 0)，下址字段信息即是下条微指令的地址；当测试判别字段有效时(×编码为 1，可以有多个测试)，根据反馈线来的“状态”信息对下址字段信息进行修改，修改后的地址即为下条微指令的地址。

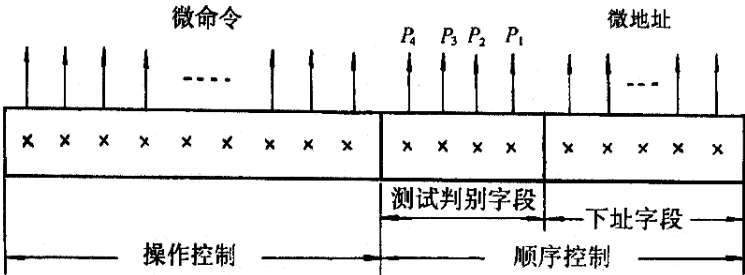


图 8.31 微指令的典型结构

微程序是由若干条微码指令组成的序列。在计算机中，一条机器指令的功能可由若干条微指令组成的序列来解释和执行，因此机器执行一条指令的过程，也就是执行一个相应的微程序的过程。就一般数字系统而言，微程序实质上就是将控制算法流程图用 EPROM 等来实现。

微程序概念的引入使计算机等大型数字系统控制器的设计发生了革命性的变化。因为微码程序技术可代替硬布线的控制技术，即由门电路和触发器等组成的硬件网络可被存有控制代码的 EPROM 存储器等所取代。

8.4.2 微程序控制器的一般结构

微程序控制器的结构与微码指令的格式密切相关。

我们画出与图 8.30 微指令相对应的微程序控制器结构框图，如图 8.32 所示。它主要由控制存储器、微地址寄存器、微码命令寄存器和地址转移逻辑等部分组成。微地址寄存器和微命令寄存器两者的总长度即为一条微指令的长度，所以两者合在一起称为微指令寄存器。为便于说明形成下一条微指令的地址，我们特意将微指令的下址字段独立出来，称为微地址寄存器。

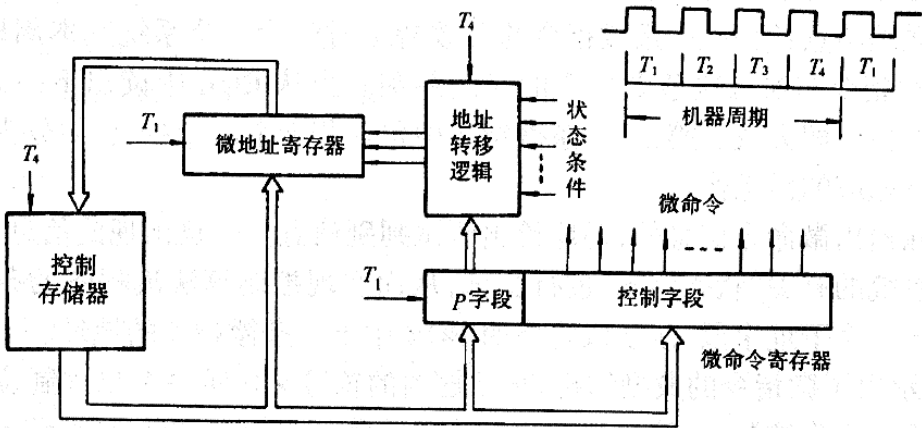


图 8.32 微程序控制器结构框图

1. 控制存储器

控制存储器是一个 ROM，其中存放微码程序，也就是全部微码指令。ROM 的容量视微码指令总数的多少而定。假如有 128 条微指令，则微地址寄存器长度为 7 位；假如有 64 条微指令，则微地址寄存器 6 位就够了。ROM 的字长取决于微指令长度，如果微指令为 32 位，则 ROM 字长就是 32 位。ROM 可使用 EPROM、E²PROM、EAROM 等，以使用户自己可以写入微码程序。如果发现错误可进行修正设计，只要擦除原来的内容而重新写入即可。可选用如 2702、2716 等型号 EPROM。

2. 微命令寄存器

用来暂存由控存读出的当前微码指令的控制信息，包括微命令字段和测试字段，可由 8D 寄存器组成。如 74LS273 八 D 触发器实现。

3. 微地址寄存器

存放下一条微指令的地址，它可由带 R 和 S 端的 D 触发器组成，其中 R 端用来清除寄存器，S 端用来强置 1。后者可用来修改寄存器的内容。也可以不用强置端修改，由写入端分时写入修改内容。可选用 74LS74 双 D 触发器。

4. 地址转移逻辑

微码指令由 ROM 读出后直接给出下一条微指令的地址，这个地址就放在微地址寄存器中。如果当前微指令的测试字段 P_i 标明要测试判别时，意味着微程序出现分支，即出现条件转移。此时，通过测试标志 P_i 和执行部件反馈的“状态条件”信号所组成的地址转移逻辑电路去修改微地址寄存器的内容（如 74LS74），并按修改好的微地址读出下一条微指令。注意：在微程序控制器中，地址转移逻辑部分仍然采用组合逻辑方法进行设计，不过方法很规整，且有专门的芯片可供选用。

图 8.32 中还标明了基本机器周期中的控制时序信号，它们用来读出 ROM、写入微指令寄存器和修改微地址寄存器中的内容。例如用上一机器周期的 T_4 时钟前沿修改微地址寄存器内容并读 ROM，经一段时间延迟后 ROM 中的信息被读出，用下一机器周期的 T_1 时钟前沿写入到微指令寄存器中。 T_2, T_3 时间用来控制处理； T_4 时再读下一条微指令。

8.4.3 微程序控制器的设计

微程序控制器设计的主要任务有两个：一是硬件设计，另一个是微程序编制。就硬件设计来说，存储器和寄存器可直接选用标准的 IC 芯片，需要设计的硬件部分仅是地址转移逻辑部分。

微程序设计，首先要确定微程序流程图，也就是控制算法流程图。但对微程序控制器来讲，状态化简和状态分配这两种传统技术已无关紧要。由于控制存储器有足够的容量，就无需过多考虑如何使状态最简。在控制存储器中从一个地址转移到另一个地址是随意的，因此所谓的“最佳状态分配”就失去了原有的意义。只要通过 E²PROM 中存储单元的信息就可修改程序，诸如增加新的控制状态，删除旧的状态等等。总之，以微指令结构为基础，就可以编制任何所要求的微程序，可长可短，灵活自如，且能实现多路并行分支。鉴于微程序控制的这些特点，微程序流程图与 ASM 流程图多少有些不同，下面我们将举例说明。

一旦微程序流程图确定下来，即可以使用微指令格式将微程序转化为二进制代

码。这项工作相对设计微程序流程图来讲，比较容易。

最后，将代码化的微程序写入 E²PROM 中，并将该芯片插入电路中，可进行调试或运行系统。

【例 8.11】按照图 8.33 数据通路，设计微指令格式，确定微程序控制器的基本方案。假定测试判别字段有 2 位，下址字段有 4 位。

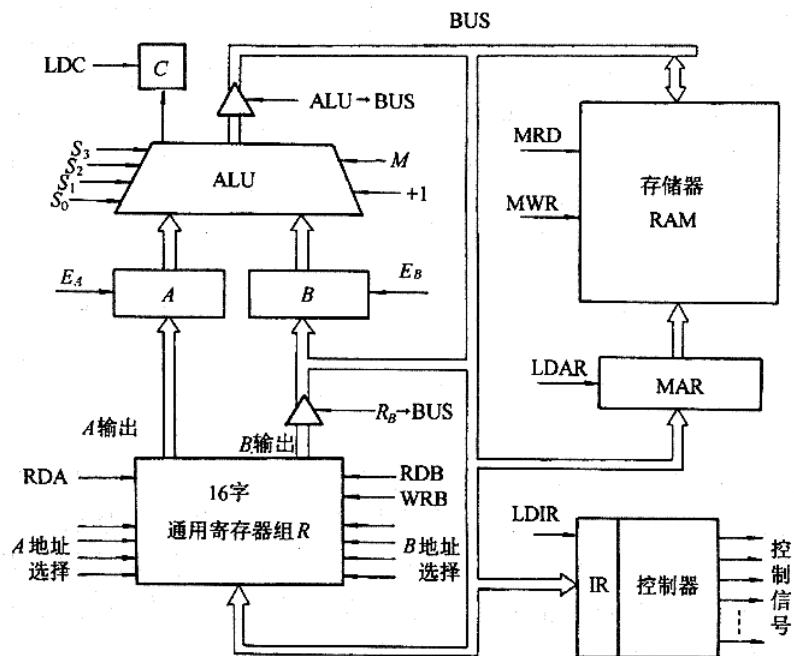


图 8.33 数据通路和控制器

注：其中使用的子系统有 16 字的通用寄存器组，暂存器 A 和 B，SN74181 型 ALU（参见附录），存储器 RAM（intel 6116 型）及其地址寄存器 MAR。除暂存器 A 以外，它们均通过三态门构成的总线 BUS 相互连接。由于采用了双输出口的通用寄存器组，且它的 A 输出口数据送到暂存器 A，因而可以保证从通用寄存器同时并行地读出两个数，其中一个放在暂存器 A，另一个通过三态门控制送到暂存器 B。寄存器组的 A 输出总线实质上是在器件内部通过三态门来选择 16 个寄存器之一进行输出的。

解 在图 8.33 所示的数据通路中，所有的控制信号均标注在逻辑子系统的侧面，共计有 26 个，因此微命令字段长度为 26 位，加上测试判别字段和微地址字段的 6 位，微指令长度共计 32 位。微指令格式如图 8.34 所示。

根据微指令格式，可确定 EPROM 的容量为 16 个存储单元，字长为 32 位。微地址寄存器为 4 位，微命令寄存器为 28 位。

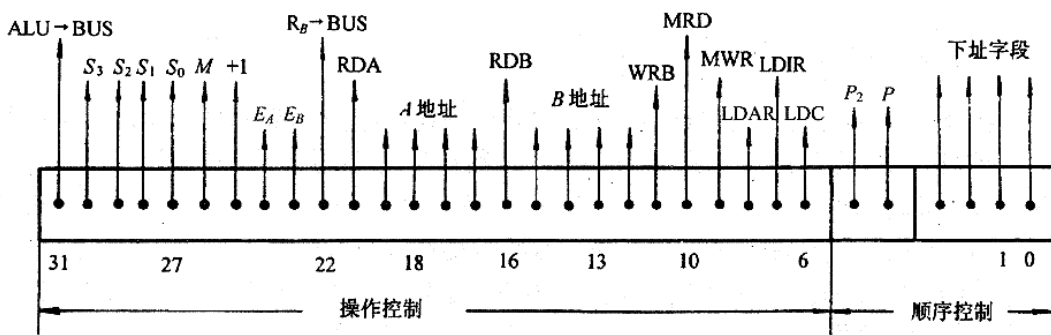
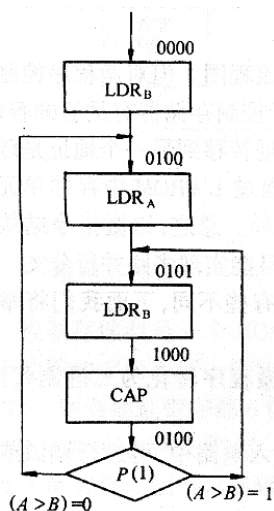


图 8.34 对应数据通路的微指令格式

【例 8.12】数字比较系统硬件方框图及 ASM 流程图如图 8.26 所示，请设计其微程序控制器。

解 采用微程序控制方式时，图 8.26 (b)所示的 ASM 流程图很容易变成微程序流程图，如图 8.35 (a)所示。由于采用控制存储器(E^2 PROM)，流程图中无需写状态名称，每一个状态框变成一条微指令。每条微指令的右上角须写上它在 E^2 PROM 中的微地址。本题中只有四条微指令，令其地址为 0000, 0100, 0101, 1000，它们是任意安排的用 4 位二进制表示的微地址。按 4 位地址推算， E^2 PROM 的存储容量为 $2^4=16$ 个单元，故最多存放 16 条微指令。

其次，第 4 条微指令执行后微程序流程图出现分支转移，故判别测试框(它依附于第 4 条微指令)中还出判别测试标志 $P(1)$ ，根据转移条件($A>B$)来决定下条微指令是执行第 2 条微指令(微地址 0100)，还是第 3 条微指令(微地址 0101)。这两条微指令地址的差别仅在于最低位上是 1 还是 0。如果转移条件($A>B$)=1，则执行第 3 条微指令，如果($A>B$)=0，则执行第 2 条微指令，为此第 4 条微指令的下地址安排为 0100。注意，微程序不出现分支时，下个微地址可任意安排(在 E^2 PROM 存储容量的范围内)，当微程序出现分支时，下个微地址的安排要考虑修改的方便性。不过 E^2 PROM 的容量很大，所以灵活性也很大。



(a)微程序流程图

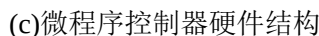
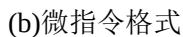


图 8.35 微程序控制器

其中微地址寄存器采用带有强置端的双 D 触发器 74LS74, 通过 μA_3 触发器 S_D 端置“1”的方法将其修改为 1, 其微地址转移逻辑表达式如下:

$$\mu A_3 = P(1) \cdot (A > B) \cdot T_3$$

当判别测试标志 $P(1)=1$ ，比较器反馈信号 $(A>B)=1$ ，且在微指令工作周期的 T_3 时刻，将 μA_3 触发器置“1”，而其他 3 个触发器不做修改，即原来的内容保持不变。

控制存储器 E^2PROM 中存放由 4 条微指令组成的微程序, 它们是按微指令格式转换的二进制代码, 如表 8.7 所示。

表 8.7 微程序代码

当前微地址	微指令二进制代码		
	微命令	判别	下地址
0 0 0 0	1 0 0	0	0 1 0 0
...	...	...	...
0 1 0 0	0 1 0	0	0 1 0 1
0 1 0 1	1 0 0	0	1 0 0 0
...	...	...	...
1 0 0 0	0 0 1	1	0 1 0 0
...	...	...	...

【例 8.13】采用例 8.11 的数据通路和图 8.34 所示的微指令格式，编制如下四种基本操作的微程序流程图，并完成微地址转移逻辑的设计。

- 00 $RAM \rightarrow R_j(2)$ (从 RAM 中取数至 R2)
 01 $R_i(3) \text{ 加 } R_j(4) \rightarrow R_j(4)$ (R3, R4 中二数算术加)
 10 $R_i(6) \rightarrow RAM$ (R6 中的数存入 RAM)
 11 $R_i(7) \oplus R_j(8) \rightarrow R_j(8)$ (R7, R8 中二数逻辑异)

假设条件：(1)四种基本操作分别用 4 个状态的编码(00, 01, 10, 11)来表示，该编码值由 IR0 和 IR1 两个触发器组成的计数器提供。对应计数器的一种状态，则执行相应的一种基本操作；(2)每当一种基本操作结束时，计数器进行加数而改变状态。但操作开始前计数器的初始状态是随机的，这意味着四种操作执行的顺序可认为是随意的，但时间上必须是连续的；(3)通用寄存器的选择也是随意的；(4)向 RAM 中取数、存数的地址均由暂存器 B 给出。

解 为了突出微程序控制的主要概念，从教学目的出发，做了上述假设条件，但控制方法符合一般数字系统的实际情况。对应四种基本操作，我们需要设计四个微程序。由于四种基本操作执行的先后顺序是随意的，因此在具体执行某种操作前，需要对反映操作特征的编码状态进行测试和判别，以实现微程序流程图的分支。这项任务由 P1 测试来完成，测试的状态条件是 IR0, IR1 的内容。其次，就四个基本操作而言，执行过程中不需要再进行测试，就是说，这四个微程序不再出现分支情况。由此可按数据通路画出微程序流程图如图 8.36 所示。

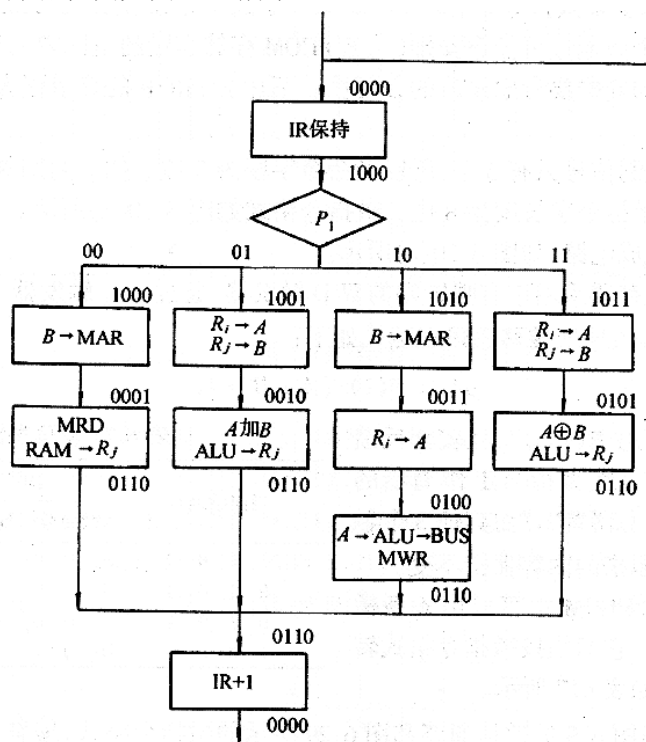


图 8.36 微程序流程图

图 8.36 中，每一个方框表示一条微指令。右上角的二进制码表示当前微指令的地址；右下角的数字表示下条微指令的地址(在微程序无分支情况下)或待修改的地址(在微程序有分支情况下)。最上面的第一条微指令的当前地址为 0000，它是所有微程序的入口地址，这可以通过系统启动时将微地址寄存器清零来给出。第一条微指令的下地址 1000 是一个待修改的微地址，在 P1 测试时，根据 IR1，IR2 的状态修改微地址寄存器的最后两位触发器，从而得出 1000，1001，1010，1011 四个微地址，实现了四个微程序的多路并行分支。在微程序不分支的情况下，下地址可以按 EPROM 的设计容量随意填写(本例中仅有 16 个单元)，但是每条微指令的微地址只能有一个，不允许两条功能不同的微指令使用同一个微地址号。最下面的一条微指令是一条公用微指令，用来进行计数器修改加 1(用 LDIR 微命令来实现)。由于这条微指令的下地址为 0000，因而又返回到第一条微指令，从而实现了四个微程序的连续运行。

每条微指令的框内表示数据通路的选择操作，它们是用微指令的操作控制字段发出的微命令来实现的。从时间上讲，每条微指令的持续时间均为一个机器周期(本例中有 4 个时钟周期组成)。每一条微指令只完成一个机器周期时间所允许的微操作。而几条微指令组合在一起可完成课题任务所要求的基本操作。

微地址转移逻辑用两个三输入与非门实现，其逻辑表达式如下：

$$\mu A_3 = \overline{P_1 \cdot IR_0 \cdot T_4}$$

$$\mu A_4 = \overline{P_1 \cdot IR_1 \cdot T_4}$$

其中 P_1 为微指令的测试判别标志， IR_0 ， IR_1 为计数器二触发器的状态值， T_4 表示时钟周期时间。这就是说，在一个机器周期的 T_4 前沿开始，通过三输入与非门将微地址寄存器的最后两位用 S 强置端输入，令其置“1”或保持不变，从而完成微地址的修改。

根据图 8.36 所示的微程序流程图和图 8.34 采用的微指令格式，可列出表 8.8 所示的用八进制数表示的微命令代码，其中微地址和 P 字段仍用二进制数表示(也可写成八进制)。

表 8.8 微程序代码

当前微地址	八进制表示的微命令	$P_2 P_1$	下一微地址
0 0 0 0	0 0 0 0 0 0 0 0 0	0 1	1 0 0* 0*
0 0 0 1	0 0 0 0 0 0 2 6 0	0 0	0 1 1 0
0 0 1 0	3 1 0 0 0 0 4 4 0	0 0	0 1 1 0
0 0 1 1	0 0 1 1 3 0 0 0 0	0 0	0 1 0 0
0 1 0 0	3 7 4 0 0 0 0 1 0	0 0	0 1 1 0
0 1 0 1	2 6 5 0 0 1 0 4 0	0 0	0 1 1 0
0 1 1 0	0 0 0 0 0 0 0 0 2	0 0	0 0 0 0
0 1 1 1	...	...	...
1 0 0 0	3 2 4 0 0 0 0 0 4	0 0	0 0 0 1
1 0 0 1	0 0 1 7 1 6 4 0 0	0 0	0 0 1 0
1 0 1 0	3 2 4 0 0 0 0 0 4	0 0	0 0 1 1
1 0 1 1	0 0 1 7 3 7 0 0 0	0 0	0 1 0 1

通过以上两个设计例子我们看到，微程序控制的主要优点是规范和灵活，易于实现多路并行分支，易于系统扩展。系统越复杂，越宜适用微程序控制方法进行设计，因为只不过是微指令的重复应用而已。对 EPROM 来讲，只不过是增加了一些存储单元，而微指令寄存器并不改变。显然，对一些非常简单的系统来说，采用微程序控制方法从成本上来讲不一定合算。另外，从速度上讲，微程序控制器的工作速度不如硬布线逻辑控制器快。

总之，在设计数字系统的控制器时，需要从速度指标、价格成本、系统复杂性、设计方法规范性等诸多方面进行折衷考虑，应当追求良好的性能 / 价格比。在这种考虑下，可以决策什么场合采用微程序控制方法，什么场合采用硬布线控制方法。

8.5 算法与流水结构

算法实际是由许多子运算组成的，在各子运算之间有一个执行方法和次序问题，这就是算法结构问题。本节主要介绍常用的顺序算法结构、并行算法结构以及流水线操作算法结构。

8.5.1 顺序(或串行)算法结构

顺序算法结构是指在执行算法的整个过程中，同一时间只进行一种或一组相关的子运算。如图 8.36 所示的是顺序算法结构。图 8.37 (a)所示的是在每时间段中，仅有一个子运算操作，各子运算之间逐个按规定的次序进行。图 8.37 (b)所示的是在同一时间里，有时仅有一个子运算操作，但有时有一组子运算操作。同一组的各个操作之间相互关联，在它们完成后再进行新的一种或一组操作。这两种情况均称为顺序算法。

在顺序算法结构中，若输入要处理的数据是单个元素 D(若干位二进制数)，完成该数据的算法流程需经 L 个时间段，而每段的平均时间为 Δt ，则完成该数据运算所需的时间为

$$\tau = L \cdot \Delta t$$

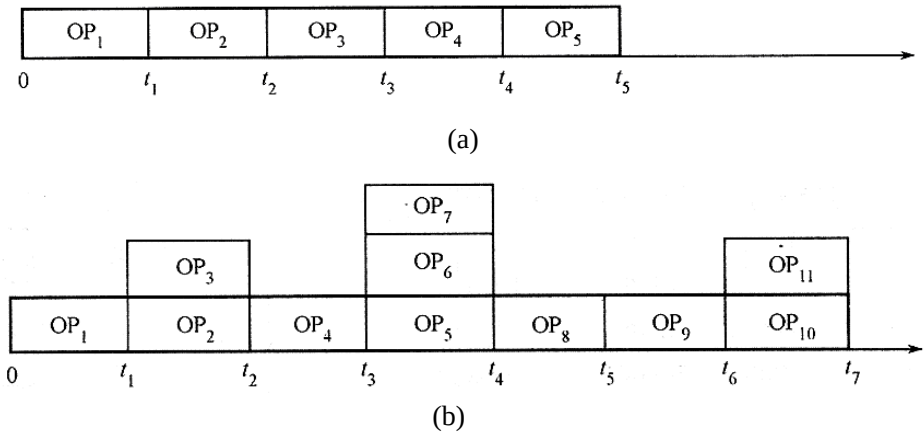


图 8.37 顺序算法结构

在很多数字系统中，待处理的数据是连续输入的数据流。数据流中每个元素均完成同样的运算，处理过程必须是对前一个数据元素计算完成之后，再进行后一个数据元素的计算处理。若含有 n 个元素的数据流输入时，其总的运算时间为

$$T_s = n\tau = n \cdot L \cdot \Delta t$$

顺序算法是最基本的算法结构，它最本质地反映了执行算法的过程结构。其优点是实现系统的硬件配置简单、成本低；缺点是工作速度不高。

8.5.2 并行算法结构

并行算法是指在同一时间段中，有多条路径在同时进行运算，在这些同时执行的子运算操作之间是相互独立的，如图 8.38 所示。

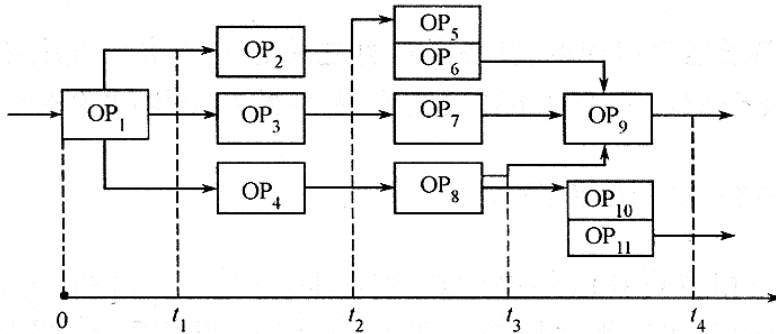


图 8.38 并行算法结构示意图

图中 OP_1 操作之后的 OP_2 、 OP_3 、 OP_4 是同时进行的，说明这三种操作是互相独立的，称之为并行算法结构或并行运算。

注意：① OP_1 到 OP_2 、 OP_3 、 OP_4 的转移决不是顺序算法中的条件转移，因为条件转移由判断条件决定，总只有一条后继操作路径。

② OP_2 、 OP_3 、 OP_4 也不是顺序算法中同时执行的一组操作，因它们之间互不关联。

③ OP_5 和 OP_6 、 OP_{10} 和 OP_{11} 为顺序运算路径中的一组相互有关的操作。

并行运算的优点是速度快，但是，它是以增加硬件结构成本为代价的。

在并行算法结构中，若待处理数据是单元素 Di (二进制若干位)，它完成运算的时间仍满足

$$\tau = L' \cdot \Delta t$$

其中 L' 是并行算法流程经过的运算段数。显然， L' 比同一系统的顺序算法流程的运算段数 L 要小得多，因此，提高了速度。

若含有 n 个元素的数据流输入时，并行结构算法总的运算时间为

$$T_p = n \cdot \tau = n \cdot L' \cdot \Delta t$$

8.5.3 流水线操作算法结构

流水线操作算法结构是针对连续输入数据流的系统而言的。它是把整个运算过程分解为若干个段，系统在同一时间可对先后输入的数据流元素进行不同段的同时运算，如图 8.38 所示。流水线操作算法与基本的顺序(串行)操作算法相比，具有如下特点：

(1) 在流水线中操作运算必须是连续任务，只有连续不断地提供任务才能充分发挥流水线的效率。

(2) 把一个运算(操作)分解为几个有联系的子运算(子操作), 每个子运算由一个专门的功能部件来实现。

(3) 在流水线的每一个功能部件后面都要有一个缓冲寄存器, 用于保存本段的执行结果。

(4) 流水线中各段的时间应尽量相等, 否则将引起“堵塞”或“断流”等现象。

(5) 流水线需要有“装入时间”和“排空时间”。只有流水线完全充满时, 整个流水线的效率才能得到充分发挥。

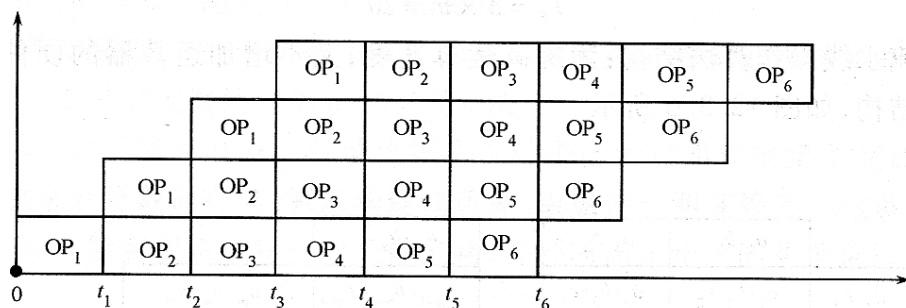


图 8.39 流水线操作算法结构

从图 8.39 所示中可以看到, 对于有 L 段运算的流水线结构, 可以同时为 L 个数据元素进行不同段的运算, 从而大大提高了运算速度。下面举例说明。

【例 8.14】 试用顺序操作算法和流水操作算法实现 $Z = \sqrt{AB+C}$ 运算, 并比较运算速度。其中 A 、 B 、 C 均为数据流, 长度为 m , 且均是 n 位。

解 根据题意, 给定的数据流共有 m 个元素, 它们是 (a_1, b_1, c_1) , (a_2, b_2, c_2) , $\dots$, (a_i, b_i, c_i) , (a_m, b_m, c_m) 。

其中

$$a_i = a_i a_i^2 \cdots a_i^n$$

$$b_i = b_i b_i^2 \cdots b_i^n$$

$$c_i = c_i c_i^2 \cdots c_i^n$$

从而可得到这一运算的算法流程图如图 8.40 所示。把系统运算分解为: “相乘”、“相加”、“开平方”共三个运算段。

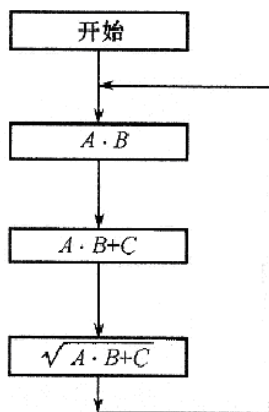


图 8.40 例 8.15 的算法流程图

(1)采用顺序算法结构: 在 Δt_1 、 Δt_2 和 Δt_3 的三段时间内分别完成 $a_i \times b_i$ 、 $a_i \times b_i + c_i$ 以及 $\sqrt{a_i \times b_i + c_i}$ 运算。为分析方便, 设 $\Delta t_1 = \Delta t_2 = \Delta t_3 = \Delta t$, 其顺序算法结构的时间关系图如图 8.41 所示。

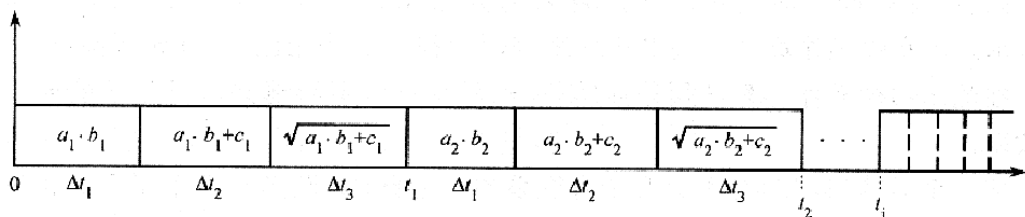


图 8.41 顺序算法结构时间关系

不难看出, 第 i 个数据元素在完成前两步运算之后, 只有求平方根电路在工作, 而乘法和加法电路均处在闲置的等待状态, 待求平方根运算完成后再接受数据流中的 $(i+1)$ 个数据元素, 这样完成整个运算的时间为

$$T_s = 3 \times m \times \Delta t$$

(2)采用流水线操作算法结构: 为提高运算速度, 又不增加运算器的硬件成本, 改用流水线操作算法结构, 如图 8.42 所示。

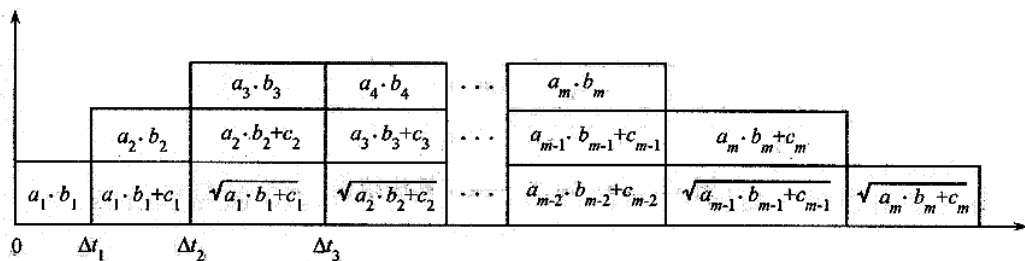


图 8.42 流水线操作算法结构时间关系

由图可以看出, 完成全部数据计算所需要的时间为

$$T = 3 \times \Delta t + (m-1) \times \Delta t$$

从上例可以推广到一般情况。若系统输入数据流的待处理数据元素为 m 个，每个元素运算共计 L 段，每段用时间 Δt ，则流水线操作结构共需要的运算时间为

$$T = L \times \Delta t + (m - 1) \times \Delta t$$

显然，流水线操作结构比顺序算法(或并行算法)所需要的运算时间 $T_s = m \cdot L \cdot \Delta t$ 少，特别是随着运算 L 和 m 的增加，流水线操作时间急剧减少，但控制运算又变得稍微复杂一些。

【例 8.15】某系统待处理数据元素为 100 个，每个元素需进行 16 段运算，且每段需运算时间为 $0.2\mu s$ ，试求顺序结构和流水线操作结构所需要的运算时间。

解 (1)顺序算法结构共需时间：

$$T_s = 100 \times 16 \times 0.2 = 320 (\mu s)。$$

(2)采用流水线操作结构时，则需要的时间为：

$$T = 16 \times 0.2 + (100 - 1) \times 0.2 = 23 (\mu s)$$

8.6 系统设计实现

系统设计出来以后，就进入设计方案的实现阶段，和设计阶段比起来重要性和难度都有所下降，这时主要的技术决策和细节设计都已经完成，主要是工作细节和工作量方面的问题。然而，在实现过程中可能出现一些新的问题，在设计阶段存在没有考虑到或错误估计的地方，就需要根据具体情况采取补救措施，甚至需要重新设计。

在设计方案中考虑的实现技术不同，实现所采取的方法肯定也不相同，如果系统最后被分解成基于现有芯片的设计，则实现阶段的主要任务就是设计 PCB 板，主要考虑电子器件如何布局、如何走线、如何保证速度、如何控制面积以及如何满足电磁兼容性等专业问题。如果在设计方案中涉及到通用处理器或者 DSP，那么实现阶段的任务还需要根据设计好的算法编制程序。如果设计方案中采用了可编程器件，那么在实现阶段需要把设计好的逻辑进行编译，转换成目标可编程器件的编程控制流，然后通过专用编程工具进行编程。如果走的是 ASIC 路线，则实现阶段的任务就是把设计交给集成电路生产厂，完成芯片的生产。如果设计最后包含了多块芯片，也存在 PCB 设计的问题。

一个复杂数字系统的设计，同时需要各种不同的实现技术，可能既需要通用处理器(8051, PIC, AVR, ARM, MIPS 类型)、DSP(ADI, TI 等公司产品)，又需要现有芯片(ADI, Fairchild, NXP, OnSemi, Renesas, ST, TI 等公司产品)，同时还需要设计专用芯片(TSMC, SMIC, GlobalFoundries)，因此，实现起来就需要多方面的能力或多方面的人才进行协作才能很好地实现整个系统。

可靠性设计：设计人员除了满足产品的功能外，会考虑怎么把产品做得可靠。

故障率设计：设计人员不只在满足功能设计和怎样通过可靠性测试，而是把设计重点转移到故障率的高低上。根据产品的定位，将产品故障率设计在 1~5%，或更低。除了冗余设计外，还要对每个元器件的故障率进行评估。

稳健设计：设计人员除了达到故障率设计外，还要对产品的客户满意度进行评估，并对客户滥用、误用进行设计，以防止损坏。这样的设计会考虑到产品使用的方方面

面。

在实现的过程中，调试过程必不可少，不管是 PCB 板、CPU 程序还是设计出来的 ASIC，都需要进行调试，一般先进行独立调试，然后再组装成一个完整的系统进行总体联调。如果发现新的问题，就要定位问题出现的原因，如果是实现阶段的问题，就必须修改实现，如果是设计或者需求的问题，则可以采取适当的补救措施，问题严重的则需要重新设计。

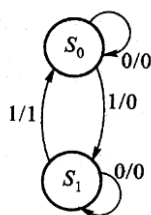
经过调试后，紧接着就是严格的测试，根据需求分析的结果形成测试方案，对系统的功能和技术参数进行详细的测试，保证产品的正确性。

测试工作往往是繁琐和复杂的工作，特别是测试一些复杂芯片时，没有适当的支持几乎是不可能的，测试成本也在不断增加。因此，目前在集成电路设计时比较注重可测性设计，就是在设计中就考虑对测试的支持，通过内部状态寄存器、内部测试电路以及扫描链等硬件电路支持边界扫描等测试技术来支持芯片测试，大大提高测试速度。

不同阶段的设计样品，经过软硬件调试、测试和改进，最终的样品经测试确保正确的系统就可以投入小批量生产了，通过小批量用户不断实际应用场景的检验，同时利用反馈信息进行应用可靠性和适应性等方面设计优化和改进。后面定型系统产品的大批量生产阶段的任务是如何在保证质量的前提下提高生产率，减少废品率，降低生产成本，这些主要涉及到一个生产过程控制的问题，存在一整套成熟的理论和经验，有兴趣的读者可以参阅相关的专业书籍。

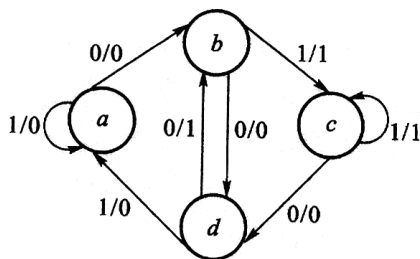
习题

[8.1] 题图 8.1 是两个状态的状态转换图，试根据此图建立 ASM 图。



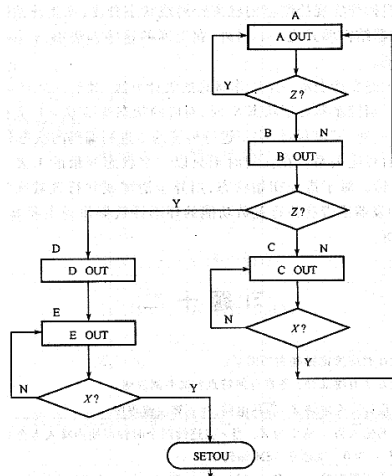
题图 8.1

[8.2] 把题图 8.2 四个状态的状态转换图试转换为 ASM 图。



题图 8.2

[8.3] 某系统的ASM图如题8.3图所示,试设计该图描述的控制器(用计数器为主设计)。



题图 8.3 某系统 ASM 图

[8.4] 设计具有 3 个触发器的计数器,按照以下序列计数: 000, 010, 111, 100, 110, 011, 001, 如此往复。验证实现是否为自启动的。

[8.5] 考虑设计 4 位的 BCD 码计数器,计数序列如下: 0000, 0001, 0010, 0011, 0100, 0101, 0110, 0111, 1000, 1001, 接下来回到 0000, 0001, 等等。

- 画出状态图和次态表。
- 使用 D 触发器实现计数器。
- 使计数器成为自启动的。

[8.6] 假设有 1 个米利机的实现含有 3 个触发器、2 个输入和 6 个异步输出。考虑这个状态机的完全状态图(即没有无关项)。回答下面的问题:

- 这个状态图中最小和最大可能的状态数量是多少?
- 起始于某一特定状态的转换数量最小和最大可能为多少?
- 结束于某一特定状态的转换数量最小和最大可能为多少?
- 输出上呈现的不同二进制模式的数量最小和最大为多少?

[8.7] 假设某个摩尔机具有 5 个触发器、3 个输入和 9 个输出。回答下面的问题:

- 这个状态图中最小和最大可能的状态数量是多少?
- 起始于某一特定状态的转换数量最小和最大可能为多少?
- 结束于某一特定状态的转换数量最小和最大可能为多少?
- 输出上呈现的不同二进制模式的数量最小和最大为多少?

[8.8] 有一个数字比较系统,它能对两个 8 位二进制数进行比较。其操作过程如下: 先将两个 8 位二进制数存入寄存器 A 和 B,然后进行比较,最后将大数移入寄存器 A 中。要求:

- 画出此系统方框图,并构造 ASM 流程图。
- 设计一个计数器型控制器。

[8.9] 根据题 8 的条件,设计一个 MUX 型控制器。

[8.10] 根据题 8 的条件, 设计一个定序型控制器。

[8.11] 根据题 8 的条件, 设计一个微程序型控制器。

[8.12] 设计十字路口交通管理信号灯系统, 它用于主干道与乡间公路交叉路口。“主干道绿灯、乡间道红灯”持续时间为 60 秒, “主干道红灯、乡间道绿灯”持续时间为 20 秒。在两个状态交换过程中出现的“主干道黄灯、乡间道红灯”和“主干道红灯、乡间道黄灯”的时间各为 4 秒。

[8.13] 设计一个彩灯控制器系统, 能让一排彩灯(12 只)自动改变显示花样。控制器有如下控制功能: ①彩灯规则变化, 变化节拍有 0.5s 和 0.25s 两种, 交替变化, 每种节拍可有 8 种花样, 各执行一个周期后轮换; ②彩灯变化方向有单向移动、双向移动。

[提示: 灯光移动用移位寄存器实现, 各种花样可以存于寄存器中, 使用时并行置入寄存器, 有的可以利用环形计数器实现。花样控制信号可用 4 位计数器控制, 1 位控制节拍, 另 3 位控制花样。]

[8.14] 要求设计一个洗衣机的控制器。当 1 枚硬币投入之后, 机器开始工作。接下来, 机器的工作步骤如下: 浸泡、洗衣、漂洗和旋转脱水。该机器具有“重复洗衣”开关, 即如果打开此开关, 洗衣机进行第二次洗衣和漂洗过程。洗衣机具有 1 个定时器, 假设每个步骤所需的时间相等。一旦硬币投入, 计时器开始计时, 当计时完成时产生 T 信号, 之后进行复位, 然后重新开始计时。如果洗衣机的顶盖在旋转脱水过程中被打开, 洗衣机就会停止旋转, 直到顶盖被关上。可以假设在顶盖被打开时, 计时器暂停计时。确定所需的输入和输出, 并且画出该有限状态机的状态图, 并用逻辑电路画出电路图。

[8.15] 将 P25 页图 8.26 的数字比较系统框图, 用具体型号的数字集成电路设计实现, 并画出详细的电路原理图。

[8.16] 将 P31 页图 8.33 的数据通路和控制器系统框图, 根据例 8.11 和例 8.13 等的实现结果, 用具体型号的数字集成电路设计实现, 并画出详细的电路原理图。

参考文献:

- [1] 白中英. 数字逻辑与数字系统(第三版). 北京: 科学出版社, 2002.04.
- [2] 张礼平. 数字逻辑教程. 上海: 华东理工大学出版社, 2006.02
- [3] 马忠义, 常蓬彬, 马浚. 数字逻辑与数字系统. 北京: 高等教育出版社. 2005.01
- [4] 沈建国, 雷剑虹. 数字逻辑与数字系统基础. 北京: 高等教育出版社. 2004.01
- [5] 刘真, 李宗伯, 文梅, 陆洪毅. 数字逻辑原理与工程设计. 北京: 高等教育出版社. 2003.11

第九章 验证和测试

9.1. 引言

在设计过程的早期考虑测试问题。在电路中进行某些小修改就容易证实设计有无缺陷。这一设计方法称为可测性设计（design for testability, DFT）。DFT 是整个设计过程中一个非常重要的组成部分，在设计流程中尽早考虑。如果你不测试它，它将不工作。一个 DFT 策略包括两部分：

- 1) 提供必要的电路以使测试过程加快并且全面。
- 2) 提供测试过程需要采用的测试激励矢量。为了降低成本，希望测试序列尽可能短，但仍能覆盖大部分可能存在的缺陷。

9.2. 测试过程

生产测试可以根据所希望的测试目的分成几类：

诊断测试用在芯片和板级调试期间，其目的是对于一个给定的失效部件识别和指出失效的部位。

功能测试确定一个制造出的元件是否能工作。这一问题比诊断测试简单，因为只需要回答是或否。由于每一个制造出来的芯片都要经过这一测试，因此它对成本有直接的影响，所以这一测试应当尽可能简单快速。

参数测试在各种工作条件（如温度和电源电压）下检查许多非离散参数，如噪声容限、传播延时和最大时钟频率。这就需要有与只需处理 0 和 1 信号的功能测试不同的测试设备。参数测试一般分为静态和动态测试。

一个典型的生产测试过程如下：首先把预先确定的测试矢量装入能够向被测器件（device under test, DUT）提供激励并采集相应响应的测试设备。测试矢量由测试程序来定义，它描述了所应用的波形、电平、时钟频率以及所期望得到的响应。需要用一个探针卡或 DUT 板把测试仪的输入和输出连到芯片或封装相应的引线上。

新部件被自动送入测试仪，测试仪执行测试程序，把一系列的输入矢量加到被测器件上，并把所得到的响应与所期望的响应进行比较。如果发现有差别，就给该部件标记上有缺陷，如打上一个红点，然后探针自动移到圆片的下一个芯片。在把圆片分割成单个芯片的划片过程中，打点的部件将被自动抛弃。在测试封装好的部件时，把已测部件从测试板上取下并根据测试结果分别放入良好和有缺陷的箱中。每个部件的测试过程只需要几秒的时间。减少一个芯片花费在测试仪上的时间是降低测试成本最有效的方法。

9.3.可测性设计

9.3.1 可测试性设计中的问题

在设计过程的早期考虑测试可能简化整个验证过程。考虑图 9.1 的组合电路。为了验证该电路的正确性，可把所有组合的输入矢量作为输入来观察相应地响应。对于 N 个输入的电路，要求有 2^N 个测试矢量。对于 $N=16$ ，就需要测试矢量。如果一个矢量加入和观察需要 $1\mu s$ ，测试这模块的全部测试就需要 $1s$ 。当考虑图 9.1 的时序电路时这种情况就变得更严重，因为该电路的输出不仅取决于所加的输入，还要取决于状态值。为了无遗漏地测试这一有限状态机 (Finite State Machine, FSM) 需要应用 2^{N+M} 个输入矢量。 M 是状态寄存器的数目。对于一个中等大小的状态机，例如 $M=10$ ，这意味着必须测试 10 亿个矢量的结果。如果仍用每个矢量需要 $1\mu s$ 的测试仪，就需要 16 分钟。

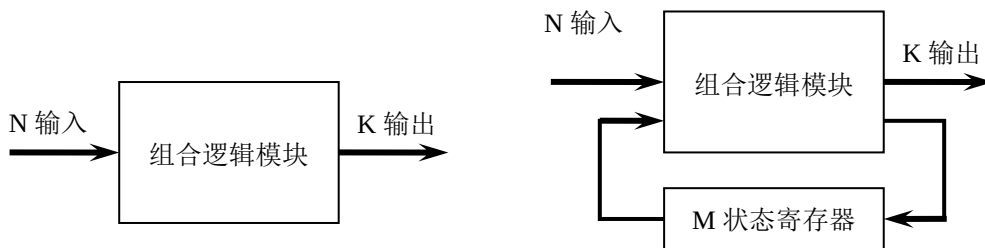


图 9.1 被测试的组合和时序器件

一种灵活的测试方法基于以下前提：

- 列举所有可能的输入矢量会含有相当多的冗余，即电路中的同一个缺陷为许多输入图形所覆盖，检测出这样一个缺陷只需要这些矢量中的一个，而其他矢量是多余的。
- 大量减少测试模式可放宽检测出所有缺陷的要求。因此一般的测试只要求 95-99% 的故障覆盖率。

通过消除冗余和适当降低故障覆盖率就有可能用有限的一组输入向量来测试大多数组合逻辑电路，然而这不解决时序电路的测试问题。为了测试一个状态机中一定的故障仅仅应用正确的输入激励是不够的。因为首先必须使这个被测部件处于所希望的状态。这需要应用一系列的输入。同时把电路响应传送到其中的输出上。测试一个 FSM 中的单个缺陷需要一系列的测试向量。解决这一问题的一种方法是在测试过程中把反馈回路断开，从而把时序电路变成组合电路。这是一种扫描测试 (scan-test) 方法。另一种方法是让电路自测试 (self-test)。电路自测试并不需要外部的向量并且可以以很高的速度进行。可测试性设计有两个重要特性：可控性和可观察性。

- 可控性表示只利用输入引线就可以使一个电路节点进入某一指定状态的难易程度。如果只用一个输入向量就可以把一个节点带到任何状态，那么容易控制。

一个具有低可控性的节点或电路需要一个很长的向量序列才能到预期的状态。可测性设计希望有高的可控性。

- 可观察性表示在输出引线上观察一个节点值的难易程度。对于一个可观察性高的节点，可以在输出引线上直接监测到节点的值。一个可观察性低的节点则需要多个周期才能使节点状态出现在输出口上。当电路的复杂性和输出引脚数目一定时，一个可测试电路应当具有较高的可观察性。

组合电路属于易观察和可控制的电路。时序电路的可测试设计方法划分成 3 类：专门测试、扫描测试和自测试。

9.3.2 专门测试

专门测试 (ad hoc test) 方法集合了一些可用来提高一个设计的可观察性和可控性的技术，它同应用类型相关。

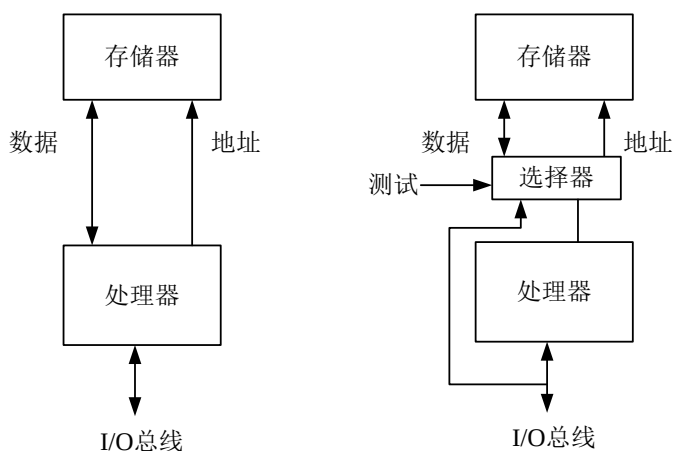


图 9.2 插入多路选择器来提高可测性

图 9.2 是简单处理器以及它的数据存储器。正常情况下存储器只有通过处理器才能访问，把一个数据值写入或读出一个存储器内容需要几个时钟周期。存储器的可控性和可观察性可以通过在数据和地址总线上增加多路开关而改善。在正常操作模式时，选择器使存储器通向处理器，而在测试期间的数据和地址端口则直接连至 I/O 引线，这样可以更为有效地对存储器进行测试。

在设计中引入一些改善电路可测试性的硬件是值得的。面积和性能上会有一些开销，但是可以改善设计可控性或可观察性。

可测试性设计意味着处理正常功能的 I/O 引线外还必须提供一些额外的 I/O 引线。图 9.2b 中的测试引线就是这样的功能。为了减少可能需要的额外压焊块的数目，可以采用在同一个压焊块上分路选择测试信号和功能信号的方法。如图 9.2b 中的 I/O 总线在正常工作期间作为数据总线，而在测试期间则用来提供测试图形和收集响应。

9.3.3 扫描测试

避免时序测试问题的一种方法是把所有的寄存器都变成可从外部转入和可读出的元件，这样被测电路就成了一个组合电路。为了控制一个节点，需要建立一个合适的向量，把它装入寄存器并传播通过逻辑。激励的结果传播到寄存器并被锁存，然后寄存器中的内容被传送到外部接口。可采用图 9.3 所示的串联扫描方法来减小电路开销。在这一方法中，寄存器修改为可以支持两种工作模式，在正常模式它们作为 N 位宽的钟控寄存器。在测试模式期间寄存器被链接在一起作为一个串联的移位寄存器。于是，测试过程按如下进行：

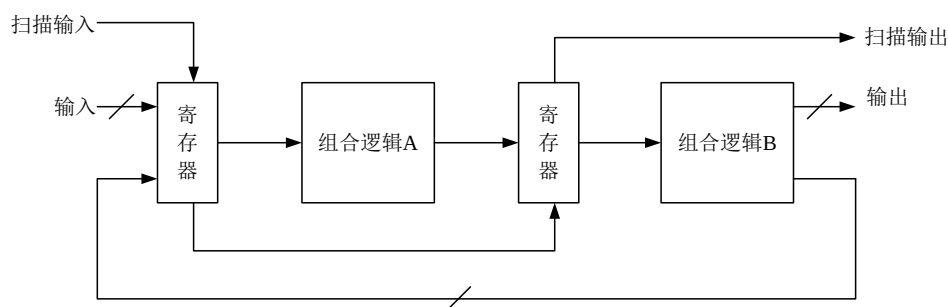


图 9.3 串联扫描方式

在测试时钟的控制下通过**扫描输入**（ScanIn）引线输入逻辑模块 A（和/或 B）的激励向量并将其移入寄存器。

激励被加到逻辑模块的输入并传播到它的输出。通过产生一个系统时钟事件把它的结果锁存到寄存器中。

寄存器中的结果通过**扫描输出**（ScanOut）引线送出电路并与期望的数据进行比较。此时一个新的激励向量可以被同时输入。

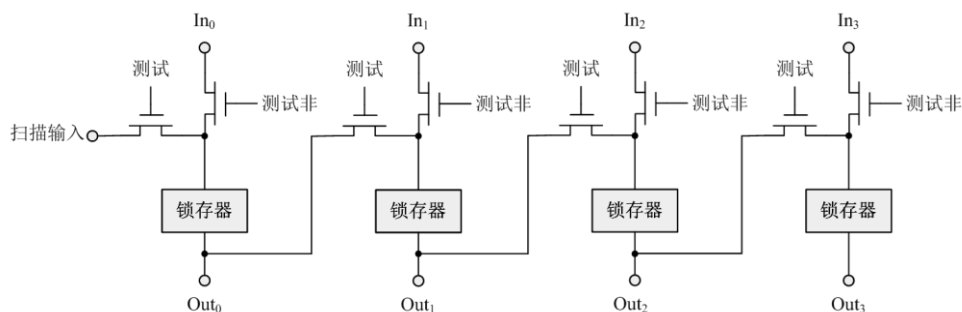


图 9.4 扫描链的 4 位寄存器

这个方法只产生最小的电路开销。扫描链的串联本质减少了布线数量，而且通常的寄存器很容易修改为支持的扫描技术。如图 9.4 所示，它显示了一个修改后具有扫描链的 4 位寄存器。惟一增加的是在输入端的一个额外的多路开关。当测试为低电平

时电路处于正常工作模式。把测试设定为高电平则选择扫描输入，并把寄存器链连接到扫描链上。寄存器的输出 Out 连接到扇出逻辑上，但同时还要增加逻辑，因为扫描输出引线连接到相邻寄存器的扫描输入。

图 9.5 为采用图 9.3 中电路的时序图，这里假设采用两相位时钟方法。对于 N 位寄存器的一个扫描链，测试（Test）信号首先提升产生 N 个时钟脉冲以装载寄存器。Test 信号下降时产生单个时钟脉冲，把正常电路工作状况下从组合逻辑得到的结果锁存到寄存器中。最后，有另外 N 个的脉冲（当 Test=1 时）把所得到的结果传送到输出端。再次注意扫描输出可以与下一个向量的输入同时进行。

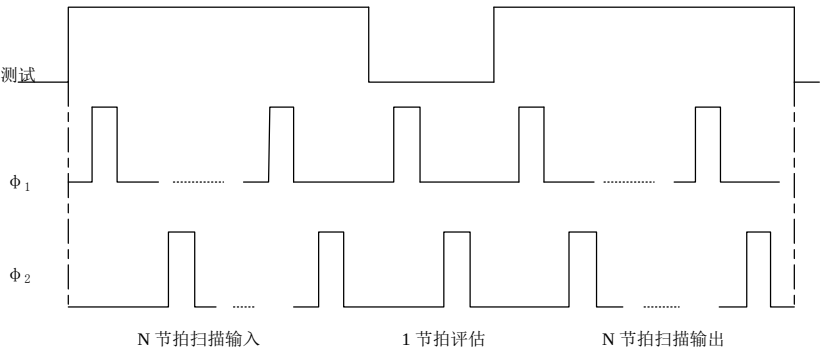


图 9.5 测试序列的时序图，N 表示测试链中寄存器数量。

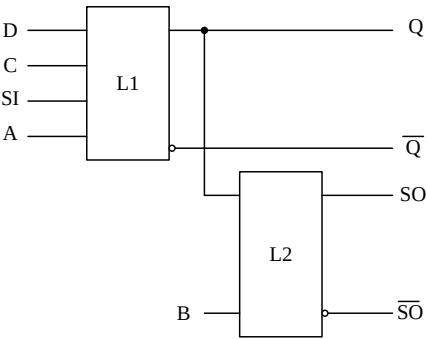


图 9.6 移位寄存器

可以设计出许多不同的串联扫描方法，如 IBM 公司设计的电平敏感扫描设计（level-sensitive scan design, LSSD）。LSSD 方法的基本功能块是移位寄存器（shift-register latch, SRL）如图 9.6 所示。它由两个锁存器 L1 和 L2 组成，后者只是用于测试目的。在电路正常工作时，信号 D、Q 和 C 分别用做锁存器的输入、输出和时钟。在这一模式中测试时钟 A 和 B 为低电平。在扫描模式中，SI 和 SO 分别作为扫描输入和扫描输出。这时候时钟 C 为低电平，而时钟 A 和 B 作为不重叠的两相位测试时钟。

LSSD 方法代表的不仅是一种测试策略，而且也是一种完整的时钟控制原理。当

严格遵循这一方法所内含的规则，就有可能在很大程度上使测试产生和时序验证自动化。这一方法的主要缺点是 SRL 锁存器的复杂性。

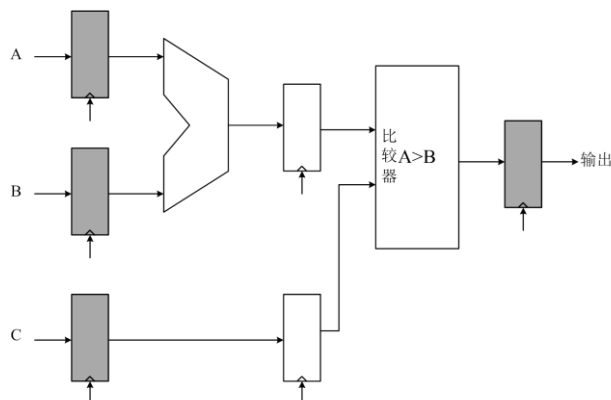


图 9.7 部分扫描的流水通道，图中灰色的寄存器包括扫描链

并不是需要使设计中所有的寄存器都是可扫描的。考虑如图 9.7 中的流水线数据通路。在这一设计中的流水线寄存器只是为了提高性能，因而在严格意义上并不增加电路的一个新状态。因此只需使输入和输出寄存器可扫描就可以了。在测试生成期间，可以把加法器和比较器一起看做是单个组合电路。惟一的区别是在进行测试时需要两个时钟周期把激励向量的响应结果传送到输出寄存器中。这一方法称为部分扫描，常用于当性能是主要关注对象的时候。

9.3.4 边界扫描设计

测试电路板能够提供丰富的测试点，因而使测试非常方便。穿孔安装方法使封装上的每一条引线可在该板的背面看到。对于测试，只要把板放在一组测试探针下（test bed）然后输入和观察感兴趣的信号就可以了。随着高级封装技术（如表面封装或芯片模块）的出现，情况发生了变化。由于探针点数目大大而引起设计的可控性和可观察性降低，可以通过把扫描测试方法延伸到部件级和板级来解决此问题。

边界扫描（boundary scan）是把一个板上个部件的输入-输出引线连接成一条串联的扫描链，并且此方法已经被标准化以确保在不同厂商之间的兼容性。从本质上来说，它。如图 9.8 所示。在正常工作时，边界扫描压焊块作为正常的输入-输出器件。在测试模式汇总，向量可以从这些压焊块处扫入扫出，从而在部件的边界上提供可控性和可观察性。测试操作的进行与上面介绍的过程类似。可以利用各种控制模块来测试各个部件以及板上的互连线。这一方法的开销是一些输入-输出压焊块以及一个附加的片上测试控制器。边界扫描链由边界扫描单元串行组成。边界扫描单元能够完成对电路节点的控制和观察功能。

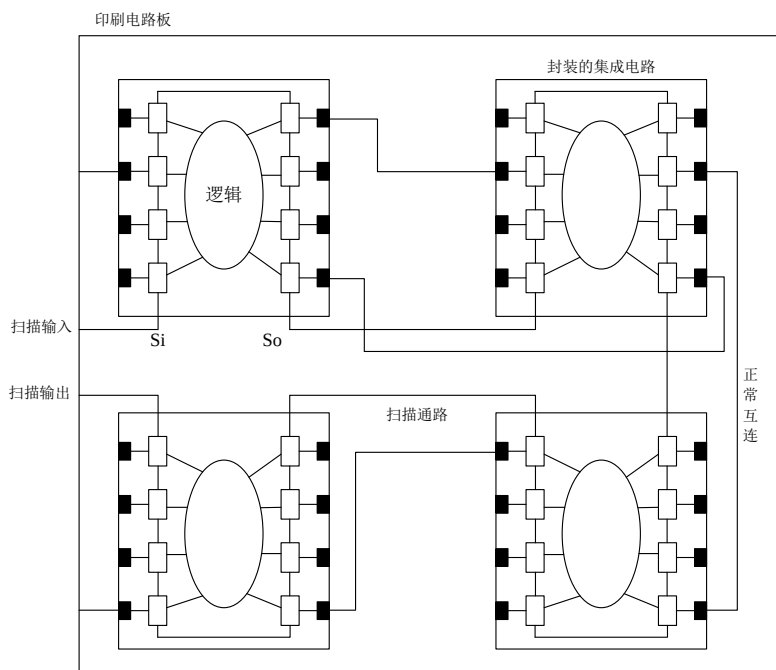


图 9.8 板级测试的边界扫描方法

9.3.5 内建自测试

可测性的另一个方法是让电路自己生成测试图形而不是要求应用外部的图形。更加吸引人的一种技术是电路自己能够决定它所得到的测试结果是否正确。根据被测电路的特点，采用这一方法也许需要增加额外的电路来产生和分析测试图形。但这部分电路的硬件中有些可能就是正常工作电路的一部分，因而已经存在，所以自测电路的面积可以很小。

一个内建自测试（Built-in self-test, BIST）设计的一般形式如图 9.9 所示。它包括向被测器件提供测试图形的方法以及把器件的响应与已知正确的序列进行比较的方法。

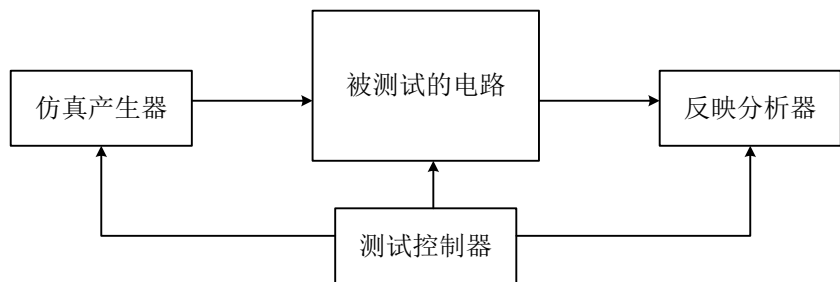


图 9.9 内建自测试通用形式示意图

有许多方法可以产生激励，用的最多的是穷尽法和随机法。在穷尽法中，测试长

度为 2^N ，其中 N 是电路输入的数目。穷尽测试意味着对于给定的所有可以得到的输入信号空间，所有可测的故障都会被检测到。一个 N 位的计数器就是穷尽图形发生器的一个很好的例子。对于 N 值较大的电路，通过整个输入空间的操作所需要的时间是无法接受的。一种替代方法是采用随机测试，这意味着随机选择 2^N 个可能输入图形中的一个子集。选择这一子集的原则是应当能得到合理的故障覆盖率。图 9.10 是一个伪随机图形发生器的例子，这是一个线性反馈移位寄存器（linear-feedback shift register, LFSR），由多个一位的寄存器串联构成。有些输出被异或（XOR）并反馈回移位寄存器的输入。一个 N 位的 LFSR 顺序通过 2^N-1 个状态，然后再重复这一序列，从而产生一个随机序列。把这个寄存器初始化为一定的种子值就会产生不同的伪随机序列。

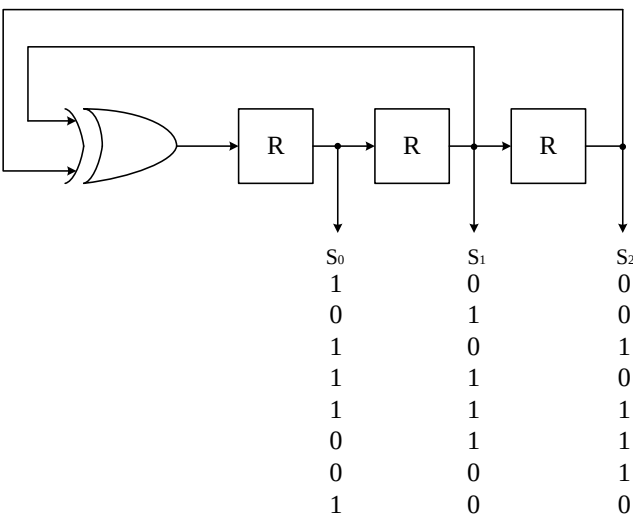


图 9.10 线性反馈移位寄存器（3 位）和它的产生序列

虽然可以把响应分析器实现为将所生成的响应与存放在片上存储器中的预期响应进行比较，但这一方法因需要过多的面积而不切实际。一个更经济有效的技术是在对它们进行比较之前把这些响应进行压缩。存放正确电路的压缩响应只需要很少数量的存储器，特别是在压缩率较高时尤为如此，因此响应分析器包括一个动态压缩被测电路输出的电路和一个比较器。被压缩的输出也常常称为该电路的签名，而整个方法称为签名分析。

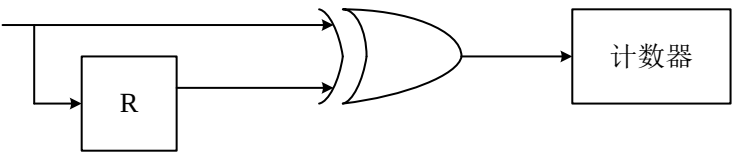


图 9.11 单比特流签名分析

图 9.11 是一个压缩一位信号流的签名分析器的例子，可以看到这一电路只是计数输入流中 0->1 和 1->0 的翻转数目。这一压缩并不能保证接收到的序列是正确的，也

就是说许多不同的序列可以有相同数目的翻转。

另一方法显示在图 9.12(a)中。它是一种改进的线性反馈移位寄存器。其优点是可以用同一硬件来完成图形生成和信号分析。每一个进来的数据被一个接一个地与 LFSR 的内容相比较(XOR)。在测试序列结束时，LFSR 中含有了这个数据序列的签名或特征字，可以用来与正确电路的特征字进行比较。这一电路不仅实现了一个随机图形发生器和签名分析器，而且还能根据控制信号 B0 和 B1 的值用来作为正常的寄存器或扫描寄存器（如图 9.12）。这一测试方法结合了所有不同的技术，称为内建逻辑块监测器(built-in logic block observation, BILBO)。图 9.12 是一个 BILBO 的典型应用。选择扫描时种子值被移入 BILBO 寄存器 A 而 BILBO 寄存器 B 被初始化。接着寄存器 A 和 B 分别工作在随机图形发生和签名分析模式。在测试序列结束时通过扫描模式从 B 中读出签名。

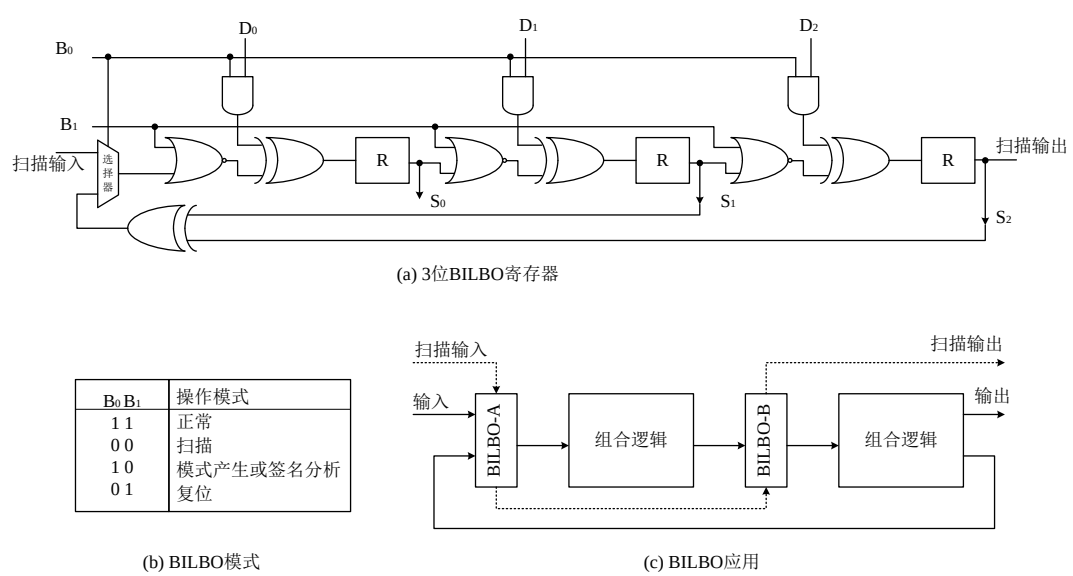


图 9.12 内建逻辑块监测器（BILBO）

自测试在测试规则结构（如存储器）时极为有用，保证一个存储器无缺陷并不容易。这一任务的复杂性在于把一个数据值读出或写入一个单元时，由于存在交叉耦合和其他寄生效应，这个值会受到存储在邻近单元中的值的影响，因此存储器测试包括以交叉变化的地址序列把许多不同的图形读出或写入存储器中。典型的图形可以是全 0 或全 1，或是 0 和 1 相间的棋盘图案。寻址方式可以是先写整个存储器，然后整个读出或者采用各种读写交替的序列。这一测试方法可以在集成电路内部建立，与一个存储器的尺寸相比它只需增加很小的开销，如图 9.13 所示。这一方法极大地改善了测试时间并减少了外部控制。随着集成部件复杂性的增加和嵌入式存储器的日益普遍，应用自测试技术变得更为重要。



图 9.13 存储器自测试

系统芯片（System-on-Chip, SoC）包含有微处理器(CPU)、数字信号处理器(DSP)、存储器、专用集成电路模块、可编程逻辑阵列、片上总线、以及片上互连网络等。这些模块中的每一个都有它自己专有的测试方法。因此要把它们组合成一种统一的策略就具挑战性。内建自测试应当是解决这一问题的有效方法。图 9.14 是一个基于 BIST 的结构化的系统芯片的测试方法。构成该系统的每一个模块通过一个包装(Wrapper)连接到片上网络。这个功能块与网络之间定制设计的接口，用以支持同步和通信这样的功能。这一包装可以扩展为包括一个测试支持模块。例如对于有扫描链的一个专用集成电路模块，测试支持模块提供了与扫描链之间的接口并为测试图形提供一个缓冲器。这一缓冲器可以通过系统总线直接读和写。同样，可以给一个存储模块配备一个图形发生器和签名分析器。系统中还需要一个总体测试控制器。多数 SoC 都有一个可编程处理器，可以在启动时对其他模块的测试和验证。测试图形和签名可以存放在处理器的存储器中，并在测试时间提供给被测试的模块。这一方法的优点是它利用了芯片上已有的资源。此外，BIST 自测试方法允许测试以实际的时钟速度运行，这就减少了测试时间。由于把外部测试仪的要求降低到了最小的程度，从而降低了它的成本。

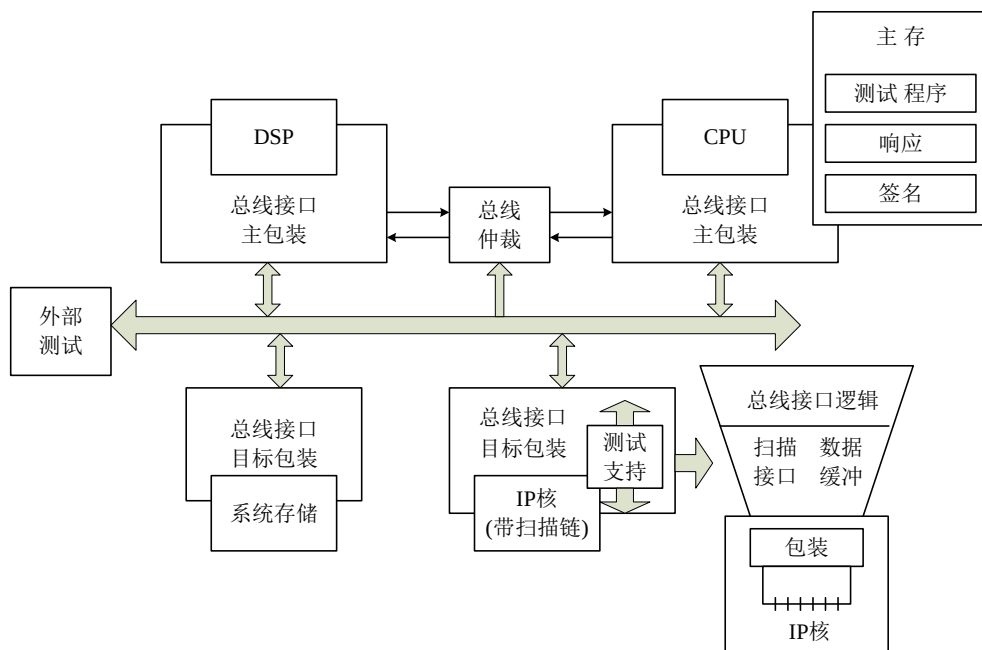


图 9.14 系统芯片测试方法

9.4. 测试图形的生成

应当使用什么样的图形才能得到一个好的故障覆盖率？如何自动生成测试图形 (automatic test-pattern generation, ATPG)。首先分析故障的概念。

9.4.1 故障模型

制造缺陷可以是各种各样的，但他们中最突出的是信号间的短路、与电源线的短路以及节点浮空。为了评估一种测试方法是否有效以及一个电路的好坏，必须把这些故障与电路模型联系起来，或者换句话说，要推导出一个故障模型。最普遍使用的模型称为固定型 (stuck-at) 故障模型。大多数测试工具只考虑短路至电源线，我们分别把短路至 GND 和 Vdd 称为固 0 (stuck-at-zero, sa0) 和固 1 (stuck-at-one, sa1) 故障。

可以证明 sa0-sa1 (固 0 固 1) 模型并不能覆盖集成电路中可能发生的所有故障，因此还应当考虑开路固定故障 (stuck-at-open) 和短路固定故障 (stuck-at-short)。然而加进这些故障会使测试图形的生成过程复杂化。而且这些故障中许多都可以用 sa0-sa1 模型来覆盖。为了说明这一点，考虑图 9.15 中的电阻负载 MOS 门。所有与电源间的短路都通过在节点 A, B, C, Z 和 X 处引入 sa0 和 sa1 故障来模拟。图中已标注了一些开路固定故障 (β) 和短路固定故障 (α , γ)。可以看到这些故障已经为各个节点上的 sa0 和 sa1 故障所覆盖。例如，故障 α 由 Asa1 覆盖， β 由 Asa0 或 Bsa0 覆盖，而 γ 相当于 Zsa1。

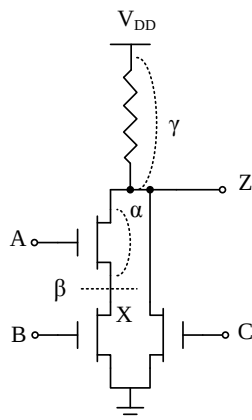


图 9.15 电阻负载 MOS 门，开路固定故障 (β) 和短路固定故障 (α , γ) 示意图

在 CMOS 电路中，短路和开路故障可形成一些未被 sa0-sa1 模型所覆盖的情形。考察图 9.16 中的两输入 NAND 门，发生了一个开路固定故障 α 。图中也显示了故障电路的真值表。对于组合 ($A=1$, $B=0$)，输出节点是浮空的。因而保持它原先的值，而正确的值应当是 1。这一故障可能被检测出来也可能不被检测出来，取决于如何设计测试激励向量。为了检测出这一故障，必须先后应用两个向量。第一个迫使输出为 0 (即 $A=1$ 且 $B=1$)，而第二个则为 $A=1$, $B=0$ 。短路固定故障在 CMOS 电路中会引起问题，因为在某些输入值时这一故障引起电源线和地线之间的直流 dc 电流，从而产生不确定

的输出电压。

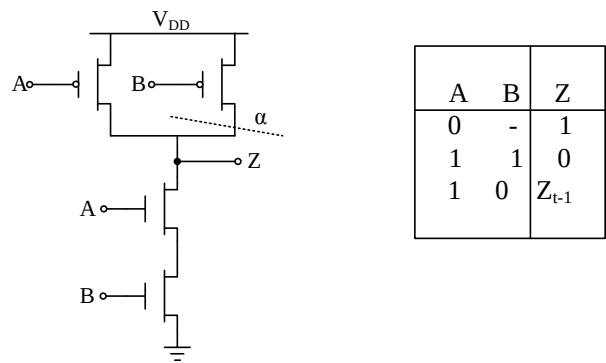


图 9.16 两输入互补 CMOS 与非门，真值表和开路故障 α 示意图

尽管 sa0-sa1 故障模型存在一些问题，但它容易使用而且覆盖故障空间相对较大，因此事实上它已成为一种标准模型。在采用时经常再辅之以其他技术，如功能测试、*IDDQ* 测试(即测试 CMOS 电路因短路引起的静态电流的改变)以及延时测试。没有一个单独的测试模型是安全可靠的，所以经常需要把几种测试方法结合起来使用。

测试图形自动生成的任务是确定最小的一组激励向量，它们能覆盖由所采用的故障模型定义的故障几种足够多的部分。一种可能的方法是从一组随机的测试图形集开始，进行故障模拟来决定有多少可能的故障被检测出。随后以所得到的结果为指导反复增减向量。另一种可能更令人感兴趣的方法是基于对一个布尔网络功能的了解推导出适合于给定故障的测试向量。为了说明这一概念，考虑图 9.17。我们的目标是确定一个能够在改电路输出端 Z 检查出节点 U 处发生 sa0 故障的输入激励。对这样一个激励的第一个要求是它应当迫使故障能够发生（可控性）。为此，要寻找一种输入图形，能够在正常条件下把 U 置于 1。这里惟一的选择是 $A=1$ 且 $B=1$ 。第二个要求是这一故障信号必须传送到输出节点 Z 以能被观察到。这一阶段称为**路径敏化**。为了使节点 U 处的任何变化都能传送到 Z，需要把节点 X 设为 1 而把节点 E 设为 0。现在可以把 Usa0(惟一)的测试向量组合起来，即 $A=B=C=D=1$ ， $E=0$ 。

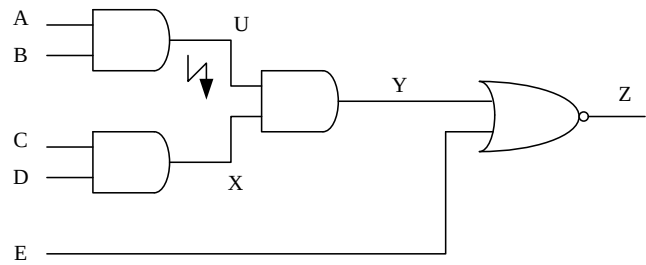


图 9.17 简单逻辑网络，节点 U 处发生 sa0 故障示意图

9.4.2. 故障模拟器

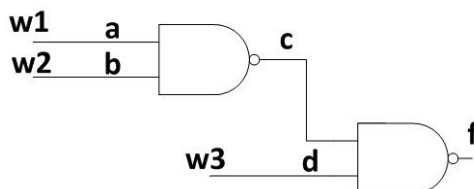
一个故障模拟器能够衡量测试程序的质量，确定它的故障覆盖率。故障覆盖率定义为由测试序列检测到的故障总数除以电路节点数目的两倍，因为每一个节点可以发生一个 sa0 和一个 sa1 故障。得到的故障覆盖数最多只会与所采用的故障模型一样。在 sa0-sa1 故障模型中某些搭接和短路故障并没有被覆盖，因而也不会出现在故障覆盖的统计中。

常用的故障模拟方法是并行故障模拟技术，这一技术把一个正确电路与许多故障电路同时模拟，每一个故障电路都注入了单个故障。将所得的结果进行比较。对于一个给定的测试向量集，如果它们的输出不同，那么就给这个故障标记为已被测出。大多数故障模拟器都运用许多技术，如首先选择被测出的机会较高的故障以加速模拟过程。硬件故障模拟器以并行处理器为基础，提供比纯软件模拟器速度快。

习题

[9.1] 请修改多路选择器构成的主从型正边沿触发器使之支持串联扫描。

[9.2] 假设题图 9.1 所示电路可能出现固定型 SA1 与 SA0 故障，需要写出寻找图中故障点的测试向量。在三输入的 8 种测试信号组合中，能测试该电路所有可能故障的最小测试集是什么？（注：最小测试集是指最少数量的输入信号 W1,W2,W3 的数字组合。）



题图 9.1 被测试电路

参考文献:

- [1] Jan M.Rabaey, Anantha Chandrakasan, Borivoje Nikolic, Digital Integrated Circuits, A Design Perspective, Second Edition. Pearson Education, Inc. 2004.
- [2] 周润德等译, 数字集成电路: 电路、系统与amp;设计 (外文书名: Digital Integrated Circuits A Design Perspective, Second Edition), 电子工业出版社, 2010 年 11 月.

第十章 微处理器设计

10.1. 微处理器结构

微处理器主要有两种结构，一种是冯诺依曼结构也称普林斯顿结构，一种是哈佛结构，两者的区别是前者将程序存储和数据存储放在同一物理存储空间，后者将程序存储和数据存储分别放在不同的物理存储空间。冯诺依曼结构具有更好的硬件效率，哈佛结构具有更好的灵活性和稳定性。本讲义主要以冯诺依曼模型展开讨论。

冯诺依曼计算机模型主要包括如图 10.1 所示的四个部分：输入、输出、存储器和微处理器。其中，微处理器可以被进一步分为两个部分：控制单元（*Control Unit*）和数据通路（*Datapath*）。图 10.2 从功能划分的角度给出了微处理器内部的控制单元和数据通路的示意性结构。

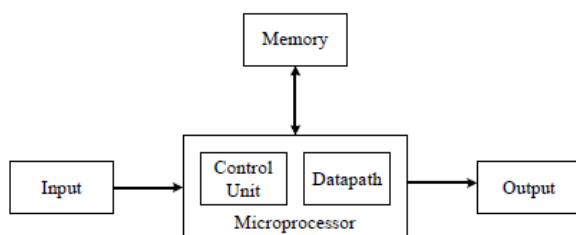


图 10.1 冯诺依曼计算机模型

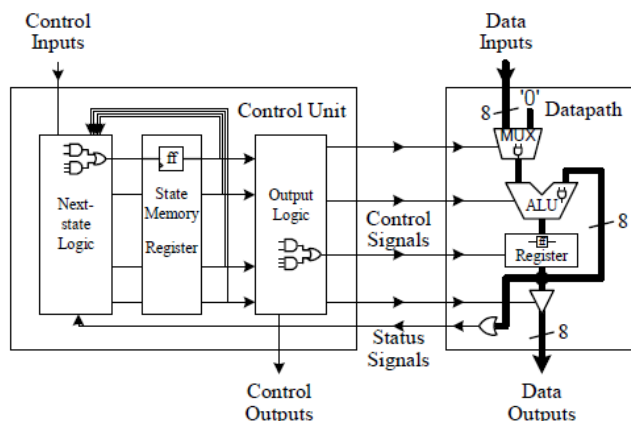


图 10.2 微处理器内部结构

如图 10.2 所示，数据通路主要包括运算单元、存储单元（寄存器）及其相互连接。其中，运算逻辑单元（*Algorithmic Logic Unit, ALU*）主要用于执行微处理器所要进行的所有对数据的操作。数据通路还包括用于存储运算数据的寄存器。数据通路中的运算单元和存储单元需要进行连接，在图 10.2 中，运算逻辑单元和寄存器之间的连接使用了一个多路选择器（*Multiplexer, MUX*）和一些数据信号线。其中数据信号线用于功能单元之间的信号连接，当一组信号线用于同一个信号的不同位数时，这组信号线也可以统一表示成总线形式，如图 10.2 数据通路中的粗线即表示一组位宽为 8 的数据总线。

多路选择器主要用于从多个输入信号中选取一个信号进行输出。图 10.2 中，一个 2 选 1 多路选择器用于从两个输入信号（一个是外部输入信号，一个是常数 0）中选取一个信号作为运算逻辑单元的输入。运算逻辑单元的输出作为寄存器的输入，寄存器的输出送给三个不同的目的地：（1）运算逻辑单元的第二个操作数；（2）一个或（OR）门用于检测该数据是否为 0；（3）一个三态缓冲器，三态缓冲器主要用于控制数据从寄存器的输出。

虽然数据通路能够实现微处理器对于数据的各种运算操作，但是它不能自动按照期望的顺序完成一系列操作。因此，如图 10.2 所示，微处理器的另外一个重要组成部分——控制单元也是必须的。控制单元，也称控制器，用于控制数据通路的所有操作从而实现微处理器运算的正确性。控制单元实际上是一个有限状态机，状态机意味着系统的运作一直处于一个状态到另外一个状态的依次切换中，而“有限”意味着系统总的状态数目是有限的。控制单元由三个部分组成：下一状态（次态）逻辑、状态存储器和输出逻辑。下一状态逻辑决定了状态机下一个时刻的状态，状态存储器用于存储状态机的当前状态，而输出逻辑产生数据链路所需的实际控制信号。

事实上，在一个典型的微处理器如 intel 奔腾 CPU（*central processing unit*）上，一般有一个指令集，利用指令集编写不同的程序从而完成不同的处理任务。这种需要专门执行一些指令集的微处理器，与图 10.2 相比，其结构可以进一步明确化，如图 10.3 所示。

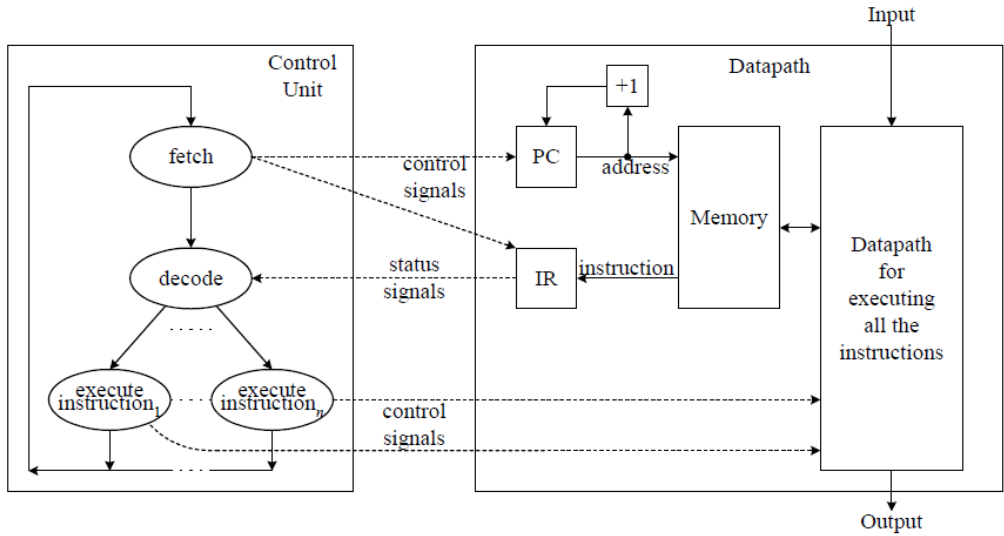


图 10.3 微处理器整体结构

在 CPU 的设计中，首先需要设计它的指令集、指令集如何编码以及指令集如何被执行。所以，我们首先要考虑的问题是，我们需要多少条指令集，具体每条指令是什么，每条指令的操作码（opcode）是什么，使用多少位宽的数据来对指令进行编码？

当指令集的上述问题确定了，接下来就是考虑如何设计能够执行指令集中所有指令的数据通路了。在这一步中，我们需要考虑的问题包括数据通路需要哪些功能单元，

需要多少寄存器，我们是使用单一的寄存器文件还是使用多个独立的寄存器，以及如何将这些不同的单元进行连接？此外，数据通路中我们还需要一个程序指针（*Program Counter*, PC）寄存器和一个指令寄存器（*Instruction Register*, IR），分别用于存储下一个指令在程序存储器中的地址信息和当前从程序存储器中获得的指令。每次当 PC 所指向的程序存储器的地址中的指令被读出到 IR 后，一般情况下 PC 的值将会被递增 1 以指向程序存储器中下一个指令的地址，例外的情况是，如果当前的指令是跳转指令，此时 PC 的值将被加载一个新的存储器地址。

微处理器的控制单元一般是循环执行三个主要步骤，通常我们称为指令周期：（1）取指：即从程序存储器中把指令取出；（2）译指：即对指令所要做的操作进行翻译；（3）执行指令。每一步骤将在有限状态机的一个状态被执行。在第三步中，一般而言每个指令通常在一个时钟周期内执行，而有些情况下一些存储器操作的指令可能需要两个甚至更长的时钟周期，这种情况下就需要更多的状态来实现正确的时序。

在第一步取指的过程中，控制单元将读出 PC 所指向的存储器中地址中的内容，并将该内容拷贝到 IR。此后假设每条指令占据程序存储器的一个存储空间，PC 递增 1 指向下一条指令的地址。在第二步的译指过程中，控制单元从 IR 中提取出操作码，决定当前执行指令，并通过跳转到需要执行的指令状态去。一旦进入某一个特定指令对应的状态，控制单元完成此状态下控制字的设定以使数据通路完成对应的操作即可。

一般而言，程序的指令都是存储在外部寄存器中的，所以除了 CPU 外，微处理器一般还需要外部存储器以及连接外部存储器与 CPU 的地址以及数据总线。因此，第一步取指的过程通常需要控制单元设置好存储器的地址，使外部存储器将对应地址的指令输出到数据总线，控制单元再从数据总线读取该数据，送给 IR。在我们的设计中，为了简单起见，我们将不使用外部程序存储器，而是将程序存储器作为数据通路的一个部分来设计，以避免对外部存储器访问过程中的握手和时序问题太多的关注，更集中于处理器设计的学习。

综上所述，对于微处理器的设计与验证，一般而言我们可以按照如下步骤进行：

- 定义指令集
- 设计数据通路
- 设计控制单元
- 使用硬件描述语言实现设计
- 编写汇编程序进行验证

10.2. 微处理器指令集

在计算机语言里的单词叫做指令（*Instructions*），计算机的词汇称为指令集（*Instruction Set*）。在一个计算机上运行的所有程序都使用相同的指令集。甚至复杂的软件应用，如文字处理和电子表格应用程序，最终被编译成一系列简单的指令，例如加、减和跳转指令。处理器指令需指明要执行的操作和要使用的操作数。操作数可能来自存储器、寄存器或指令本身。

处理器硬件只能理解1和0的数字，所以指令必须编码为二进制数格式，也被称为机器语言。就像我们用字母来编码人类的语言，计算机使用二进制数编码的机器语言。微处理器是读取和执行的机器语言指令的数字系统。然而，考虑到阅读机器语言很乏味且难理解，所以我们更愿意用一种符号格式表示各种指令，也称为汇编语言（*Assembly Language*）。

不同的处理器架构的指令集更像不同的方言。几乎所有的处理器架构定义了基本的指令，如加、减和跳转指令，对内存或寄存器进行操作。一旦你学会了一个指令集，理解另外一个就相当容易了。

一种处理器指令架构没有去定义底层的硬件实现。通常情况下，存在一个相同的架构可以由许多不同的硬件实现。例如，英特尔和超微（AMD）出售属于同一IA-32架构的各种微处理器。它们都可以运行同样的程序，但它们采用不同的底层硬件，在性能、价格和功耗上有不同的权衡。一些微处理器为高性能服务器优化，而另一些为笔记本电脑延长电池使用寿命而优化。寄存器、存储器、ALU和其他构件块的特定组合，以形成一个微处理器被称为微结构，不同的微结构也可以采用相同的指令架构。

这里我们介绍最早由John Hennessy和他的同事在斯坦福大学在20世纪80年代开发的MIPS指令架构。

Patterson和Hennessy阐述了设计MIPS架构的四个原则：（1）简洁规整；（2）加快经常性事件的速度；（3）较小的速度更快；（4）好的设计要求良好的妥协。

汇编语言

汇编语言是计算机编码母语的人类可读性表示。每个汇编语言指令同时指定要执行的操作和相应的操作数。我们介绍简单的算术指令，并显示这些操作是如何用汇编语言编写。然后，我们定义MIPS指令操作数：寄存器，存储器和常数。

最常见的操作计算机上执行是加法。

代码示例10.1 加法

High-Level Code	MIPS Assembly Code
<code>a = b + c;</code>	<code>add a, b, c</code>

汇编指令的第一部分add，叫做助记符，指示执行什么操作。对源操作数b和c，执行加操作，并且将结果写入到目标操作数a。

代码示例10.2 减法

High-Level Code	MIPS Assembly Code
<code>a = b - c;</code>	<code>sub a, b, c</code>

代码示例10.2显示的减法指令，类似于加法指令。指令格式与add指令相同，除了该操作定义sub与加法指令不同。

Simplicity favors regularity: 指令的操作数的数目一致。在这种情况下，两个源操作数和一个目的操作数，更容易进行编码和硬件处理。更复杂的高级代码可以转换成多个MIPS的指令实现，如以下代码所示。

代码示例10.3 复杂运算

High-Level Code	MIPS Assembly Code
<pre>a = b + c - d; // single-line comment /* multiple-line comment */</pre>	<pre>sub t, c, d # t = c - d add a, b, t # a = b + t</pre>

代码示例的汇编语言程序需要一个临时变量`t`，存储中间结果。使用多个汇编语言指令以执行更复杂的操作。

Make the common case fast: MIPS指令集只包括简单、常用的指令。MIPS的指令数目很少，使得解码指令和操作数所需的硬件可以简单、小和快速的。使用的多个简单的指令序列的执行，达到不常见的更复杂的功能。因此，MIPS属于精简指令集计算机

(RISC)体系结构。许多复杂的指令，如Intel的IA-32架构，是复杂指令集计算机(CISC)。例如，IA-32定义了一个“串移动”指令，该指令将文件从一个存储器部分到另一个字符串（一系列字符）。这样的操作在RISC机器需要多条、甚至可能上百简单指令。然而，在CISC架构实施复杂指令的代价是添加硬件开销，但这样也会减慢简单指令。

RISC架构最大限度地减少了硬件的复杂性和必要的指令编码，保持一套独特的小指令集。

一条指令只操作数进行操作。但处理器只对1和0进行操作，而不能对变量名操作。指令需要从中取回二进制数据的物理位置。操作数可以存储在寄存器或存储器中，或者是存储在指令本身的常数（*Constants*）。计算机使用不同位置的操作数，以优化速度和数据容量。存储为常量或寄存器里的操作数能快速访问，但只能存放少量数据。额外大量数据必须从存储器读取，但访问速度缓慢。因为对32位的数据进行操作，MIPS称为32位体系结构。

指令需要快速访问操作数，才能快速运行。但在存储器里的操作数需要很长的时间来检索。因此，大多数的指令架构指定少量寄存器，存放常用的操作数。MIPS架构采用32个寄存器，称为寄存器组（*register set*）或寄存器文件（*register file*）。寄存器数量越少，它们的访问越快。

Smaller is faster: 从你的办公桌上少量的相关书籍查找信息比在图书馆的书库中搜索信息要快得多。同样，从一小组寄存器（例如32个）读取数据比从1000个寄存器或一个更大的存储器中读出速度更快。一个小的寄存器文件通常用一个小SRAM阵列构造。SRAM阵列使用小的解码器，并连接相对较少的存储单元位线，所以它比一个大存储器能缩短关键路径。

代码示例10.4 寄存器操作

High-Level Code	MIPS Assembly Code
<pre>a = b + c;</pre>	<pre># \$s0 = a, \$s1 = b, \$s2 = c add \$s0, \$s1, \$s2 # a = b + c</pre>

代码示例10.4显示了寄存器操作数的`add`指令。MIPS寄存器名称由\$符号开头。变量`a`、`b`和`c`可以自由地放置在`$S0`、`$S1`和`$S2`。该指令将存在`$S1(b)`和`$S2(c)`的32位值相加，32位结果写入`$S0(a)`。

MIPS架构定义了32个寄存器。每个寄存器有一个名字，和一个从0到31的数字编号。
\$0总是包含0值，因为这个常量0在计算机程序中使用非常频繁。

表10.1 MIPS寄存器组

Name	Number	Use
\$0	0	the constant value 0
\$at	1	assembler temporary
\$v0-\$v1	2-3	procedure return values
\$a0-\$a3	4-7	procedure arguments
\$t0-\$t7	8-15	temporary variables
\$s0-\$s7	16-23	saved variables
\$t8-\$t9	24-25	temporary variables
\$k0-\$k1	26-27	operating system (OS) temporaries
\$gp	28	global pointer
\$sp	29	stack pointer
\$fp	30	frame pointer
\$ra	31	procedure return address

如果寄存器是操作数是唯一的存储空间，我们将限于不超过32变量的简单程序。然而，数据也可以存储在存储器中。相比于寄存器文件，存储器具有更多数据的空间，但访问需要较长的时间量。而寄存器文件体积小速度快，内存大速度慢。出于这个原因，通常使用的变量储存在寄存器中。通过使用存储器和寄存器的组合，程序可以相当快速地访问大量数据。MIPS架构使用32位的存储器地址和32位的数据字。

MIPS使用*load word*指令，*lw*，将存储器中的数据字读取到一寄存器中。所述*lw*指令指定基地址（*base address*）和偏移（*offset*）的总和作为存储器的有效地址（*effective address*）。基地址（指令括号中）是寄存器。偏移量是一个常数（括号之前）。

代码示例10.5 读取字寻址存储器

Assembly Code

```
# This assembly code (unlike MIPS) assumes word-addressable memory
lw $s3, 1($0)    # read memory word 1 into $s3
```

同样，MIPS使用*store word*指令，*sw*，将寄存器中的数据字写入到内存中。任何寄存器均可以用来提供基址。

代码示例10.6 写字寻址存储器

Assembly Code

```
# This assembly code (unlike MIPS) assumes word-addressable memory
sw  $s7, 5($0)    # write $s7 to memory word 5
```


MIPS使用字节可寻址存储器。在存储器中的每个字节都有一个唯一的地址。字节寻址存储器用*big-endian* 或 *little-endian*方式组织。在这两种格式中，最高有效字节 (*most significant byte*, MSB) 在左侧，最低有效字节 (*least significant byte*, LSB) 位于右侧。在 *big-endian* 机器，字节的编号0从大 (*most significant*) 端开始。在 *little-endian* 机器，字节编号0从小 (*least significant*) 端开始。两种格式的字地址相同，一个字有四个字节。一个字中只有字节的地址不同。

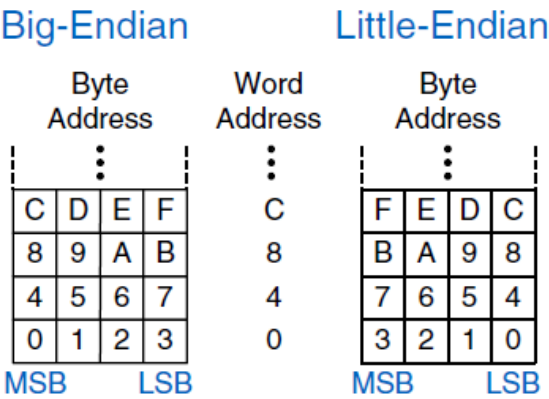


图10.4 内存字节寻址

读取字和存储字，*LW*和*SW*，的MIPS指令也可以使用常量。这些常数被称为立即数，因为它们的值可立即从指令得到，并且不需要寄存器或存储器的访问。立即数加，*addi*，是另一个常用的MIPS立即数指令。

代码示例10.7 立即数操作

High-Level Code	MIPS Assembly Code
<pre>a = a + 4; b = a - 12;</pre>	<pre># \$s0 = a, \$s1 = b addi \$s0, \$s0, 4 # a = a + 4 addi \$s1, \$s0, -12 # b = a - 12</pre>

在指令中指定的立即数是一个16位二进制补码数据，范围是[-32768, 32767]。减法等效于加一个负数，因此简单起见，在MIPS架构中没有*subi*指令。

单一指令格式是简单，但不够灵活。MIPS指令集支持3个指令格式，这是一种妥协。一种格式，用于如*add*和*sub*指令，有三个寄存器操作数。另一种格式，用于如*lw*和*sw*指令，具有两个寄存器操作数和一个16位立即数。第三种格式，有一个26位的立即数，无寄存器。

机器语言

汇编语言是便于人们阅读，然而数字电路只明白1和0。因此，用汇编语言编写的程序要从助记符转译成只使用1和0的机器语言 (*machine language*)。

MIPS采用32位指令。Simplicity favors regularity，最通常的选择是要将所有指令编码为存储在存储器一个字。即使某些指令可能不需要编码的所有32位，可变长度的指令

会增加太多的复杂性。MIPS定义三种指令格式：R型、I型和J型。较少的格式数目允许所有类型之间一定的规律性，并且硬件因此较简单，同时又适应不同的指令需要，例如需要编码大常数的指令。R型指令对三个寄存器进行操作。I型指令对两个寄存器操作和一个16位立即数。J型（跳转）指令对一个26位立即数操作。

R-type Instructions

名字R型是寄存器类型（*register-type*）的缩写。R型指令使用三个寄存器作为操作数：两个作为源，一个作为目标。图10.5显示了R型机器的指令格式。32位指令有六个数字域：*op*，*rs*，*rt*，*rd*，*shamt*和*funct*。每个字段为五或六比特，如图10.5所示。

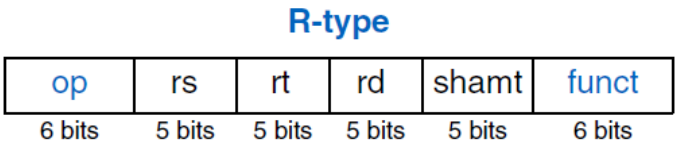


图10.5 R型机器指令格式

指令执行操作中的两个字段编码很重要：*op*（也称为*opcode*或操作码）和*funct*（也称为功能码）。所有的R型指令的*opcode*为0，特定的R型操作由*funct*字段确定。例如，*add*指令的*opcode*和*funct*字段分别是0 (000000₂) 和32 (100000₂)。类似，*sub*指令的*opcode*和*funct*字段分别是0和34。

操作数被编码在*rs*、*rt*和*rd*三个字段。前两个寄存器*rs*和*rt*是源寄存器；*Rd*是目标寄存器。

第五字段*shamt*，只用于移位操作。在这些指令中，存储在*shamt*字段的5位二进制值表示移位量。对于所有其它R型指令，*shamt*为0。

图10.6显示了R型指令*add*和*sub*的机器码。请注意，目标寄存器在汇编语言指令中是第一个寄存器，但它在机器语言指令中是第三个寄存器字段（*rd*）。例如，汇编指令 *add \$S0, \$S1, \$ S2*的源数据*RS*=\$ S1（17），*RT*=\$ S2（18）结果存到 *RD*=\$ S0（16）。

Assembly Code	Field Values						Machine Code					
	op	rs	rt	rd	shamt	funct	op	rs	rt	rd	shamt	funct
add \$s0, \$s1, \$s2	0	17	18	16	0	32	000000	10001	10010	10000	00000	100000 (0x02328020)
sub \$t0, \$t3, \$t5	0	11	13	8	0	34	000000	01011	01101	01000	00000	100010 (0x016D4022)
	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

图10.6 R型指令机器码

I-Type Instructions

名字I型是立即数类型（*immediate-type*）的缩写。I型指令使用两个寄存器操作数和一个立即数。图10.7显示了I型机的指令格式。32位指令有四个字段：*op*，*rs*，*rt*和*imm*。前三个字段，*op*、*rs*和*rt*，类似于R型指令。*imm*字段保存16位立即数。

I-type

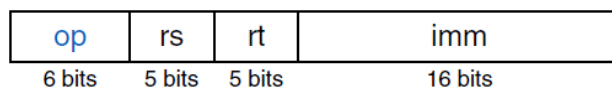


图10.7 I型指令格式

该操作完全由opcode确定。操作数由rs、rt和imm三个字段确定。rs和imm始终用作源操作数。Rt在某些指令（如*addi*和*lw*）用作目标操作数，但其它指令可作为另一种源操作数（如*sw*）。

图10.8显示I型指令编码的几个例子。负值立即数是使用16位二进制补码表示的。rt在汇编语言指令列在第一个用作目标寄存器，但它在机器语言指令中是位于第二个寄存器字段的。

Assembly Code	Field Values				Machine Code				
	op	rs	rt	imm	op	rs	rt	imm	
<i>addi</i> \$s0, \$s1, 5	8	17	16	5	001000	10001	10000	0000 0000 0000 0101	(0x22300005)
<i>addi</i> \$t0, \$s3, -12	8	19	8	-12	001000	10011	01000	1111 1111 1111 0100	(0x2268FFF4)
<i>lw</i> \$t2, 32(\$0)	35	0	10	32	100011	00000	01010	0000 0000 0010 0000	(0x8C0A0020)
<i>sw</i> \$s1, 4(\$t1)	43	9	17	4	101011	01001	10001	0000 0000 0000 0100	(0xAD310004)
	6 bits	5 bits	5 bits	16 bits	6 bits	5 bits	5 bits	16 bits	

图10.8 I型指令机器码

I型指令有一个16位的立即数字段，但立即数是在32位操作中使用的。例如，*lw*需要在32位的基址寄存器加一个16位的偏移量。如果是正的立即数，16位立即数扩展为32位，上半部分全为0；如果是负的立即数，扩展为32位时上半部分全为1。这被称为符号扩展。2的补数符号的扩展不改变它的值。

J-type Instructions

名字J型是跳转型（*jump-type*）的缩写。此格式只用跳转指令使用。此指令格式使用单26位的地址操作数，*addr*，见图10.9。像其他格式，J型指令由一个6位的opcode开始。剩余位用于指定一个地址*addr*。

为了解释机器语言，必须解译32位指令字的每个字段。不同的指令使用不同的格式，但所有格式由6位opcode字段开始。因此，开始最好的地方是看指令的opcode。如果它是0，则指令是R型，否则它是I型或J型。

J-type

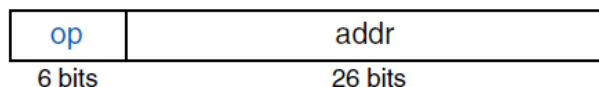


图10.9 J型指令格式

【例】将下面的机器语言代码翻译为汇编语言。

0x2237FFF1

0x02F34022

解：首先，我们把每一条指令写成二进制，并在最高六位查找每个指令的opcode，如图10.10所示。操作码opcode决定如何解释其余位。opcode分别为001000₂ (8₁₀) 和000000₂ (0₁₀)，分别表示addi和R型指令。R型指令的funct字段是100010₂ (34₁₀)，这表明它是一个sub指令。图10.10显示了相当于两个机器指令的汇编代码。

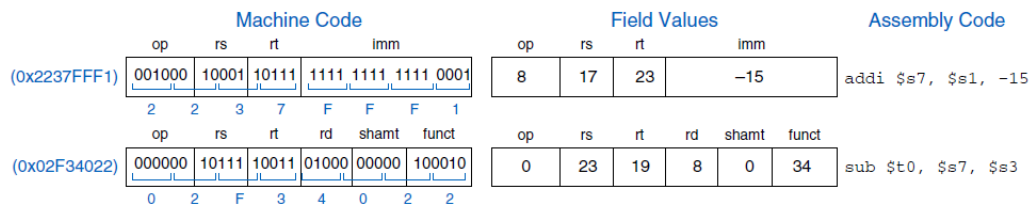


图10.10 机器码到汇编码转换

用机器语言写的程序是一个32位数字代表的指令序列。像其它二进制数，这些指令也可以存储在存储器中。这就是所谓的存储程序（*stored program*）的概念，它是为什么计算机如此强大的关键原因。运行不同的程序，并不需要大量的时间和精力来配置或重新布线硬件，只需要编写新的程序到存储器里。取代专用硬件，存储程序提供了通用（*general purpose*）计算。以这种方式，一台计算机可以简单地通过改变所存储的程序，执行从一个计算器、到文字处理和视频播放器的应用。

10.3. 硬连线型微处理器设计

【例1】我们先考察一个简单的微处理器 EC-1。EC 是 easy computer 的简写，表示该处理器非常简单，所能作的操作也非常有限，要使用尽可能少的变量。但是，麻雀虽小，五脏俱全，通过这一个简单的微处理器设计的学习，对微处理器的设计将会有一个整体的理解。

1) 指令集-1

EC-1 微处理器的指令集以及对应的编码如图 10.11 中的定义。“指令(Instruction)”一列列出了我们编写汇编程序所用到的指令。“指令编码(Encoding)”一列给出了对应于每个指令的二进制编码。“操作(Operation)”一列给出了对应于每个指令具体的操作。

Instruction	Encoding	Operation	Comment
IN A	011 xxxxx	$A \leftarrow \text{input}$	Input to A
OUT A	100 xxxxx	$\text{output} \leftarrow A$	Output from A
DEC A	101 xxxxx	$A \leftarrow A - 1$	Decrement A
JNZ address	110 xaaaa	if ($A \neq 0$) then $PC = \text{aaaa}$	Jump to address if A is not zero
HALT	111 xxxxx	Halt	Halt execution

Notations:

- A = accumulator.
- PC = program counter.
- aaaa = four bits for specifying a memory address.
- x = don't cares.

图 10.11 微处理器指令集

从图 10.11 中可以看出，我们这一微处理器的指令集只有五个指令。为了实现对五个指令进行编码，操作数需要三个比特（三个比特最多可以获得 8 种组合）。从“指令编码（Encoding）”一列可以看到，编码中的最左边三位就是对应于每个操作的编码。比如说，IN A 指令的操作码是 011，OUT A 指令的操作码是 100 等等。其中，000，001 和 010 三个编码没有定义，可以用于空操作（no-operation, nop）。由于每个指令的位宽固定为 8 比特，除了指令 JNZ 外，对应于其它指令的最后 5 个比特都没有被使用。通常情况下，对于一些更复杂的指令集，后面的这些位数将可能被用作操作数来指定哪个寄存器或者哪个资源要被用到。在我们的例子中，只有指令 JNZ 使用了后面的 4 个比特，即表中的 aaaa，来描述跳转指令的跳转地址。

指令 IN A 输入一个 8 比特的数值，并存储到累加器 A 中。累加器 A 是一个用于数据操作的 8 比特寄存器。指令 OUT A 将累加器 A 中的数值拷贝到输出端口。在这个版本的微处理器中，由于累加器的内容总是可以在输出端口得到，所以这个指令实际上是不需要的，这里使用这个指令主要是考虑程序需要一个输出指令。指令 DEC A 把累加器 A 中的数值减 1 后再存入到累加器 A 中。指令 JNZ（Jump Not Zero）测试 A 中的数值是否为 0，假如为 0，这个指令什么也不做，假如不为 0，则将指令中的后四位，即 aaaa（表示跳转地址）加载到寄存器 PC 中。当这个值加载到 PC 后，实际上我们就可以完成程序的跳转了。最后，指令 HALT 通过使 CPU 停止在 halt 状态从而停止微处理器的运行，直到系统复位跳出 halt 状态。

2) 数据通路-1

EC-1 微处理器的指令集定义好之后，我们现在可以开始设计实现所有指令集中操作的定制数据通路了。设计好的数据通路如图 10.12 所示。这一数据通路可以看成三个独立的部分：（1）执行指令周期中的取指操作，并递增 PC 值或者加载新的 PC 值；（2）存储器；（3）实现指令集中所有指令的操作。

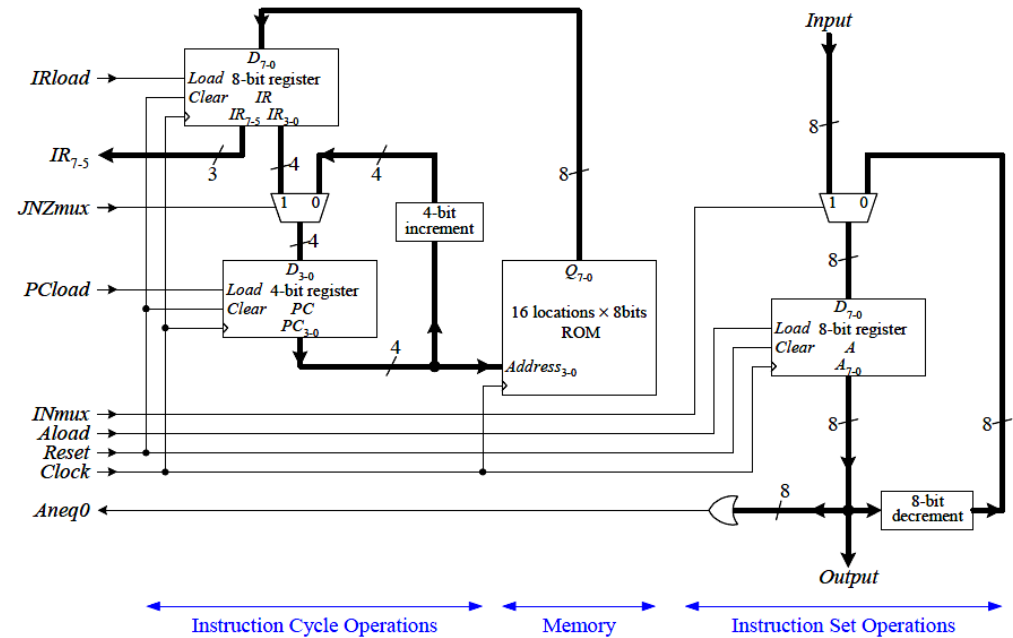


图 10.12 EC-1 处理器的数据通路

用于指令周期操作的数据通路部分主要包括一个 PC 寄存器和一个 IR 寄存器。指令的位宽决定了 IR 寄存器的大小,而程序寄存器的大小即程序寄存器的地址数目决定了 PC 寄存器的大小。对于图 10.12 中的数据通路,我们需要一个具有 16 个存储单元的程序存储器,每一个存储单元中的数据位宽为 8 位,因此我们需要一个 4 比特的地址 ($2^4=16$)。因此,PC 寄存器是 4 比特的位宽,而 IR 寄存器是 8 比特位宽。使用一个 4 比特的递增单元来实现 PC 寄存器中内容的递增。PC 寄存器中的数据加载有两种可能,一是从地址递增单元的输出加载进来,二是 JNZ 指令给出的地址加载进来。使用一个 2 选 1 多路选择器实现这一功能,多路选择器的一路输入来自地址递增单元的输出,另外一路输入来自 IR 寄存器的最低四位,即图 10.11 中的 aaaa。

如前所述,为了使得我们的设计不要太复杂,更容易从整体上掌握微处理器的设计,我们不使用外部程序存储器,而是将程序存储器作为数据通路的一部分进行设计。因此,在我们的设计中,程序存储器使用一个具有 16 个地址,每个地址数据为 8 比特的只读存储器 (Read-only Memory, ROM)。使用只读存储器是因为指令集中没有指令需要将数据写入到存储器。由于存储器的地址总是由 PC 寄存器中的内容决定的,PC 寄存器的输出直接连接到 4 比特的存储器的地址线。程序存储器的输出需要连接到 IR 寄存器,也即程序存储器的 8 比特输出 Q_{7-0} 需要直接连接到 IR 寄存器的输入端,以完成取指操作。

数据通路中用于完成指令集中所有指令操作的部分包括一个 8 比特的累加器 A 和一个 8 比特的递减单元,以及一个用于选择累加器 A 输入的 2 选 1 多路选择器。递减单元用于完成对累加器 A 中的数据数据的递减操作。对指令 IN A 而言,累加器 A 的输入来自于数据输入端口,而对于指令 DEC A 而言,累加器 A 的输入来自于递减单元的输出。累加器的输出直接连接到数据输出端口,所以指令 OUT A 实际上无需特别的数据通路的操作,而相当于一直进行 OUT A 的操作。指令 JNZ A 需要一个 8 比特的 OR 门连接到累加器的所有输出上,来测试条件 A 是否不等于 0。指令 JNZ A 的实际操作是 IR 寄存器的低 4 位数据加载到 PC 寄存器中。指令 HALT 不需要完成任何额外的操作。

数据通路中的控制字主要有 5 个控制信号,即 IRload, PClload, INmux, Aload 和 JNZmux。数据通路给控制单元提供一个状态反馈信号, Aneq0。执行指令周期操作的控制字以及指令集将在接下来的部分进行进一步讨论。

3) 控制单元-1

控制单元的状态图如图 10.13 (a) 所示,每一个状态的操作特别是每个状态的控制字的设置如图 10.13 (d) 所示。第一个状态是 START (000) 作为初始复位状态。在这一状态,无所任何操作。此外,这一状态提供了一个额外的时钟周期,在有些操作需要额外的时钟周期才能够完成的情况下,是有用的。虽然对于那些无需额外时钟周期即可完成的操作,技术上而言应该是直接回到 FETCH 状态而不是 START 状态,但是为了次态表和激励方程设计的简便性,此处统一在所有指令操作完成后先回到 START 状态。

在所有的五个指令中,只有指令 JNZ A 需要一个额外的时钟周期完成它的操作。

这是因为当 JNZ 的条件测试为真时, PC 将被加载一个新的地址。然而, PC 加载新地址只有在下一个有效时钟沿到达时候才真正发生。所以, 如果我们让有限状态机在下一个时钟周期直接进入 FETCH 状态的话, IR 寄存器中加载的仍然是程序存储器中旧地址的数据, 而不是新的 PC 指向的地址。因此, 当我们让有限状态机先回到 START 状态时, PC 将会在这个状态被更新成新的值, 而在下一个 FETCH 状态时候, 程序存储器中的新的地址的数据将会被读出并加载到 IR 寄存器中。

从有限状态机的 START 状态出发, 控制单元将会无条件跳转到 FETCH 状态。在 FETCH 状态中, 通过控制 IRload 信号有效, 将 PC 寄存器指向的程序存储器地址中的数据加载到 IR 寄存器, 然后 PC 寄存器中的内容递增 1 后, 通过控制信号 PCload 有效将递增后的数据加载到 PC 寄存器中。在 DECODE 状态, 控制单元将测试 IR 寄存器的高三位, 即 IR₇₋₅, 然后通过对操作数解码, 跳转到对应的状态。

在对应于五个指令的五个执行状态中, 数据通路控制对应的控制信号来执行对应的指令操作。比如说, 对于指令 IN A 操作, 设置 INmux 信号为 1 来选择输入信号为外部端口, 设置 Aload 信号为 1 来加载输入信号到累加器 A。注意, 为了输入指令能够读取到正确的输入值, 输入数据需要在微处理器复位之前就设置好。而且, 由于 INPUT 状态不需要等待一个类似于回车键的信号, 因此, 即使有多个数据输入, 也只能有一个数据会被读入进来。指令 DEC A 需要设置 INmux 信号为 0、Aload 信号为 1, 以使递减单元的输出被连接到累加器并加载进来。

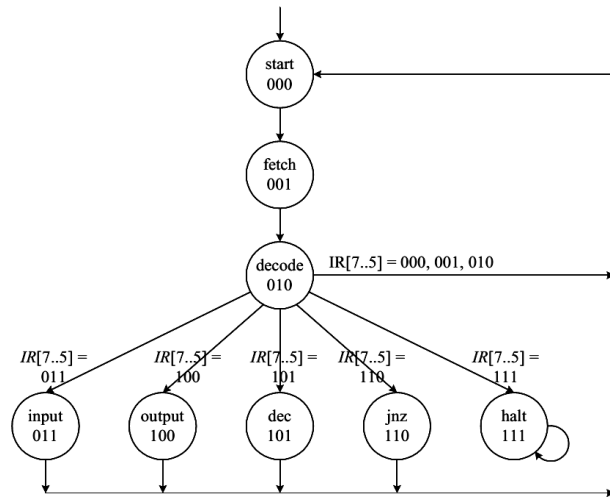
指令 JNZ 使 JNZmux 信号有效以将 IR 寄存器中的低 4 位地址信号加载到 PC 寄存器。PC 寄存器是否需要从 IR 的低 4 位地址信号加载进来, 取决于状态反馈信号 Aneq0。因此, 控制信号 PCload 信号只有在 Aneq0 等于 1 的时候才会被设置为有效。依赖于 Aneq0 信号设置 PCload 信号, 可以使得状态图少用一个状态, 从而使得有限状态机更小, 实现一个 Mealy 有限状态机。否则, 有限状态机将需要给 JNZ 这一指令分配两个状态, 即当 Aneq 为 0 时设置 PCload 有效, 当 Aneq 不为 0 是设置 PCload 无效。

当有限状态机进入到 HALT 状态, 将会无条件的停留在 HALT 状态, 使得微处理器看上去像停止操作一样。

对应于状态图的次态表、实现表, 以及从实现表得到的三个激励方程如图 10.13 (b) 和 (c) 所示。整个状态机一共 8 个状态, 所以使用 3 个 D 触发器来实现控制单元电路。注意到表中为 0 的项占绝大多数, 只有解码状态有一些不同的取值, 所以激励方程的得到还是比较容易的。

输出方程如图 10.13 (e) 所示, 可以直接从图 10.13 (d) 中的输出表得到。

最后, 基于激励方程和输出方程, 我们可以得到整个控制单元的完整电路实现。本例中 EC-1 微处理器的完整控制电路如图 10.13 (f) 所示。



(a)

Current State $Q_2Q_1Q_0$	Next State $Q_{2next}Q_{1next}Q_{0next}$ (Implementation $D_2D_1D_0$)								
	IR_7, IR_6, IR_5								
	000 nop	001 nop	010 nop	011 input	100 output	101 dec	110 jnz	111 halt	
000 start	001	001	001	001	001	001	001	001	
001 fetch	010	010	010	010	010	010	010	010	
010 decode	000	000	000	011	100	101	110	111	
011 input	000	000	000	000	000	000	000	000	
100 output	000	000	000	000	000	000	000	000	
101 dec	000	000	000	000	000	000	000	000	
110 jnz	000	000	000	000	000	000	000	000	
111 halt	111	111	111	111	111	111	111	111	

(b)

$$D_2 = Q_2Q_1Q_0 + Q_2'Q_1Q_0'IR_7$$

$$D_1 = Q_2Q_1Q_0 + Q_2'Q_1Q_0'(IR_6IR_5 + IR_7IR_6) + Q_2'Q_1'Q_0$$

$$D_0 = Q_2Q_1Q_0 + Q_2'Q_1Q_0'(IR_6IR_5 + IR_7IR_5) + Q_2'Q_1'Q_0'$$

(c)

Control Word	State $Q_2Q_1Q_0$	$IRload$	$PCload$	$INmux$	$Aload$	$JNZmux$	$Halt$
0	000 start	0	0	0	0	0	0
1	001 fetch	1	1	0	0	0	0
2	010 decode	0	0	0	0	0	0
3	011 input	0	0	1	1	0	0
4	100 output	0	0	0	0	0	0
5	101 dec	0	0	0	1	0	0
6	110 jnz	0	if ($Aneq0 = 1$) then 1 else 0	0	0	1	0
7	111 halt	0	0	0	0	0	1

(d)

$$\begin{aligned}
 IRload &= Q_2'Q_1'Q_0 \\
 PCload &= Q_2'Q_1'Q_0 + Q_2Q_1Q_0' \text{ (Aneq0)} \\
 INmux &= Q_2'Q_1Q_0 \\
 Aload &= Q_2'Q_1Q_0 + Q_2Q_1'Q_0 \\
 JNZmux &= Q_2Q_1Q_0' \\
 Halt &= Q_2Q_1Q_0
 \end{aligned}$$

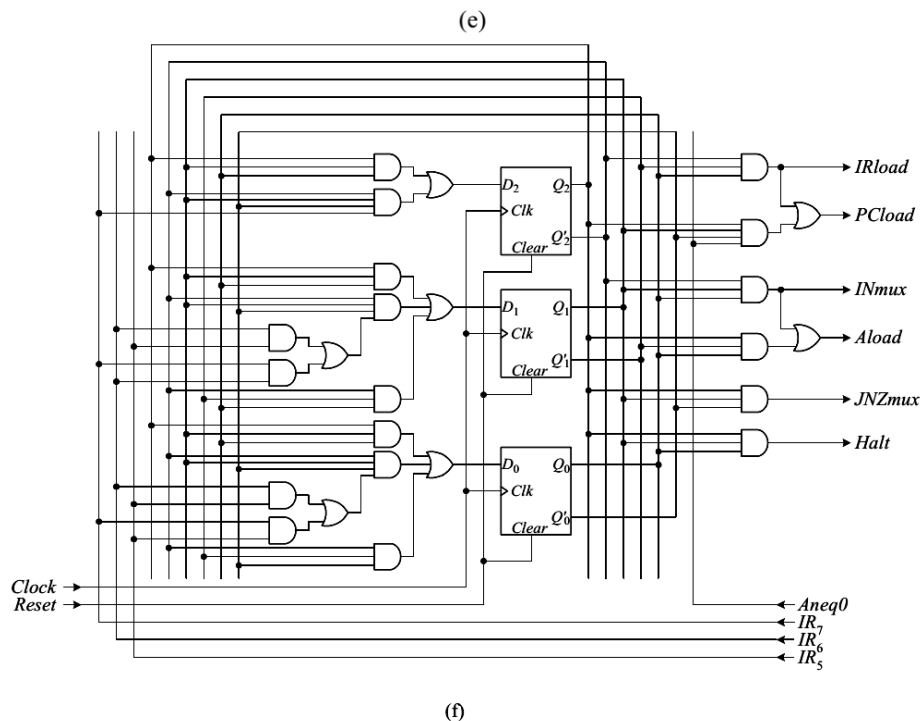


图 10.13 EC-1 微处理器控制单元

(a) 状态图; (b) 下一状态表; (c) 实现表; (d) 激励方程; (e) 控制字/输出表; (f) 电路

4) 完整电路-1

将图 10.12 中的数据通路电路与图 10.13 (f) 中的控制单元电路中对应的控制信号和状态反馈信号进行连接, 即可得到图 10.14 所示的 EC-1 微处理器的完整电路。

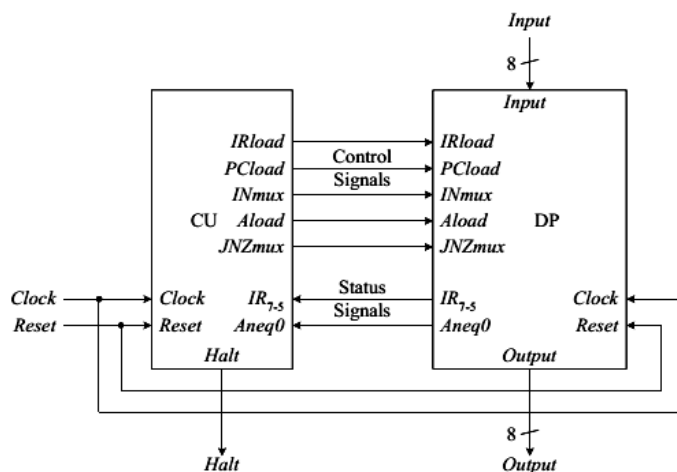


图 10.14 EC-1 微处理器的完整电路

5) 示例汇编程序-1

微处理器设计从程序存储器中读取一系列程序指令进行执行。因此，为了测试我们的 EC-1 微处理器，我们需要依据指令集中提供的指令写一个测试程序。

由图 10.11 可知，我们的 EC-1 微处理器只有五个指令。因此，我们的汇编示例程序中，将使用这 5 个指令写一个程序来输入一个数据，将此数据进行递减直至 0。汇编程序如图 10.15 (a) 所示。

```
IN A      -- input a value into the A register
loop: OUT A -- output the value from the A register
DEC A     -- decrement A by one
JNZ loop  -- go back to loop if A is not zero
HALT      -- halt
```

(a)

memory address	instruction encoding
0000	01100000; -- IN A
0001	10000000; -- OUT A
0010	10100000; -- DEC A
0011	11000001; -- JNZ loop
0100	11111111; -- HALT

(b)

图 10.15 EC-1 微处理器运行的递减计数程序：(a) 汇编程序；(b) 二进制代码

由于我们的 EC-1 微处理器没有编译器，所以我们需要手工编译这个程序。这个程序的二进制可执行代码可以将指令用图 10.11 中每个指令所对应的 3 位二进制操作码和 5 位二进制操作数组合得到，如图 10.15 (b) 所示。除了指令 JNZ 外，所有的指令都是不需要操作数的，所以这些指令的操作数可以任意设置，每位设置为 0 或者 1 均可。比如，从图 10.15 可以，指令 IN A 的操作码是 011，因此第一个指令的编码为 01100000。类似的，指令 OUT A 的操作码为 100，所以这个指令的编码可以设置为 10000000。

对于指令 JNZ 而言，最右边 4 位表示当检测条件为真时程序跳转的地址。在我们的例子中，假设第一个指令 IN A 存储在程序存储器的地址为 0000，由于指令 JNZ 将跳转到第二个指令 OUT A（在程序存储器中的地址是 0001），所以，JNZ 指令的最后四位操作数被设置为 0001。由于指令 JNZ 的操作码为 110，所以综合起来指令 JNZ 完整的编码是 11000001。

一般情况下，程序指令是存储在计算机 CPU 的外部存储器的，计算机操作系统会通过一定的方法独立的访问程序存储器中的程序指令。但是在我们的设计中，为了保持设计的简洁，我们把程序存储器跟 CPU 一起设计在数据通路中了。并且，我们的设计中也没有操作系统能够来独立的将程序指令读出来。因此，我们的程序必须在数据通路和微处理器综合之前加载到存储器中去。无论是 Xilinx FPGA 还是 Altera FPGA，都可以使用其中内置的程序库来实现程序存储器，比如 Altera FPGA，可以使用 Altera LPM 组件库，更多信息可以参考具体 FPGA 提供商的文档。本设计中，存储器的内容可以使用一个叫 PROGRAM.MIF 的文本文件来初始化。因此，为了将程序加载到存储器，我们需要将程序的二进制码编写到文本文件中，然后重新综合微处理器

电路。在我们的例子中，PROGRAM.MIF 文件的内容如图 10.13 所示，在-- 之后的内容表示注释，所有大写字符表示是文件的关键字。

```
-- Content of the ROM memory in the file PROGRAM.MIF

DEPTH = 16; -- Number of memory locations: 4-bit address
WIDTH = 8;  -- Data Width of memory: 8-bit data

ADDRESS_RADIX = BIN;  -- Specifies the address values are in binary
                        -- Other valid radices are HEX, DEC, OCT, BIN
DATA_RADIX = BIN;      -- Specifies the data values are in binary

-- Specify memory content.
-- Format of each memory location is
--      address : data

CONTENT
BEGIN

    [0000..1111] : 00000000;  -- Initialize locations range 00-1F to 0

-- Program to countdown from an input to 0

    0000 : 01100000; -- input A
    0001 : 10000000; -- output A
    0010 : 10100000; -- decrement A
    0011 : 11000001; -- jnz 0001
    0100 : 11111111; -- halt

END;
```

图 10.13 存储 EC-1 程序的 PROGRAM.MIF 文件

在每次修改了 PROGRAM.MIF 文件之后，需要记得重新综合你的设计。无论是 Xilinx FPGA 还是 Altera FPGA 的设计工具，在设计改变后，都会有提示需要重新进行综合。

6) 硬件电路实现-1

根据冯诺依曼计算机结构，一个完整的计算机，除了包含微处理器之外，还包含存储器、输入和输出设备。在上述例子中，我们设计了一个微处理器，并将程序存储器也一并设计在其中。所以，最后一步是加上输入和输出设备。图 10.17 中给出了微处理器和外部输入输出设备之间的一个应用示例。使用 Xilinx 或者 Altera 的开发板，使用其中的八个拨码开关作为输入设备，使用 2 个七段数码管作为输出设备。由于微处理器的输出为 8 比特二进制数，我们需要 2 个 8 比特二进制编码转十进制显示的解码器，这样我们可以在 2 个七段数码管上看到输出的 8 比特二进制数。我们还需使用一个 LED 来显示我们的微处理器是否已经停止工作，使用一个按钮作为复位按钮。此外，板上有一个较高频率（如 25MHz）的时钟，通过分频后变成 4Hz 作为微处理器的时钟，使用很低频率的时钟，我们可以清楚的观察到微处理器工作时候的状态变化。

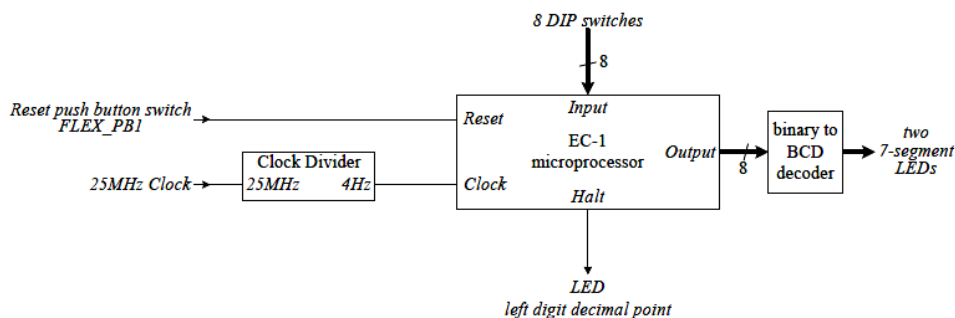


图 10.17 EC-1 微处理器的硬件实现

【例 2】在前述微处理器 EC-1 设计的基础上，我们进一步将指令集做一点扩展，系统比 EC-1 略微复杂一点，我们称之为 EC-2 微处理器。

1) 指令集-2

如图 10.18 所示，EC-2 微处理器的指令集有 8 个指令。这些指令仍然是 8 位的，我们使用其中三位来对操作进行编码，如“Encoding”一列所示。

LOAD 指令用来把存储器特定地址中的内容赋给累加器 A，地址是由指令的低 5 位表示。STORE 指令与 LOAD 类似，是把 A 中存储的内容赋给存储器特定地址单元。ADD 和 SUB 指令分别表示把 A 的内容和存储器特定地址的内容相加、相减，并把其结果再赋给 A。IN 指令表示输入一个值并存入 A。JZ 指令表示如果 A=0 则把特定地址赋给 PC。把新地址赋给 PC 会导致微处理器跳转到存储器这个新地址继续执行程序。JPOS 表示如果 A 是正数则把 A 中的内容作为特定地址赋给 PC。因为 A 是用补码表示，所以如果 A 的最高位为 0 时，则 A 是正数。最后，HALT 指令使得微处理器停止运行。

Instruction	Encoding	Operation	Comment
LOAD A, address	000 aaaaa	$A \leftarrow M[aaaaa]$	Load A with content of memory location aaaaa
STORE A, address	001 aaaaa	$M[aaaaa] \leftarrow A$	Store A into memory location aaaaa
ADD A, address	010 aaaaa	$A \leftarrow A + M[aaaaa]$	Add A with M[aaaaa] and store the result back into A
SUB A, address	011 aaaaa	$A \leftarrow A - M[aaaaa]$	Subtract A with M[aaaaa] and store result back into A
IN A	100 xxxxx	$A \leftarrow \text{input}$	Input to A
JZ address	101 aaaaa	if (A = 0) then PC = aaaaa	Jump to address if A is zero
JPOS address	110 aaaaa	if (A > 0) then PC = aaaaa	Jump to address if A is positive
HALT	111 xxxxx	Halt	Halt execution

Notations:

A = accumulator.
M = memory.
PC = program counter.
aaaaa = four bits for specifying a memory address.
x = don't cares.

图 10.18 EC-2 指令集

2) 数据通路-2

EC-2 的数据通路如图 10.19 所示，主要包括执行指令周期操作的部分和实现指令集中所有指令操作的部分。

数据通路中的执行指令周期操作的部分和之前的 EC-1 非常相似，包括指令寄存器（IR）、程序指针（PC）寄存器和 PC 的递增单元。二者细小的差别在于 PC 和递增单元的位宽：此处我们需要有 32 个位置的存储器，因此地址、PC 和递增单元的位宽均为 5 比特。

与 EC-1 相比，数据通路这一部分主要的改变是在 PC 的输出和存储器地址的输入之间的加入了一个 2 选 1 多路选择器。多路选择器的一个输入来自于 PC，另一个输入则来自 IR 的低四位， $IR_{4:0}$ 。原因是因为现在有两种不同类型的可访问存储器的操作。第一种仍然是取指操作，其访问的存储器地址由 PC 的内容给定。第二种则为这四个指令：LOAD、STORE、ADD、SUB，它们都是用地址作为操作数。因此，它们的存储器地址是由 IR 的低五位给定的。此 2 选 1 多路选择器的选通信号是 Meminst。

EC-2 的存储器大小增加到 32 个位置，因此需要 5 比特位宽，存储器还是作为数据通路的一部分，是微处理器的一个内部单元。为了既能满足把数据从存储器中读出的指令要求，也能满足需要把 A 的值赋给特定地址存储器的 STORE 指令，需要用一个 RAM 来代替之前的 ROM。为了实现 STORE 操作，我们把 A 的输出与存储器的数据输入 $D_{7:0}$ 相连。当信号 MemWr 有效时，则把 A 的值写入内存的指令地址。

存储器的输出 $Q_{7:0}$ 与 IR 的输入相连，并通过 4 选 1 多路选择器与 A 的输入相连。正如 EC-1 的设计一样，与 IR 的输入相连是为了取指操作。而与累加器 A 的相连是为了执行 LOAD 的指令，即把特定地址存储器的数据赋给 A。因为存储器输出只是向累加器 A 赋值的源中的一个，因此多路选择器是需要的。

数据通路的实现指令集中所有指令操作的电路部分包括 8 比特累加器 A、8 比特加-减单元和 4 选 1 多路选择器。加-减单元是为了执行 ADD 和 SUB 指令。当 Sub 信号置 1 时执行减操作，置 0 时执行加操作。4 选 1 多路选择器允许 A 的输入来自 3 个源。对于 ADD 和 SUB 指令，A 的输入来自于加-减单元的输出，对于 IN 指令，A 的输入来自于外部输入端，对于 STORE 指令，A 的输入来自存储器的输出。多路选择器的选通信号为 $Asel_{1:0}$ ，第四个多路选择端没有使用。

与 EC-1 相似，整个数据通路的输出端直接与 A 的输出相连，在输出端累加器的值总是有效的，因而对于 A 的输出值无须特定的输出指令。

对于两个条件跳转指令 JZ 和 JPOS，数据通路提供给状态机两个反馈信号，Aeq0 和 Apos。分别由两个比较器产生，如果 A 的值是 0，则 Aeq 信号输出为 1，故只需一个 8 输入 NOR 门。如果 A 的值是正数，则 Apos 为 1，因为 A 是补码表示，首位为 0 表示正数，首位为 1 表示负数，所以 Apos 只须等于 A_7 的取反信号。

综上所述，这个数据通路有八个控制信号，IRload, JMPmux, PCload, Meminst, MemWr, $Asel_{1:0}$, Aload 和 Sub。数据通路提供 2 个反馈信号，Aeq0 和 Apos。用来产生执行指令周期操作和指令集操作的控制字的控制单元将在后续部分讨论。

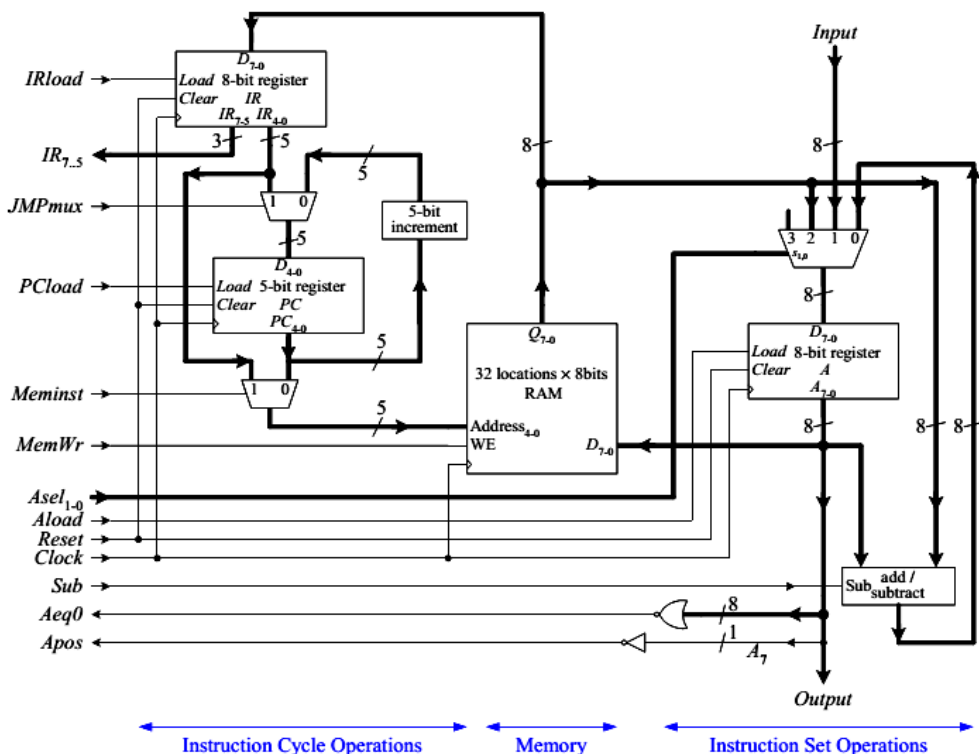


图 10.19 EC-2 的数据通路

3) 控制单元-2

控制单元的状态图如图 10.20 (a) 所示，在每个状态中控制信号的具体设置如图 10.20 (d) 所示。前三个状态 START、FETCH 和 DECODE 的设置和之前的 EC-1 控制单元一样，但是，EC-2 的 DECODE 状态需要通过分支到 8 个不同的状态以便执行对应的 8 个指令来解码 8 位指令编码。如前一样，指令编码的解码依赖于 IR 的前三位。

与 EC-1 不同的是，EC-2 三个指令 LOAD、ADD 和 SUB 需要涉及存储器访问问题。这里的问题是只有取完这些指令后，这些指令中所包含的地址才是可利用的，进一步说，只有解码指令之后，控制单元才知道存储器哪个特定地址是需要被读取的。正常情况下，为了使得指令能够实现对其中一个存储器特定地址的访问，在 DECODE 状态和具体指令执行状态之中会插入一个额外的存储器访问状态，保证存储器将有一个时钟周期输出数据，而具体指令将在下一个时刻执行。当然这是假设存储器只需要一个时钟周期去进行读操作，如果存储器访问需要更多时间，则在 DECODE 状态和执行状态之中需要更多的状态插入保证存储器的数据访问。

在我们的设计中，为了最小化状态的数量，我们使用 DECODE 状态执行完成存储器特定地址读取的操作，这样，当控制单元到达执行状态时，存储器已经准备好了数据。至于来自于存储器的数据是否实际使用将取决于正在执行的指令。如果指令不需要来自存储器的数据，则它被忽略，否则的话数据将被读取。在此设计中，这一方法可行是因为它与我们指令集中的其余指令的操作并不冲突。通过设置 DECODE 状态的 Meminst 信号有效，

使得存储器读取操作执行，读者可参见图 10.20（d）的输出表，它反映了在 DECODE 状态时 Meminst 栏为 1。

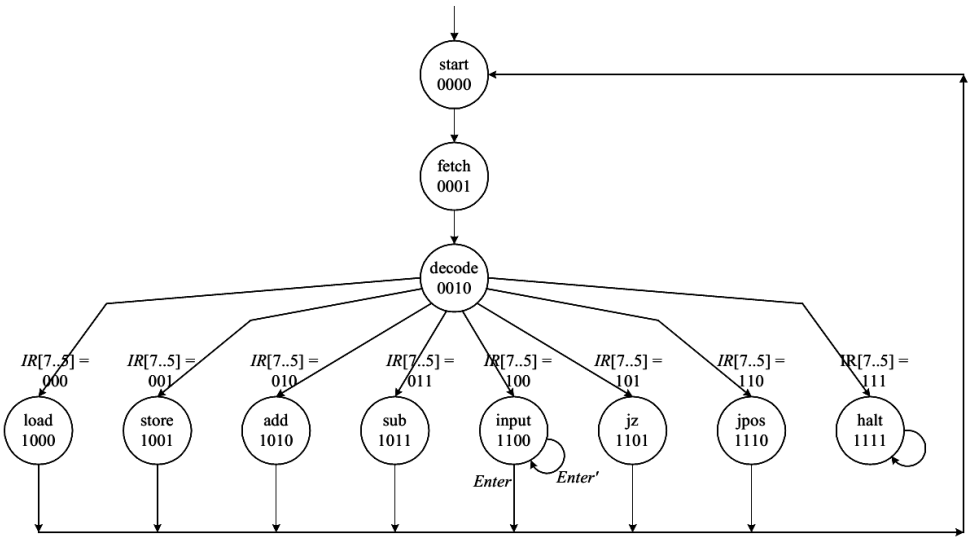
每个指令实际操作的完成是通过使控制数据通路的操作控制信号正确的使能（有效）。每个指令控制信号的设置如图 10.20（d）中的相应行所示。在此，通过观察数据通路的操作，读者应能够明白为什么每个配置是这样设置的。举个例子，对于 LOAD A 指令，Asel₁ 信号需要置 1 而 Asel₀ 需要置 0，为了多路选择器输入端口 2 使得存储器输出能到达 A 的输入。而 A 的实际加载是通过使 Aload 信号置 1 实现的。为了执行 STORE A 指令，需要通过使 Meminst 置 1 使得存储器地址来自于 IR，当 MemWr 置 1 时，则将写入存储器。

在跳转到 START 状态前，状态表的 INPUT 状态将等待 ENTER 键信号。这样，如果程序中有多个输入状态，我们能够正确的读取多个值。请注意在 ENTER 信号置 1 之后，是没有等待 ENTER 信号复位的状态的，即为了等待 ENTER 置 0 释放的状态。因此，在每次按下 ENTER 键时，需要一个电路产生只持续一个时钟长度的有效按键信号。

次态表和来源于次态表的激励方程分别如图 10.20（b）和（c）所示。由于篇幅限制，没有列出所有输入信号的可能组合。除了 INPUT 状态的所有状态，都取决于 IR 的前 3 位，而 INPUT 状态只取决于 Enter 信号，表中的空白项可看作均为 0。控制单元的电路可用 4 个 D 触发器实现。

输出方程如图 10.20（e）所示，它可直接从输出表（d）直接推导出。

最后，基于激励方程和输出方程我们可以得到控制单元的电路。EC-2 微处理器的完整的控制单元的电路如图 10.20（f）所示。



(a)

Current State $Q_3Q_2Q_1Q_0$	Next State $Q_{3next}Q_{2next}Q_{1next}Q_{0next}$ (Implementation $D_3D_2D_1D_0$)									
	IR_7, IR_6, IR_5								Enter	
	000 load	001 store	010 add	011 sub	100 input	101 jz	110 jpos	111 halt	0	1
0000 start	0001	0001	0001	0001	0001	0001	0001	0001		
0001 fetch	0010	0010	0010	0010	0010	0010	0010	0010		
0010 decode	1000	1001	1010	1011	1100	1101	1110	1111		
1000 load	0000	0000	0000	0000	0000	0000	0000	0000		
1001 store	0000	0000	0000	0000	0000	0000	0000	0000		
1010 add	0000	0000	0000	0000	0000	0000	0000	0000		
1011 sub	0000	0000	0000	0000	0000	0000	0000	0000		
1100 input									1100	0000
1101 jz	0000	0000	0000	0000	0000	0000	0000	0000		
1110 jpos	0000	0000	0000	0000	0000	0000	0000	0000		
1111 halt	1111	1111	1111	1111	1111	1111	1111	1111		

(b)

$$D_3 = Q_3'Q_2'Q_1Q_0' + Q_3Q_2Q_1'Q_0'Enter' + Q_3Q_2Q_1Q_0$$

$$D_2 = Q_3'Q_2'Q_1Q_0'IR_7 + Q_3Q_2Q_1'Q_0'Enter' + Q_3Q_2Q_1Q_0$$

$$D_1 = Q_3'Q_2'Q_1'Q_0 + Q_3'Q_2'Q_1Q_0'IR_6 + Q_3Q_2Q_1Q_0$$

$$D_0 = Q_3'Q_2'Q_1'Q_0' + Q_3'Q_2'Q_1Q_0'IR_5 + Q_3Q_2Q_1Q_0$$

(c)

State $Q_3Q_2Q_1Q_0$	$IRload$	$JMPmux$	$PCload$	$Meminst$	$MemWr$	$Asel_{1,0}$	$Aload$	Sub	$Halt$
0000 start	0	0	0	0	0	00	0	0	0
0001 fetch	1	0	1	0	0	00	0	0	0
0010 decode	0	0	0	1	0	00	0	0	0
1000 load	0	0	0	0	0	10	1	0	0
1001 store	0	0	0	1	1	00	0	0	0
1010 add	0	0	0	0	0	00	1	0	0
1011 sub	0	0	0	0	0	00	1	1	0
1100 input	0	0	0	0	0	01	1	0	0
1101 jz	0	1	$Aeq0$	0	0	00	0	0	0
1110 jpos	0	1	$Apos$	0	0	00	0	0	0
1111 halt	0	0	0	0	0	00	0	0	1

(d)

$$IRload = Q_3'Q_2'Q_1'Q_0$$

$$JMPmux = Q_3Q_2Q_1'Q_0 + Q_3Q_2Q_1Q_0'$$

$$PCload = Q_3'Q_2'Q_1'Q_0 + Q_3Q_2Q_1'Q_0Aeq0 + Q_3Q_2Q_1Q_0'Apos$$

$$Meminst = Q_3'Q_2'Q_1Q_0' + Q_3Q_2'Q_1'Q_0$$

$$MemWr = Q_3Q_2'Q_1'Q_0$$

$$Asel_1 = Q_3Q_2'Q_1'Q_0'$$

$$Asel_0 = Q_3Q_2Q_1'Q_0'$$

$$Aload = Q_3Q_1'Q_0' + Q_3Q_2'Q_1$$

$$Sub = Q_3Q_2'Q_1Q_0$$

$$Halt = Q_3Q_2Q_1Q_0$$

(e)

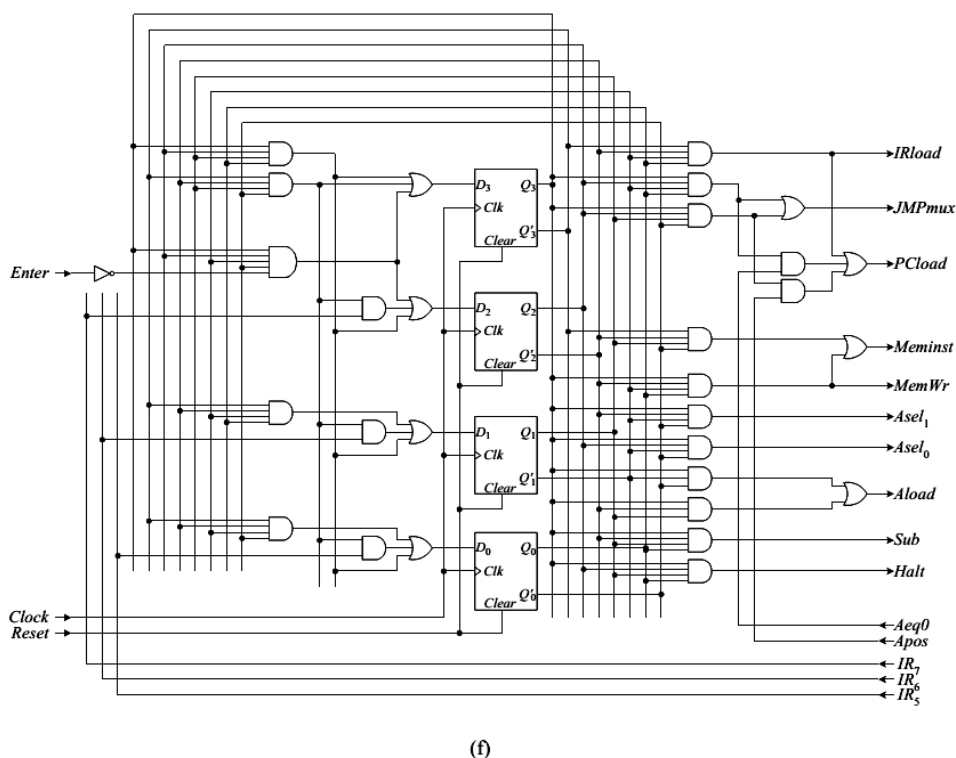


图 10.20 EC-2 的控制单元

(a) 状态图; (b) 次态表; (c) 激励方程; (d) 控制字/输出表; (e) 输出方程; (f) 电路图

4) 完整电路-2

通过连接图 10.19 的数据通路和图 10.20 (f) 的电路图并结合设计的控制和状态信号, 可以得到 EC-2 微处理器的完整电路。如图 10.21 所示。

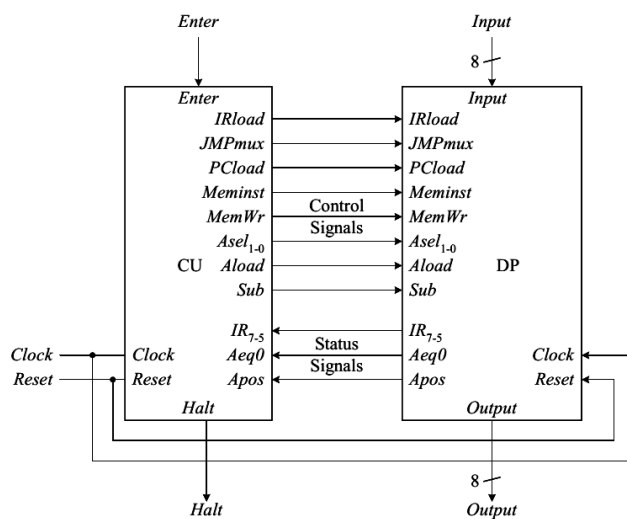


图 10.21 EC-2 微处理器完整电路

5) 示例程序-2

我们的微处理器的内存将用文件 PROGRAM.MIF 的内容进行初始化。示例文件如图

10.22 所示。文件包含 3 个程序：GCD 为计算两个输入的最大公约数的程序；SUM 计算 1 至 n 之间所有数的和的程序；COUNT 展示把输入 n 减至 0 的程序。

```
-- Content of the RAM memory in the file PROGRAM.MIF

DEPTH = 32; -- Depth of memory: 5-bit address
WIDTH = 8;  -- Width of memory: 8-bit data

ADDRESS_RADIX = BIN;-- All values in binary (HEX, DEC, OCT, BIN)
DATA_RADIX = BIN;

-- Opcodes for the EC-2
-- 000 = load A,aaaaa
-- 001 = store A,aaaaa
-- 010 = add A,aaaaa
-- 011 = sub A,aaaaa
-- 100 = in A
-- 101 = jz aaaaa
-- 110 = jpos aaaaa
-- 111 = halt

-- Specify the memory content.

-- Format of each memory location is
--      address : data

CONTENT
BEGIN
[00000..11111] : 00000000; -- Initialize locations range 00-1F to 0000

-----
-- There are three programs listed below: GCD, Sum, and Count
-- Only the program listed last is ran.
-- To try out a different program, move the code for the program you
-- want to the end of the list, re-compile, and download to the FPGA
-----

-- GCD
-- Program to calculate the GCD of two numbers

00000 : 10000000; -- input A
00001 : 00111110; -- store A,x
00010 : 10000000; -- input A
00011 : 00111111; -- store A,y

00100 : 00011110; -- loop: load A,x      -- x = y?
00101 : 01111111; -- sub A,y
00110 : 10110000; -- jz out              -- x=y
00111 : 11001100; -- jp xgty             -- x>y

01000 : 00011111; -- load A,y            -- y>x
01001 : 01111110; -- sub A,x              -- y-x
01010 : 00111111; -- store A,y
01011 : 11000100; -- jp loop

01100 : 00011110; -- xgty: load A,x      -- x>y
01101 : 01111111; -- sub A,y              -- x-y
01110 : 00111110; -- store A,x
01111 : 11000100; -- jp loop

10000 : 00011110; -- load A,x
10001 : 11111111; -- halt

11110 : 00000000; -- x
11111 : 00000000; -- y

-----
-- SUM
-- Program to sum N downto 1

00000 : 00011101; -- load A,one          -- zero sum by doing 1-1
00001 : 01111101; -- sub A,one
00010 : 00111110; -- store A,sum

00011 : 10000000; -- input A
00100 : 00111111; -- store A,n

00101 : 00011111; -- loop: load A,n      -- n + sum
00110 : 01011110; -- add A,sum
00111 : 00111110; -- store A,sum
```

```

01000 : 00011111; -- load A,n          -- decrement A
01001 : 01111101; -- sub A,one
01010 : 00111111; -- store A,n

01011 : 10101101; -- jz out
01100 : 11000101; -- jp loop
01101 : 00011110; -- out: load A,sum
01110 : 11111111; -- halt

11101 : 00000001; -- one
11110 : 00000000; -- sum
11111 : 00000000; -- n

-----
-- COUNT
-- Program to countdown from input N to 0

00000 : 10000000; -- input
00001 : 01111111; -- Sub A,11111
00010 : 10100100; -- jz 00100
00011 : 11000001; -- jp 00001
00100 : 11111111; -- halt
11111 : 00000001; -- constant 1

END;

```

图 10.22 包含 EC-2 的三段程序的 PROGRAM.MIF 文件

6) 硬件实现-2

与 EC-1 类似，图 10.23 给出了 EC-2 微处理器的输入输出设备之间的接口。输入由 8 个 DIP 开关组成，输出是 2 个七段数码管显示。使用一个 LED 来显示我们的微处理器是否已经停止工作，使用一个按钮作为复位按钮。当 Enter 开关每次被按压时，一个单触发电路用来产生一个时钟周期的脉冲。将系统输入的时钟进行分频可以看到一些中间结果的显示。

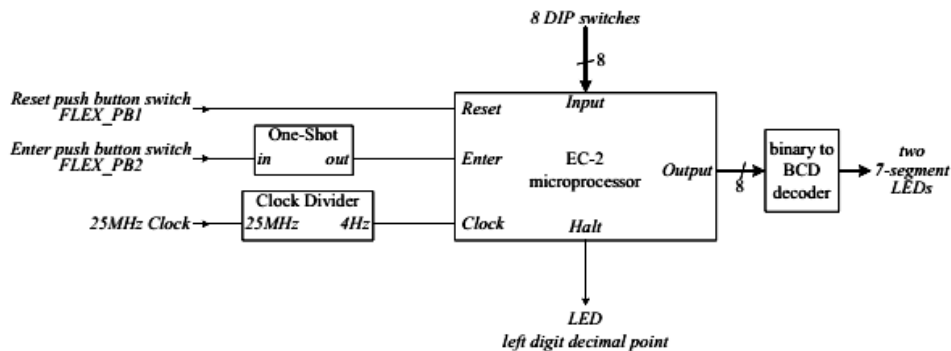


图 10.23 EC-2 的硬件实现

10.4. 微码型微处理器设计

CPU 是计算机的核心，它主要包括运算器和控制器两个部分，这两部分是由总线连接在一起，从设计的角度细分的话，还包括寄存器和外部 I/O 接口。在 CPU 内部，各部件之间的信号传输都是通过总线完成。一般来说，按信号的种类可将 CPU 内部的信号分为三类：1.数据信号，比如用于计算的实际数据。2.地址信号，地址是用于访问

内存 RAM 或 ROM 使用的。3.控制信号，由控制器生成用于控制 CPU 内各部件协同处理。

按总线的使用方式，有时可将 CPU 分为单总线和多总线等。单总线将只使用一个总线传输数据信号和地址信号。多总线 CPU 则各使用一个总线传输相应类型的信号。而控制信号一般是由控制器直接传送到各个部件的控制端口上，如果不从逻辑上将这些信号传输的线总称为控制总线，一般会忽视这个总线。

以下分别分析 CPU 设计中几大部件的细节。

1) 总线设计 数据总线和地址总线：处理 8 位数据的 CPU，它至少有 8 位的数据总线。由于如果使用单总线结构，8 位地址只能访问 256 字节数据和代码，这有点过小。因此，将地址总线独立出来，并使用 12 位的地址总线，这样一共可访问 4KB 代码和数据，对于一个手工搭建的计算机，这差不多够了，而且由于地址总线的独立性，未来的位数扩展也不会太复杂。例如，早期 Atari 2600 游戏机的 ROM 最多也只有 4KB，却开发出了很多非常不错的游戏。

另外，任一时刻，对于连接在总线的部件来说，只能有一个部件向总线输出数据，不然就会发生冲突，CPU 就无法继续运行。为了实现这一点，对于每一个连接在总线上的部件来说，都需要用一组三态缓冲器来控制其是否向总线输出数据，而每个部件所需的三态缓冲个数和这个部件的位数对应，每个三态缓冲控制 1 位数据的输出。所有这些部件的三态缓冲的协同工作，保证不发生冲突的任务是由控制器完成的。

关于 IO 接口部分：计划采用简单做法，即对每一个 IO 通道直接采用一个带三态缓冲（Tri-State Buffer）控制的 8 位寄存器（由 8 个 D flip-flop，即触发器构成）连接到数据总线上。并且不将 IO 设备映到内存空间（很多 ARM CPU 都是映射的），这样设计上会比较简单，而且功能上够用，也容易用未来实现的汇编程序控制 IO 设备。

2) 寄存器设计 寄存器个数的设计是比较自由的，比如 Nibbler 只有一个数据寄存器，那就是累加器，这样的设计会大大简化指令集，也会使硬件电路变得简单。

(1) A 寄存器，即累加器 Accumulator。用作 2 个 ALU 数据来源中的其中一个，并同时也用来保存 ALU 的计算结果。

(2) T 寄存器，即临时寄存器(Temporary Instruction)。用作 2 个 ALU 数据来源中的另一个，并同时也用来保存从 RAM 中取回的数据，即相当于 MD 寄存器(Memory Data)。

(3) B 寄存器，通用寄存器(General-Purpose Register)。用户可任意使用，也可用作栈顶指针。

另外，肯定还有以下寄存器用于完成取指和程序计数功能：

(4) IR 寄存器，即指令寄存器(Instruction Register)，8 bit。用于保存从内存中取回的待执行的指令。

(5) PC 寄存器，即程序计数器(Program Counter)，12 bit。用于保存下一个位于内存中的待取字节的地址。

(6) MA 寄存器，即内存地址寄存器(Memory Address)，12 bit。用于保存访问内存数据的地址。在实际实现时，这个寄存器可以简化去掉。

以上寄存器列表是初步设计，实际实现时可能略有删减。比如 Nibbler 的设计中没有 T、B、MA 这三个寄存器，即 Nibbler 只有一个数据寄存器：A 寄存器。

在逻辑电路设计阶段，8 bit 寄存器可用 8 个 flip-flop (12 bit 的则用 12 个 flip-flop)，即触发器并联而成。将每一个寄存器的输入端口都连接到相应的总线上，输出端口通过相应位宽的三态缓冲器再连接到对应总线上。由于 flip-flop 种类较多，为了简化主要使用 D flip-flop。

累加寄存器 A 通过三态缓冲连接到数据总线上。8 个三态缓冲可用 74HCT244 实现。

3) ALU 即算术逻辑单元 (Arithmetic Logic Unit)，用于完成 CPU 内的算术和逻辑运算，它是一个组合电路，无需时钟信号协助就能完成计算 (不同于时序电路，需要时钟信号)。算术运算主要包括：加、减、乘、除，逻辑运算主要包括：与、或、非、异或等。

算术运算中，加法是最基本的运算，而减法、乘法运算都可以化为加法运算。而逻辑运算中，与和非两个运算可以组合出其它的逻辑运算。因此，在指令数有限的情况下无需实现所有的算术和逻辑运算，只需选择部分便可。

关于 ALU 在 CPU 中是如何连接的：根据具体运算指令，ALU 需要 1 至 2 个 Operand，即操作数。Nibbler 的设计中，由于没有 T 寄存器，它是将 A 寄存器内的数和数据总线上的数直接计算，然后再将结果保存回 A 寄存器。这样的设计省掉了 T 寄存器的存和取，电路简化，计算速度反而更快，前提是这样的设计能满足实际需求。

74LS00 系列芯片中提供了一款 74LS181 芯片，它是一个多功能的 4 位 ALU，提供了加(ADD)、减(SUB)、移位(Shift)、比较大小(CMP)、与(AND)、或(OR)、异或(XOR)、与非(NAND)、或非(NOR)等共计 16 种 4 位数据运算。

4) 控制器 CU 即 Control Unit，是 CPU 中第二大部件，没有它的话，CPU 不能自动完成计算和取指等一系列操作。控制器的设计必须在完成所有以上步骤之后再开始，它肯定是 CPU 设计中的最后一个环节。控制器是时序电路，必须借助时钟信号来驱动 (区别于 ALU 的组合电路)。

控制器发出的每一组时序控制信号都和具体的指令运行阶段有关，因此必须先得完成指令集、以上各 CPU 部件的连接，设计好 Data path (即数据通道) 等工作后才能开始设计控制器。

控制器的实现方式有两种：

(1) 微代码 (Micro Code) 方式。这是早期 CISC 型 CPU 广泛采用的方式，比如 80X86 系列 CPU。

(2) 硬布线 (Hard Wired) 方式。这是后期 RISC 型 CPU 所广泛使用，比如 ARM 系列 CPU。

微代码的实现方式在 70~90 年代的 CPU 中使用较为常见。其优点是：比较灵活，定制能力强，当指令集发生某些变更无需改变硬件设计，而是通过重新设计微代码烧进 ROM 中即可。缺点是：速度相对较慢，因为它的每一次 CPU 状态都要访问 ROM 来输出该状态对应的控制信号，而访问 ROM 的速度肯定比直接运行门电路要慢。这个访 ROM 取控制信号的时间可能会是速个 CPU 运行环节中最慢的一环，因此可能直

接导致整体 CPU 的速度的下降。因为 CPU 的时钟频率必须要按最慢的那个部件的时间来设置。

硬布线方式在 90 年代以后的 RISC 型 CPU 中较上常见。其优点是：速度快，因为无需访问 ROM，而是直接通过逻辑门电路生成每一个 CPU 状态对应的控制信号。缺点是：它的门电路比较复杂，对于想用 7400 系列芯片搭建出来比较困难。比如，假设有 16 条指令，除了每条指令都会有固定的取指状态之外，不同的指令还会有：立即数寻址、直接寻址、执行等其它状态，假设平均每条指令有 3 个状态，那么一共将有 $16 \times 3 = 48$ 个状态。因此，控制器需要一个 6 位的计数器，即 Counter（为什么是 6 位：因为 48 介于 2 的 5 次方和 2 的 6 次方之间，5 位不够，6 位就够了）来生成足够 48 个的状态，然后还需要一个 6→64 的译码器，即 Decoder，将每一个状态都对应到唯一的输出信号端口上，而每一个输出信号端口则连接着不同 CPU 部件的控制端口。光是 6→64 的 Decoder 的引脚数就至少要 70 根以上，没有这样的 7400 芯片，但也许能找位数少的 Decoder 拼接出来，不过还是比较复杂的。

由于控制器的实现不同于 ALU 等部件，它完全是按照自定义的指令集和各各件之间连接的情况而定。如果使用硬布线方式，除了逻辑设计阶段我会用各种门来实现外，实际实现阶段也没有类似 74181 这样的芯片可用。当然，设计上是可以适当简化，使得方便使用现有的 7400 芯片去搭建控制器。这是个探索和折中的过程。是整个 CPU 设计过程中最复杂的部分。

Nibbler 采用的是微代码方式，它使用两片 28C16 EEPROM 芯片，共同输出 16 位的控制信号（即 16 位宽的控制总线）。

5) I/O 部分设计 I/O，即输入输出（Input/Output）功能，是完整的计算机必要的组成部分。这部分的设计宗旨是：尽可能简单可行。

6) 存储器部分设计 这部分就直接使用 RAM 和 ROM 芯片，将相应的地址引脚连接到地址总线上，如果实现中有 MA 寄存器，则 RAM 芯片的地址引脚先连接到 MA 寄存器的输出端口，MA 的输入端口则直连接至地址总线上。

需要考虑的一个问题是，存储架构是采用冯诺依曼架构，还是采用哈佛架构。

冯诺依曼架构是将数据和代码都存在一起，而哈佛架构则将两者分开存储。早期计算机几乎都采用冯氏架构，目前已有非常多的 CPU 和计算机采用哈佛架构了，比如 ARM 中的 Cortex-M 系列 CPU，被广泛应用到嵌入式计算中。

Nibbler 采用的是哈佛架构。从分析 Nibbler 的逻辑电路图来看，似乎它把 4KB 的 ROM 和 $4K \times 4\text{bit}$ 的 RAM 连接在了同一地址空间，总共提供了 4KB 或 $4K \times 4\text{bit}$ 的存储空间。

RAM: CY62256LL, SRAM, DIP-28 直插封装。

ROM: 28C64, EEPROM, 直插封装。

以上芯片容量均超过 4KB，实现时，多余的不使用的地址端口将被接地。具体请参考芯片的数据手册（Data Sheet）。

【例 3】Nibbler 4 Bit CPU

Nibbler 计算机已经是功能非常完整的计算机了，而且设计非常优化。它仅采用了 13 个 7400 系列芯片。驱动 1602 LCD 作为显示器，可以实现相对较为复杂的功能。而且作者开源的代码中包含了若干汇编代码示例，以及它的控制器实现采用的是微代码方式，同时作者还提供了微控制代码的生成程序，因此它可以作为一个 4 位的计算机设计模板，实现属于你自己指令集的计算机。

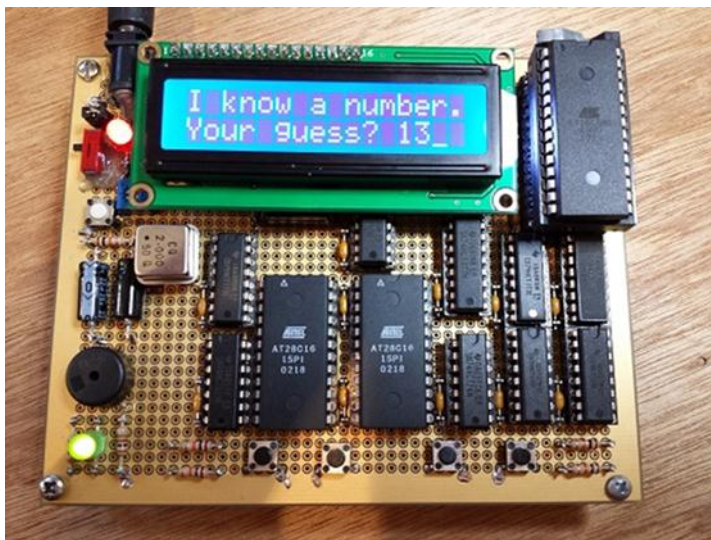


图 10.24 微码型 Nibbler 计算机的实物照片

1) Architecture 体系结构

主要的设计目标是让一切都容易理解，为了保持事情的简单性，有意省略了明显的性能优化。Nibbler CPU 的计算核心 4 位 ALU 是 74LS/HCT181 芯片，它的具体功能和引脚排列参见附录 A。然而，Nibbler CPU 有一个 12 位的地址空间和 16 字符×2 行的 LCD 显示器，不是仅仅几个闪烁的 LED。

图 10.27 展示了 Nibbler 的体系结构，采用哈佛架构设计。这意味着程序和数据存储在单独的地址空间中，并在单独的总线上运行。数据总线是 4 位宽的，所以是一个 4 位 CPU。程序总线为 8 位宽：4 位的指令操作码（opcode），4 位的直接操作数或目标地址的高位。

程序计数器（PC）提供程序 Prog.ROM 的地址。程序字节与其他状态位相结合，为微码 μ ROM 提供地址。这些 μ ROM 在适当的时间生成内部控制信号，用于 CPU 中其他芯片所需的置数（load）、使能（enable）和递增（increment）。每个 μ ROM 输出 16 个总控制信号中的 8 个。程序字节（program byte）也用于构造 RAM 和跳转（jump）目标的地址，以及将立即数放在数据总线上。

因为指令操作码是 4 位宽的，所以共有 16 种可能的指令类型。所有指令都需要两个时钟周期来执行，相位由寄存器 PHASE 控制。在第一个时钟周期，也称为 0 相（phase 0），指令操作码（opcode）和操作数（operand）从存储器中取出并存储在取指（Fetch）寄存器中。在第二时钟周期，称为 1 相（phase 1），执行指令所需的计算或操作。

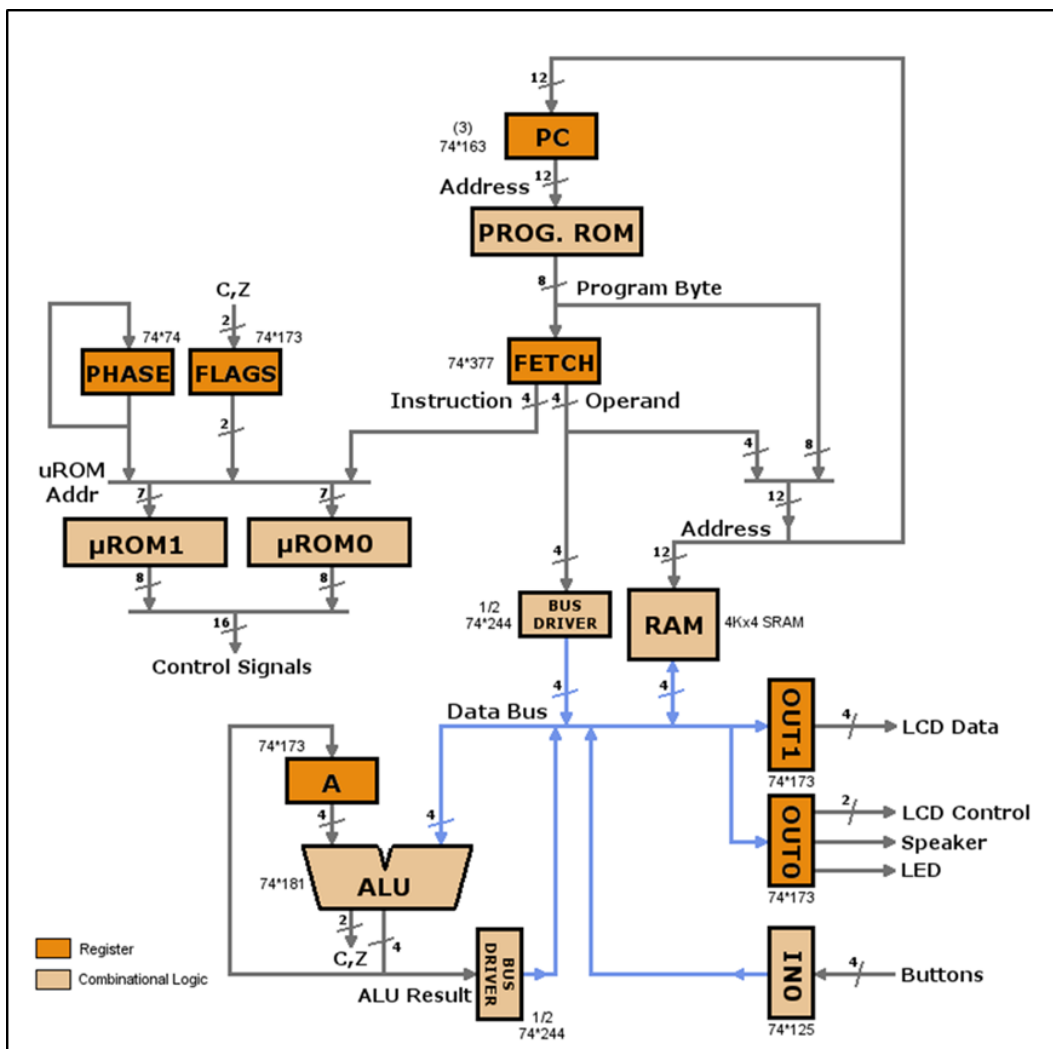


图 10.25 微码型 Nibbler 计算机的体系结构

采用 load-store 设计，将所有算术和逻辑计算结果存储到称为“A”的单个 4 位累加器寄存器中。数据可以在“A”和 RAM 中的内存位置之间移动，否则所有 CPU 指令只在“A”上运行。

程序和数据地址都是 12 位宽的，结果是总可寻址存储空间为：4096 个 8 位的字节程序，和 4096 个 4 位的半字节数据。由于指令操作码是 4 位宽，涉及绝对内存地址的指令是 $4+12=16$ 位的大小，或者是两个程序字节。仅涉及 4 位立即数的指令大小为 $4+4=8$ 位，或一个程序字节。

设计可以支持多达 32 个不同的 I/O 设备：16 个输入和 16 个输出。然而，Nibbler 只使用了 1 个 IN 端口和 2 个 OUT 端口。输入来自 4 个瞬时按键，分为上/下/左/右配置，并连接到单独的 IN 端口。输出利用两个 OUT 端口，包括 1 个用于调试的编程 LED，以及 1 个用于软件控制声音的压电扬声器，以及 1 个 6 位的 16×2 字符的 LCD 显示器接口。

2) Instruction Set 指令集

Nibbler 4 位 CPU 的指令有两种类型：立即数和可寻址的。这两种类型都以 4 位指令操作码开始，来识别特定指令。像 **ADDI** 这样的立即指令包括嵌入在指令中的 4 位操作数值，因此指令总共有 8 位。寻址指令如 **JMP** 包括在数据空间（RAM）或程序空间（ROM）的 12 位地址，指令长度总共有 16 位。指令编码非常简单：

type	7	6	5	4	3	2	1	0	7	6	5	4	3	2	1	0
addressed	i3	i2	i1	i0	a11	a10	a9	a8	a7	a6	a5	a4	a3	a2	a1	a0
immediate	i3	i2	i1	i0	o3	o2	o1	o0								

因为只有 4 位的指令操作码，最多有 16 个不同的指令，所以选择哪些指令来实施是具有挑战性的。如果要支持超过 16 个指令，则只能缩小地址位数或立即操作数位数。由于这是一个 4 位 CPU，4 位指令操作码是恰当的。

跳转指令有五种类型：

JMP – 无条件转移

JC – 如果进位则跳转

JNC – 如果无进位则跳转

JZ – 如果零则跳转

JNZ – 如果非零则跳转

这是一套很好的跳转指令，方便编程。虽然它们很方便，但是 **JNC** 和 **JNZ** 的负跳转指令不是绝对必要的，因为总是可以用正跳和无条件跳转来重写它们：

【例 3-1】**JNC** noCarry

```
carry:
    do stuff
noCarry:
    do other stuff
```

可以变为

```
JC carry
JMP noCarry
carry:
    do stuff
noCarry:
    do other stuff
```

LD 和 **ST** 指令从数据空间（RAM）加载一个半字节到累加器，或者从累加器存储一个半字节到数据空间。

LIT 指令加载一个立即数半字节值到累加器。它也可以被命名为 **LDI**。

OUT 指令从累加器中存储半字节到 16 个输出端口中的一个，而 **IN** 指令从 16 个输入端口中加载半字节到累加器中。这些指令用于 I/O。

ADDI 指令加半字节立即数值到累加器中，而 **ADDM** 将数据空间中的半字节加到累加器中。在这两种情况下，加法的进位输入（carry-in）是 0，而进位输出（carry-out）和零标志由加法的结果设置。

CMPI 指令将一半字节立即数与累加器进行比较（累加器减去数值），而 **CMPM** 将数据空间中的半字节与累加器进行比较。累加器的值没有被修改，但进位输出和零标志是由减法的结果决定的。因为 ALU 正在执行减法，所以进位输入总是为 1，以便使算术工作正确。

NORI 指令执行与立即数的 NOR（或非），**NORM** 执行与数据空间值的 NOR。进位输出（carry-out）和零标志是由 NOR 的结果设置的。

3) Schematic 原理图

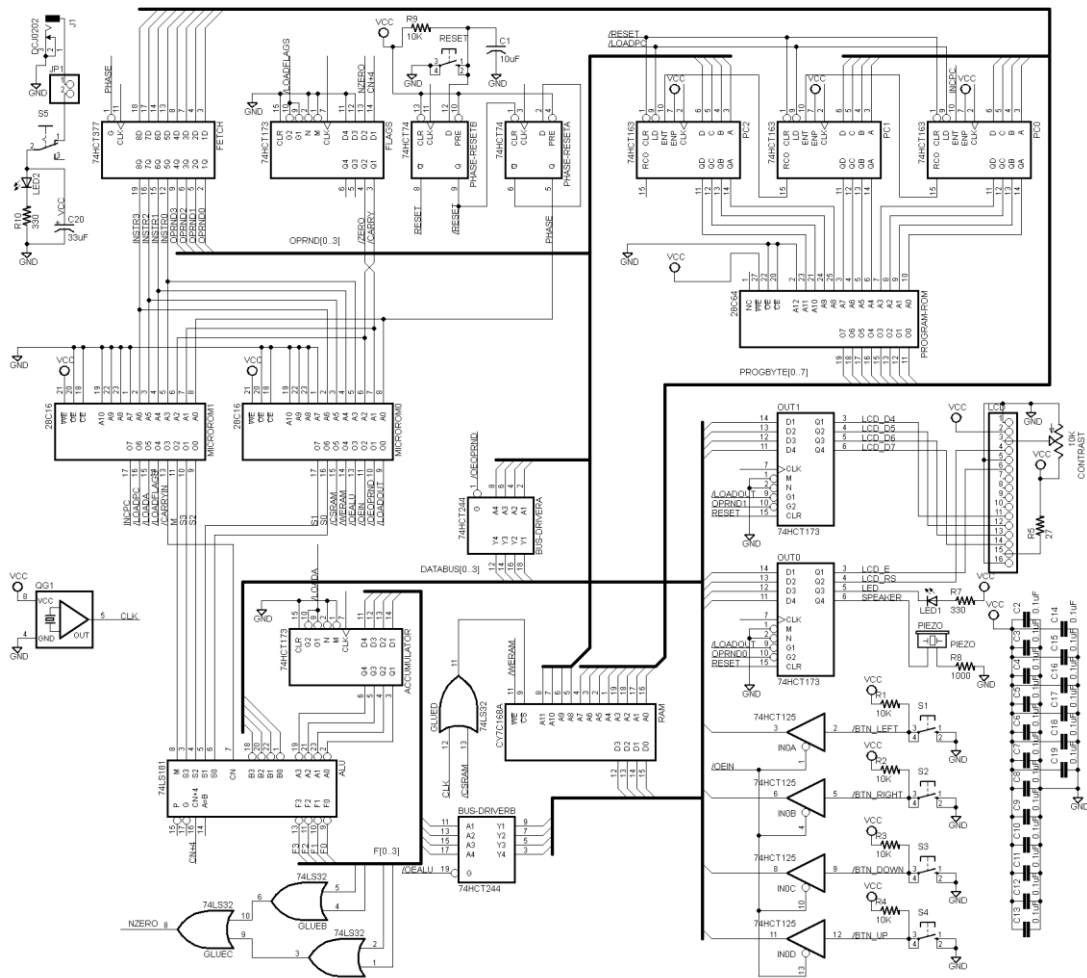


图 10.26 Nibbler 计算机的电原理图

除了 RAM 和 ROM 之外，所有芯片都是常见的 7400 系列的器件。大部分使用了 7400HCT 逻辑系列的器件，这样可以提供与常规 7400LS 系列器件类似的速度，而功耗更低。

Limitations 局限性

值得注意的是，一些东西 Nibbler 是没有的。它没有地址译码器，因为不需要将多个芯片映射到相同地址空间的不同区域。程序 ROM 占用了所有的程序地址空间，RAM

占用了所有的数据地址空间。I/O 外围设备不是内存映射的，而是使用指定端口的输入和输出指令来传输数据。

Nibbler 还没有任何地址寄存器，这意味着它不能支持任何形式的间接寻址，也不支持硬件控制的堆栈。所有内存引用都必须使用绝对地址。这虽是一个重大的限制，但它符合项目的设计目标。使用跳转表和专用内存位置，Nibbler 程序可以实现一个简单的调用/返回机制，而不需要一个真正的堆栈。

4) 硬件电路与功能分析

(1) Program Data 程序数据

原理图上的器件布局与结构图的相对位置相同。在中间右边是程序 Prog.ROM (28C64)，其中存储当前正在运行的程序。这是一个 8K×8 的 EEPROM，但 Nibbler 的地址大小只允许 4K 的程序，所以地址线的 A12 输入脚未使用，直接硬连线到地 0。程序存储器是 8 位宽的，因此 ROM 的所有 8 个 I/O 线都被使用。根据指令的类型，这些指令可以是 4 位指令操作码加 4 位立即操作数；或者 4 位指令操作码加 4 位地址，再跟着 8 位的地址。在每个指令执行的开始时，这个程序字节被加载到取指 (Fetch) 寄存器中。

当前正在执行的程序指令的地址存储在程序计数器 (PC) 中。PC 由 3 个 4 位计数器 74HCT163 组成，串接在一起构成 12 位的逻辑寄存器。在大多数指令之后，PC 将递增指向下一条指令。对于跳转指令，PC 还可以置数为一个新的地址。该地址来自取指寄存器 (Fetch) 的操作数值 (高 4 位) 和程序 ROM 字节 (低 8 位)。

(2) Control and Microcode 控制与微码

在原理图的左上方是与当前指令的执行有关的 3 个芯片。取指寄存器 (Fetch) 是一个 8 位寄存器 74HCT377，高 4 位保存当前指令的操作码，低 4 位保存指令或地址的数据。ALU 标志 (Flags) 存储在 4 位标志寄存器 74HCT173 中。因为只有进位 (carry) 和零 (zero) 两个标志，所以 4 个比特中的 2 个未被使用。这组中的最后一个芯片是双 D 触发器 74HCT74。一个触发器用于同步复位信号，另一个是相位比特，它总是在 0 和 1 之间切换，以指示指令正在执行两个时钟周期中的哪一个。当相位为 0 时 (Phase 0)，在时钟周期结束时 Fetch 寄存器加载。

指令操作码 (4bit)、ALU 标志 (2bit) 和相位 (1bit) 组合形成两个微码 ROM (28C16) 的 7 位地址，如图左中部所示。两个 μ ROM 的输出构成了协调所有其它芯片行为所需的 16 个控制信号。微码存储在两个 2K×8 EEPROM 中，所以每个 μ ROM 上的 11 个地址输入中的 4 个未被使用，并且硬连线到 0。

(3) ALU Datapath 数据通道

在原理图的左下方是 74LS181 ALU 和累加器寄存器 “A” 74HCT173。ALU (算术和逻辑单元) 可以对其两个输入执行任一常见得算术或逻辑运算。在这个例子里，一个输入总是来自累加器，而另一个输入来自数据总线。ALU 的计算结果被存储回累加器中。ALU、累加器和数据总线都是 4 位宽的，这就是 Nibbler 成为 4 位 CPU 的原因。

74LS181 芯片的运算功能选择是由 S0~S3、M、Cn 这 6 个端口决定的，具体可查

阅它的 Data Sheet (附录 A)。比如想让 74LS181 芯片执行 A-B 运算, S0~S3 的值须分别为 0110, 同时 M 和 Cn 的值须为 0 和 0, 这样 A-B 的值就会从 ALU 芯片的 F 端口输出。Nibbler 用到的 74LS/HCT181 芯片功能, 分别如下:

ALU 功能	S0~S3 值	M 值	Cn 值
A	0000	0	1
NOR	0001	1	×
SUB	0110	0	0
ADD	1001	0	1
B	1010	1	×

(4) Carry-In and Carry Flag 进位输入和进位标志

如果仔细看, 你会看到 ALU 的进位输入 (carry-in bit) 是微码提供的控制信号, 而不是标志寄存器的进位标志 (carry flag)。这是一个微妙但重要的一点: 进位标志是算术指令的输出, 根据进位标志的设置/未设置, 则可以用来进行条件跳转, 但是它不会反馈到 ALU 中, 从而影响到后面的计算。这意味着, 当执行多个半字节成对相加时, 程序必须在每个半字节相加之后检查进位标志, 并且如果设置了进位标志, 则将额外的 1 加到下一个加法中。

这是一个特意的设计选择。如果进位标志确实连接到 ALU 的进位输入比特, 那么程序在执行任何单半字节相加之前都需要清除它, 而这比多个半字节相加更为常见。此外, 进位输入不能简单地硬连接到 0, 因为 CMP (比较) 指令需要进位为 1, 以便正常工作。所以进位必须由微码提供。

(5) RAM 随机存取存储器

RAM 芯片 CY7C168A 在原理图的底部中间位置。它的 I/O 线连接到数据总线, 地址来自取指寄存器 (Fetch) 的操作数 (作为高 4 位) 和程序 ROM 字节 (低 8 位)。这是一个 4Kx4 SRAM, 与 Nibbler 的地址大小和数据宽度匹配。

值得注意的是, CLK 信号与/CSRAM 控制信号相或, 输出连接到 RAM 的芯片选择输入端/CS。这意味着 RAM 将只在每个时钟周期的后半部分使能, 而且只有在微码专门使能它时才被使能。这是当写使能信号/WE 和 RAM 地址可能尚未有效时, 在时钟周期的上半部分防止错误写入 RAM 的简单方法。

(6) IN and OUT Ports 输入输出端口

输入和输出端口也连接到数据总线, 如原理图的右下方所示。IN0 是一个 4 位总线驱动器 74HCT125, 它用于输出有上拉电阻的四个按键的状态。因为只有一个 IN 输入端口, 所以没有对端口号进行译码, 这样这个 74HCT125 实际上会响应 IN 指令的任何端口号。如果添加更多的 IN 端口, 则需要额外的端口号译码逻辑。

两个输出端口 OUT 是 4 位寄存器 74HCT173。OUT1 连接到 16×2 字符的 LCD 显示器的数据总线 databus[4..7], 该 LCD 显示器采用常见的 HD44780 控制器。虽然该 LCD 控制器具有 8 位接口, 但它也可以在 4 位模式下工作, 在这种情况下只使用最高的 4 位 LCD 数据线总线。OUT0 将两条线连接到 LCD, 用于控制 LCD 数据传输所需的 RS 和 E 信号。来自 OUT0 的另外两条线, 一条连接到 LED 上, LED 灯的开/关转

换可以作为基本的调试辅助；另一条连接到一个扬声器，可以在软件中以位脉冲形式产生不同频率的简单方波音。

注意 74HCT173 有两个置数使能输入/G1 和/G2，并且两者必须都是低才能将数据加载到芯片。两个芯片的 G1 连接到/LOADOUT 控制信号。但与 IN 端口一样，输出 OUT 端口号不是完全译码的，以避免需要额外的译码逻辑。相反，端口号的比特 0 连接到 OUT0 的/G2，并且比特 1 连接到 OUT1 的/G2。这意味着 OUT0 实际上会响应比特 0 为 0 的任何端口号，并且 OUT1 对比特 1 为 0 的任何端口号进行响应。尽管这可能没有用处，使端口号比特 1 和 0 同时为 0，甚至可以同时置数这两个端口。

(7) Bus Drivers 总线驱动

数据总线上的最后两个元件是一对 4 位总线驱动器，如在原理图中心底部所示。这是一个 8 位驱动 74HCT244，分成两个部分。一半用于将 ALU 结果驱动到数据总线上，这是将数据存储到 RAM 或 OUT 端口时所必需的。另一半用于将操作数从取指寄存器驱动到数据总线上，这对包含立即常数的指令是必要的。

(8) Combinatorial Feedback Loops 组合反馈环路

如果仔细查看架构图和原理图，您可能注意到 ALU 输出有一个组合反馈环路，通过总线驱动器 B，返回到 ALU 的 B 输入端。如果 ALU 功能被设置为类似 A+B，而总线驱动 B 使能，那么显然会产生反馈问题，因为 ALU 的输出也将是它的输入。不太明显的是，即使 ALU 功能被设置为 A，也可以创建反馈环路，只是输入 B 上的值是逻辑无关的。仔细研究逻辑门是如何由晶体管构成的，可以证明逻辑上无关的并不总是意味着电气无关，数据总线上的瞬态无效电压可以将 ALU 锁定在一个恶性循环中，其中的无效输入会导致无效输出，进一步导致更多无效输入。这是第一版 Nibbler 的一个罕见问题，并通过更详细的讨论，这个问题最终被解决了。

根据芯片数据手册中获取的 SN74LS181 的内部逻辑的知识，反馈环路问题可在微码中修复，使用 0000 输出替代原来的 ALU 功能选择 1111 即可。从理论上讲，从不同逻辑家族或不同制造商生产的 74LS181 可能具有不同的内部逻辑，微码为基础的修复可能不起作用。如果您正在构建一个 Nibbler 克隆版并碰到这个问题，在数据总线和 ALU 的 B 输入之间可以放置一个像 74HCT7541 的施密特触发缓冲器。

5) Microcode and Instruction Set 微码设计

CPU 指令中，4 比特指令操作码 i[0..3]与 ALU 进位标志 C、零标志 Z 以及相位比特相结合，以形成两个微码 ROM 的 7 位地址。 μ ROM 的输出形成协调所有其他芯片行为所需的 16 个控制信号。微码 ROM 的内容如表 10.2 所示。

表的第一行显示，对于所有指令在第 0 相是相同的。这点很好，因为这时指令还没有从内存中读取，所以 CPU 甚至不知道它执行什么指令！微码将累加器 A 的内容驱动到数据总线上，即使它们不需要——如果总线浮空 CMOS 芯片可能会出现问题。此外，在第 0 相（phase 0），程序寄存器 ROM 字节硬件加载到取指寄存器（Fetch），并且微码（incPC=1）将程序计数器增长到下一字节。

第 1 相（phase 1）是所有有趣的工作发生的地方。

表 10.2 Nibbler 指令集和微码

instruction	i3	i2	i1	i0	/C	/Z	phase	incPC	/loadPC	/loadA	/loadFlags	/carryIn	M	S3	S2	S1	S0	/csRAM	/weRAM	/oeALU	/oeIn	/oeOprnd	/loadOut
any	x	x	x	x	x	x	0	1	1	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JC	0	0	0	0	0	x	1	0	0	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JC	0	0	0	0	1	x	1	1	1	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JNC	0	0	0	1	0	x	1	1	1	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JNC	0	0	0	1	1	x	1	0	0	1	1	1	0	0	0	0	0	1	1	0	1	1	1
CMPI	0	0	1	0	x	x	1	0	1	1	0	0	0	0	1	1	0	1	1	1	0	1	1
CMPM	0	0	1	1	x	x	1	1	1	1	0	0	0	0	1	1	0	0	1	1	1	1	1
LIT	0	1	0	0	x	x	1	0	1	0	0	dc	1	1	0	1	0	1	1	1	1	0	1
IN	0	1	0	1	x	x	1	0	1	0	0	dc	1	1	0	1	0	1	1	1	0	1	1
LD	0	1	1	0	x	x	1	1	1	0	0	dc	1	1	0	1	0	0	1	1	1	1	1
ST	0	1	1	1	x	x	1	1	1	1	1	1	0	0	0	0	0	0	0	0	1	1	1
JZ	1	0	0	0	x	0	1	0	0	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JZ	1	0	0	0	x	1	1	1	1	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JNZ	1	0	0	1	x	0	1	1	1	1	1	1	0	0	0	0	0	1	1	0	1	1	1
JNZ	1	0	0	1	x	1	1	0	0	1	1	1	0	0	0	0	0	1	1	0	1	1	1
ADDI	1	0	1	0	x	x	1	0	1	0	0	1	0	1	0	0	1	1	1	1	1	0	1
ADDM	1	0	1	1	x	x	1	1	1	0	0	1	0	1	0	0	1	0	1	1	1	1	1
JMP	1	1	0	0	x	x	1	0	0	1	1	1	0	0	0	0	0	1	1	0	1	1	1
OUT	1	1	0	1	x	x	1	0	1	1	1	1	0	0	0	0	0	1	1	0	1	1	0
NORI	1	1	1	0	x	x	1	0	1	0	0	dc	1	0	0	0	1	1	1	1	1	0	1
NORM	1	1	1	1	x	x	1	1	1	0	0	dc	1	0	0	1	0	1	1	1	1	1	1

【例 3-2】**ADDI** 指令是一个很好的典型例子。它的指令操作码是二进制 1010，或十六进制的 \$A。对于该指令，无论 C 或 Z 标志如何，都要使用相同的控制信号。/oeOprnd 将为 0，使能总线驱动器能够将立即操作数驱动到数据总线上，并连接到 ALU 的一个输入。ALU 的另一个输入总是与累加器寄存器相连。/carryIn、M、和 S[0..3] 控制信号将 ALU 设置成为加法模式中，不带进位输入。/loadA 将是 0，因此 ALU 结果将被存储回到累加器。/loadFlags 也将是 0，所以进位和零点标志将按加法的结果更新。

【例 3-3】**JZ**（零则跳转）指令是另一个很好的例子。它的指令操作码是二进制的 1000，或十六进制的 \$8。如果 Z 标志是 0（意思是 true），那么 incPC 将是 0（PC 不会递增），/loadPC 将是 0（PC 将被加载新地址）。因此，如果 Z 是 0 将跳转。如果 Z 标志为 1（意味着 false），那么 incPC 将是 1（PC 将递增），/loadPC 将是 1（PC 将不置数）。因此，如果 Z 标志为 1，CPU 只跳过目标地址字节，并前进到下一个指令，而不需要跳转。跳转指令不需要涉及 ALU，但它们将累加器值驱动到数据总线上，因此总线不会浮空。

注意表 10.2 中一些 ALU 控制信号与某些指令无关，并且以灰色显示，并标记为“dc”，表示不用关心（don't care）。

6) Synthetic Instructions 合成指令

那些不包括在表里的指令怎么办，比如减法、与、或、非？令人高兴的是，所有这些都可以根据现有的指令合成。编写的汇编器程序中包含了一些宏，因此它们可以用作内置指令。

【例 3-4】**NOT: NORI #0**

ORI x: NORI x; NORI #0

ORM x: NORM x; NORI #0

ANDI x: NORI #0; NORI ~x

SUBI: ADDI (~x+1)

通常无法合成的通用指令是 **ANDM** 和 **SUBM**，因为它们需要使用临时寄存器来保存数据空间值的补码。数据空间中的专用位置可以用于此目的，因为 Nibbler 只有

一个寄存器。

在许多情况下，通过一些思考可以完全消除对这些合成指令的需要。以检查输入中的特定比特为例。在一个带有 **ANDI** 指令的 CPU 上，类似这样：

```
【例 3-5】#define BIT2 $4 ; $4 = 0100
IN #0 ; 从端口 0 读数到累加器
ANDI BIT2
JNZ bit2is1
```

对于 Nibbler，你可能会想到用上面提到的合成指令重写这个 **ANDI**：

```
【例 3-6】#define NOT_BIT2 $B ; $B = 1011
IN #0 ; 从端口 0 读数到累加器
NORI #0
NORI NOT_BIT2
JNZ bit2is1
```

但这可以通过简化两个负 **NORI** 和 **JNZ** 指令为单个正 **JZ** 指令来缩短：

```
【例 3-7】#define NOT_BIT2 $B ; $B = 1011
IN #0 ; 从端口 0 读数到累加器
NORI NOT_BIT2
JZ bit2is1
```

7) Software Tools 软件工具

通过创建的一些软件工具来帮助，使得 Nibbler 的开发更容易。大多数都是用像 C# 的 .NET 语言编写的，使用 Windows 的免费 Visual Studio Express IDE。

(1) Microcode Builder 微代码生成器

最简单的工具是微码生成器，它为两个微代码 ROM 的内容生成两个二进制文件。开始时，打算在文本文件中用寄存器传输语言描述（RTL，register transfer language）的微码操作，然后微码 Builder 会把它汇编成二进制。利用微码 Builder 可以设计专属于你自己的定制 CPU。

但是，很明显微码也是相当简单的，放弃了使用 RTL 的想法，只是直接写了一些 C 代码来生成所需的二进制数据。该程序包含大量的比特移位功能：

```
// 9 JNZ
for (int c = 0; c < 2; c++)
{
    microcode[(9 << 3) | (c << 2) | (0 << 1) | 1] = 0xF03F; // Z
    microcode[(9 << 3) | (c << 2) | (1 << 1) | 1] = 0x303F; // NZ
}
```

这些十六进制数只是微码表中的行，16 个控制信号表示为 4 个半字节的十六进制值。

(2) Assembler 汇编器

Nibbler 汇编器用来为 CPU 编写程序，使用符号命名指令名称和分支标记。它最初是为 Tiny CPU 项目编写的，然后修改为 Nibbler 的目标。汇编器作为命令行程序运

行。它将一个.asm 文件作为输入，并将代码汇编成.bin 二进制输出文件。它还生成一个.Sym 符号文件，包含代码中所有标签和常量的值，以及每个代码行的汇编地址。符号文件稍后由模拟器使用，以便执行源级调试。

汇编器没有任何类似宏或条件编译的特殊功能，但它确实支持：

- 十进制常量
- 十六进制常数，前面是\$
- 字符常量，包含在单引号‘中
- <and >运算符提取字节常量的高或低半字节
- 常数命名使用 #define (数据和地址常量)
- 标记的命名，并跳转到一个命名的标记
- 未命名的+和-标记，用于向前和向后跳转
- 注释，开始;

对于汇编器能够处理的代码的一个很好例子，请看猜数字程序的源代码(旧版本)。

(3) Simulator 仿真器

最复杂的工具是机器仿真器。最初为 BMOW 1 开发的，这个基于 GUI 图形的工具模拟 CPU 的数据和控制路径。它支持源级符号调试、反汇编、微码调试、代码断点、数据断点、存储器检查以及 LCD 和输入按键的 I/O 仿真。

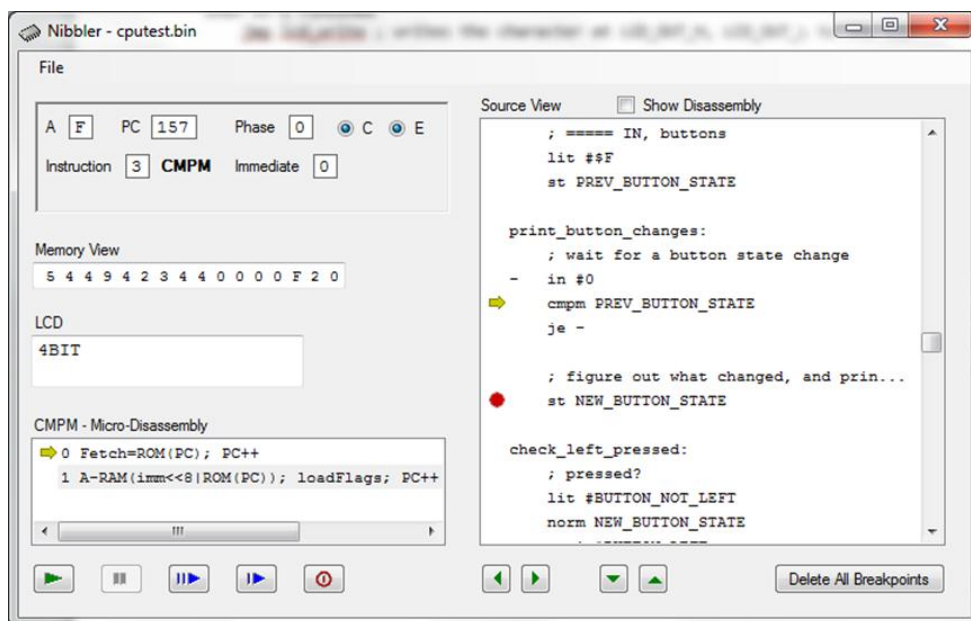


图 10.27 Nibbler 的仿真器界面

仿真模拟器使用程序二进制、程序源和符号文件（如果有）和微代码 ROM 二进制文件。在上面的图像中，仿真器正在对按键观察者示例程序进行源级调试。

当前执行 CMPM PREV_BUTTON_STATE 指令执行已停止，如黄色箭头所示。在两个指令后有红色圆圈的行，表示有一个断点。如果模拟器启动并且模拟按钮被按下，程序将从循环中中断并在断点处停止。（图像中的 JE 指令后来被替换为 JZ）。

8) 示例程序

【例 3-8】RAM 测试 (testram.asm)

```
; OUT ports
#define PORT_CONTROL $E

; memory locations
#define TEST_LOCATION $038

init:
    ; turn LED on
    lit #$0
    out #PORT_CONTROL

testram:
    lit #0

    addi #15
    st TEST_LOCATION
    cmpm TEST_LOCATION
    jnz fail

    addi #15
    st TEST_LOCATION
    cmpm TEST_LOCATION
    jnz fail

    addi #15
    st TEST_LOCATION
    cmpm TEST_LOCATION
    jnz fail

    addi #15
    st TEST_LOCATION
    cmpm TEST_LOCATION
    jnz fail

    addi #15
    st TEST_LOCATION
    cmpm TEST_LOCATION
    jnz fail

    addi #15
    st TEST_LOCATION
    cmpm TEST_LOCATION
    jnz fail

    addi #15
    st TEST_LOCATION
```



```

    cmpm TEST_LOCATION
    jnz fail

    addi #15
    st TEST_LOCATION
    cmpm TEST_LOCATION
    jnz fail

    addi #15
    st TEST_LOCATION
    cmpm TEST_LOCATION
    jnz fail

    addi #15
    st TEST_LOCATION
    cmpm TEST_LOCATION
    jnz fail

    addi #15
    st TEST_LOCATION
    cmpm TEST_LOCATION
    jnz fail

    addi #15
    st TEST_LOCATION
    cmpm TEST_LOCATION
    jnz fail

    addi #15
    st TEST_LOCATION
    cmpm TEST_LOCATION
    jnz fail

    jmp testram

fail:
    ; turn LED off
    lit #4
    out #PORT_CONTROL
    jmp fail

unreachable:
    jmp fail

```

9) Summary 小结

Nibbler 是一个 4 位的 CPU，用标准 7400 系列逻辑芯片组建，包括计数器、寄存器、缓冲器和门电路等。非常简单容易理解和自己动手组装，但能运行游戏和其它有趣的程序。Nibbler 装在一个 5×4 英寸(127×102 mm)大小的试验电路板上，这个 CPU 总共只有 13 个芯片。

相关详细资料请参考：<https://www.bigmessowires.com/nibbler/>

10) Specs 规格说明

- 4 位定制 CPU
- 12 位寻址，4K 地址空间大小
- 哈佛体系结构—独立的程序和数据存储空间
- 2.46 MHz 时钟速度，每条指令 2 个时钟=123 万个指令/秒
- 在 CPU 数据和控制通道中使用 13 个 7400 系列芯片
- 2 个 28C16 EEPROMs 用于存储产生内部控制信号的微代码
- 4K x 4 SRAM (CY7C168A)
- 28C64 EEPROM 存储程序
- 4 个按键输入
- 16 x 2 字符 LCD 显示器
- 1 个音频扬声器
- 5V 电源，大约 100 mA 电流

PCB 版 Nibbler 照片

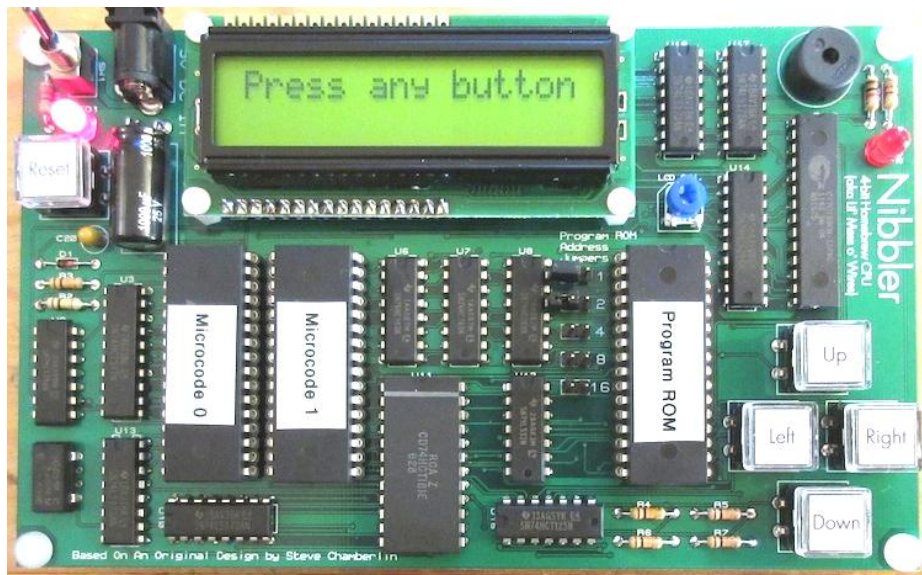


图 10.28 PCB 版 Nibbler

PCB 版 Nibbler 电路图和设计文件请参考 <https://github.com/pong74ls/Nibbler>

【注】<http://www.bigmessowires.com/nibbler.zip> 包含 Nibbler 所有的设计文件和工具。

10.5. 微处理器的 Verilog 实现

Verilog 语言是一种硬件描述语言，描述的是数字电路的硬件结构与实现。所以，与 C 语言等其它高级语言设计的差异在于，我们使用 Verilog 语言设计的时候，要做到心中有电路。心中有电路至少有两层含义：一方面，使用 Verilog 语言时，不能像更高级的语言那样随心所欲，而要接地气的考虑所写的语言是否是可综合的；另一方面，要建立的概念是，利用可综合的 Verilog 语句写出来的每一段代码，都对应于 FPGA 上实现的硬件电路单元，如多路选择器、状态机等。

如下我们看两段 Verilog 代码的例子，这两段代码都是后面 EC-2 微处理器的数据通路中要用到的。

```
/******  
module mux21 (d0,d1,JMPmux,D40);  
input [4:0] d0,d1;  
input JMPmux;  
output [4:0] D40;  
reg[4:0] D40;  
  
always @(d0 or d1 or JMPmux)  
if (JMPmux) D40=d1;  
else D40=d0;  
  
endmodule  
/******
```

这是一个 2 选 1 多路选择器的 Verilog 代码，输入信号为 d0，d1，均为 5 比特位宽的信号，若 JMPmux 为 1，则输出 D40 选择输入 d1，否则选择输入 d0。这段代码经过 ISE 综合后，得到的电路如图 10.29 所示，对每一位输入而言，通过 JMPmux 信号经过与门进行选择 d0 和 d1 中的对应位。由此可见，可综合的 Verilog 代码就对应于 FPGA 中实际的电路。

第二个例子是数据通路中的 PC 寄存器的 Verilog 代码：

```
/******  
module pc(D40,load,Clear,clk,pc_addr );  
input load, Clear, clk;  
input [4:0] D40;  
output reg [4:0] pc_addr;  
  
always @(posedge clk or posedge Clear)  
if(Clear)  
pc_addr<=5'b00000;  
else begin  
if(load) pc_addr<=D40;  
end  
endmodule  
/******
```

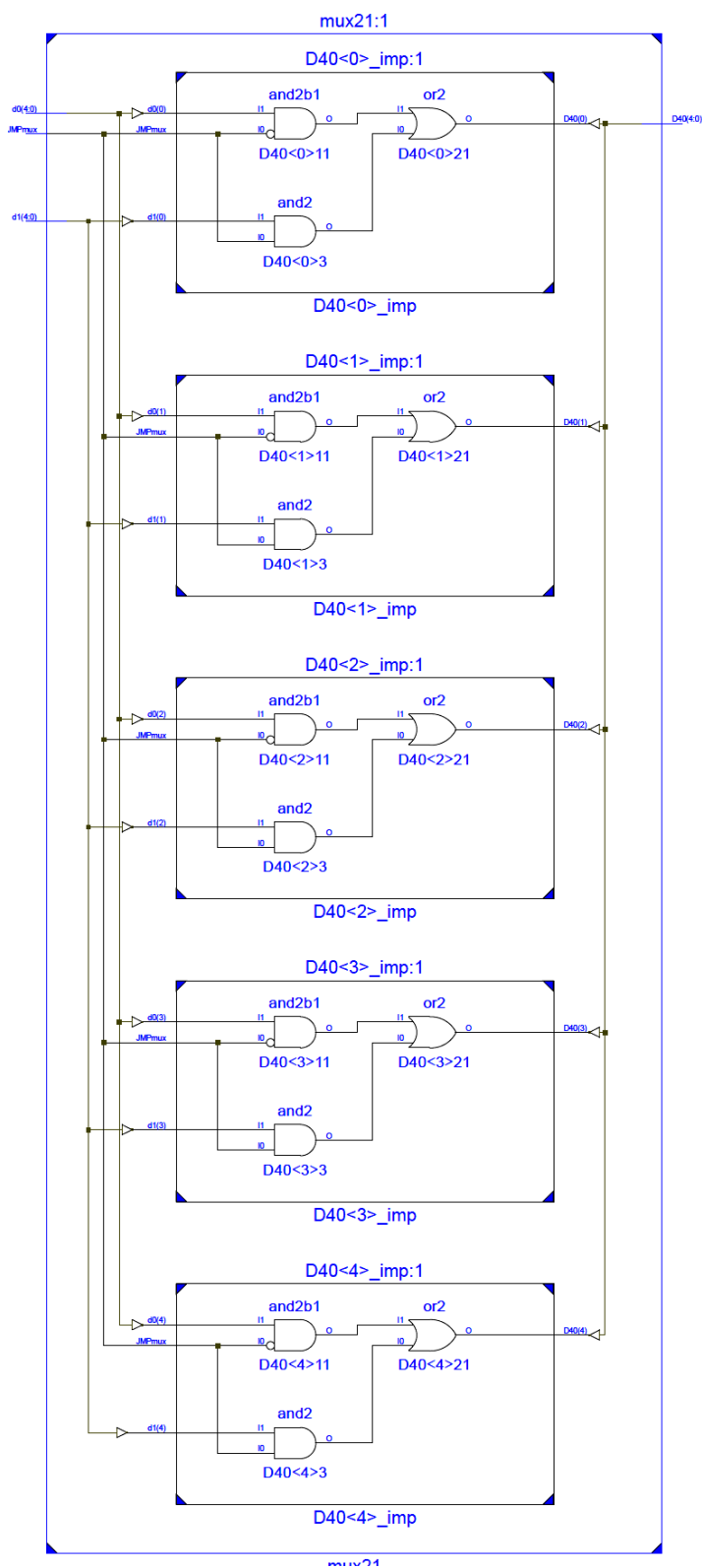


图 10.29 2-1 多路选择器 Verilog 代码综合后电路图

PC 寄存器的 Verilog 代码综合出来的电路如图 10.30 所示。由图可知，Verilog 代码综合后对应的就是一个带有清零和使能的 4 比特位宽的触发器。

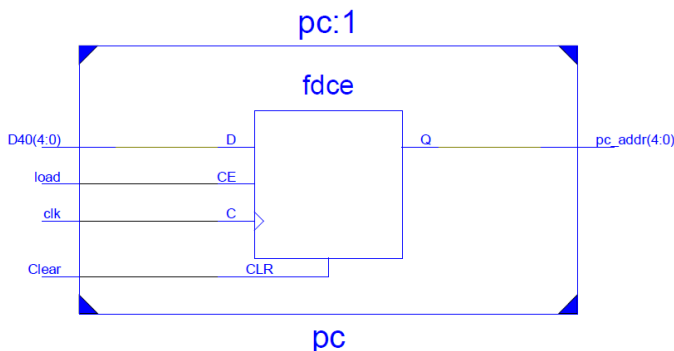


图 10.30 带使能和清零的程序指针 PC 寄存器 Verilog 代码综合出的电路

从上述两个例子看出，可综合的 Verilog 语句写出的代码，实际上就可以看成我们传统的各种分立器件芯片。因此，建立这一概念之后，我们前面的微处理器设计示例即可使用 Verilog 语句进行描述。

1) 微处理器 EC-2 的 Verilog 实现

在图 10.23 中，最主要的部分是 EC-2 微处理器。由图 10.21 可知，这一微处理器由 DP（数据通路）和 CU（控制单元）两部分组成。如下我们列出 CU 和 DP 两部分代码，并分别对代码进行解释。首先考察 CU 的代码如下：

```

/*****
module CU(Enter,Clock,Reset,Aeq0,Apos,IR75,IRload,JMPmux,PCLoad,
          Meminst,MemWr,Asel,Aload,Sub,Halt );
input Enter,Clock,Reset,Aeq0,Apos;
input [2:0] IR75;
output IRload,JMPmux,PCLoad,Meminst,MemWr,Aload,Sub,Halt;
output[1:0] Asel;

//定义状态编码
parameter [3:0] start=4'b0000, fetch=4'b0001, decode=4'b0010, load=4'b1000,
store=4'b1001, add=4'b1010, sub=4'b1011, inputt=4'b1100, jz=4'b1101, jpos=4'b1110,
halt=4'b1111;

reg [3:0] current_state, next_state;
reg IRload,JMPmux,PCLoad,Meminst,MemWr,Aload,Sub,Halt;
reg [1:0] Asel;

//采用同步时序描述状态转移
always @(posedge Clock or posedge Reset)
if(Reset)
current_state<=start;
else

```

```
current_state<=next_state;
```

```
//采用组合逻辑判断状态转移条件，描述状态转移规律
```

```
always @(current_state or IR75 or Enter)
```

```
begin
```

```
    case(current_state)
```

```
        start: next_state=fetch;
```

```
        fetch: next_state=decode;
```

```
        decode: if(IR75==3'b000) next_state=load;
```

```
                else if(IR75==3'b001) next_state=store;
```

```
                else if (IR75==3'b010) next_state=add;
```

```
                else if (IR75==3'b011) next_state=sub;
```

```
                else if(IR75==3'b100) next_state=inputt;
```

```
                else if(IR75==3'b101) next_state=jz;
```

```
                else if(IR75==3'b110) next_state=jpos;
```

```
                else next_state=halt;
```

```
        load: next_state=start;
```

```
        store: next_state=start;
```

```
        add: next_state=start;
```

```
        sub: next_state=start;
```

```
        inputt: if(Enter) next_state=start;
```

```
                else next_state=inputt;
```

```
        jz: next_state=start;
```

```
        jpos: next_state=start;
```

```
        halt: next_state=halt;
```

```
        default: next_state=start;
```

```
    endcase
```

```
end
```

```
//使用同步时序电路描述每个状态的输出
```

```
always @(posedge Clock or posedge Reset)
```

```
if (Reset)
```

```
    begin
```

```
        IRload<=0; JMPmux<=0; PCload<=0; Meminst<=0; MemWr<=0; Asel<=2'b00;
```

```
        Aload<=0; Sub<=0; Halt<=0;
```

```
    end
```

```
else
```

```
    begin
```

```
        case(next_state)
```

```
        start: begin
```

```
            PCload<=0; IRload<=0; JMPmux<=0; Meminst<=0; MemWr<=0; Asel<=2'b00;
```

```
            Aload<=0; Sub<=0; Halt<=0;
```

```
        end
```

```
        fetch: begin
```

```
            Meminst<=0; IRload<=1; JMPmux<=0; PCload<=1; MemWr<=0; Asel<=2'b00;
```

```
            Aload<=0; Sub<=0; Halt<=0;
```

```
        end
```

```
        decode: begin
```

```

        PCload<=0; IRload<=0; JMPmux<=0; Meminst<=1; MemWr<=0;
        Asel<=2'b00; Aload<=0; Sub<=0; Halt<=0;
    end
load: begin
    PCload<=0; IRload<=0; JMPmux<=0; Meminst<=0; MemWr<=0; Asel<=2'b10;
    Aload<=1; Sub<=0; Halt<=0;
end
store: begin
    PCload<=0; IRload<=0; JMPmux<=0; Meminst<=1; MemWr<=1; Asel<=2'b00;
    Aload<=0; Sub<=0; Halt<=0;
end
add: begin
    PCload<=0; IRload<=0; JMPmux<=0; Meminst<=0; MemWr<=0; Asel<=2'b00;
    Aload<=1; Sub<=0; Halt<=0;
end
sub: begin
    PCload<=0; IRload<=0; JMPmux<=0; Meminst<=0; MemWr<=0; Asel<=2'b00;
    Aload<=1; Sub<=1; Halt<=0;
end
inputt: begin
    PCload<=0; IRload<=0; JMPmux<=0; Meminst<=0; MemWr<=0; Asel<=2'b01;
    Aload<=1; Sub<=0; Halt<=0;
end
jz: begin
    PCload<=Aeq0; IRload<=0; JMPmux<=1; Meminst<=0; MemWr<=0; Asel<=2'b00;
    Aload<=0; Sub<=0; Halt<=0;
end
jpos:begin
    PCload<=Apos; IRload<=0; JMPmux<=1; Meminst<=0; MemWr<=0; Asel<=2'b00;
    Aload<=0; Sub<=0; Halt<=0;
end
halt: begin
    PCload<=0; IRload<=0; JMPmux<=0; Meminst<=0; MemWr<=0; Asel<=2'b00;
    Aload<=0; Sub<=0; Halt<=1;
end
default: begin
    PCload<=0; IRload<=0; JMPmux<=0; Meminst<=0; MemWr<=0;
    Asel<=2'b00; Aload<=0; Sub<=0; Halt<=0;
end
endcase
end
endmodule
/*****

```

以上为 CU 的代码，代码开始部分依次为输入输出接口定义，以及状态编码的定义，其中输出端口定义即为控制字。接下来即为 CU 的主体部分的状态机，此处我们采用了三段式状态机的写法。第一段的 always 语句，对应的是状态机的状态寄存器在每一个有效时钟沿到达时刻，状态由当前状态转换到下一状态的时序逻辑电路。第二

段的 always 语句，对应的是次态逻辑产生的组合逻辑电路，使用 case 语句，根据当前的状态和跳转条件，决定了下一个状态。第三段 always 语句，对应的是各个控制字的输出，此处使用了寄存器化的输出从而减少输出毛刺，所以此处 case 语句的输出条件为下一状态的状态值，而非当前状态的状态值。

DP 的 Verilog 代码如下：

```

/*****
module DP (IRload,JMPmux,PCload,Meminst,Reset,Clock,IR75,MemWr,Input,
          Asel,Aload,Output,Apos,Aeq0,Sub );

input IRload,JMPmux,PCload,Meminst,Reset,Clock,MemWr,Aload,Sub;
input [7:0] Input;
input [1:0] Asel;
output [2:0] IR75;
output [7:0] Output;
output Aeq0,Apos;
wire [7:0] w7,w8,w1;
wire [4:0] w2,w3,w4,w5,w6;

//实例化下列子模块
ir ir1(.D(w1),.load(IRload),.Clear(Reset),.clk(Clock),.IR75(IR75),.IR40(w2));
mux21 mux1(.d0(w3),.d1(w2),.JMPmux(JMPmux),.D40(w5));
pc pc1(.D40(w5),.load(PCload),.Clear(Reset),.clk(Clock),.pc_addr(w4));
increment increment1(.i1(w4),.i2(w3));
mux21 mux2(.d0(w4),.d1(w2),.JMPmux(Meminst),.D40(w6));
ramm ramm1(.data(Output),.addr(w6),.qout(w1),.we(MemWr),.clk(Clock),.reset(Reset));
mux41 mux3(.d0(w7),.d1(Input),.d2(w1),.Asel(Asel),.D70(w8));
a a1(.load(Aload),.Clear(Reset),.clk(Clock),.D(w8),.Output(Output),.Apos(Apos),.Aeq0(Aeq0));
sub sub1(.d(Output),.q(w1),.sub(Sub),.ot(w7));

endmodule
*****/

```

上述程序定义了 DP 的输入输出后，实例化了 9 个子模块，即对应于图 10.19 中的指令寄存器 ir，2 选 1 多路选择器 mux21，程序指针寄存器 PC，累加单元，程序存储器 ramm，4 选 1 多路选择器 mux41，累加器 a，加法-减法器 sub。对应于这些子模块的 Verilog 代码如下，其中 mux21 和 PC 的 Verilog 代码以及综合出来的电路已在前面给出，此处不再列出。

```

/*****
//IR 模块程序
module ir(D,load,Clear,clk,IR75,IR40 );
input load,Clear,clk;
input [7:0] D;
output reg [2:0] IR75;
output reg [4:0] IR40;
*****/

```



```

always @(posedge clk)
if(Clear) begin
    IR75<=0;
    IR40<=0;
end
else if(load) begin
    IR75<=D[7:5];
    IR40<=D[4:0];
end
endmodule

```

```

//RAM 模块
module ramm(data,addr,qout,we,clk,reset);
input[7:0]data;
input reset;
input [4:0]addr;
input we,clk;
output[7:0] qout;
reg[7:0]qout;
parameter memsize=32;
reg[7:0] ramm[memsize-1:0];
integer i;

```

```

always @(posedge clk or posedge reset )
if(reset) begin
//对 RAM 初始化 COUNT 指令
    ramm[0]<=8'b10000000;
    ramm[1]<=8'b01111111;
    ramm[2]<=8'b10100100;
    ramm[3]<=8'b11000001;
    ramm[4]<=8'b11111111;
    ramm[31]<=8'b00000001;
end
else begin
    if(we)
        ramm[addr]<=data;
    else
        qout<=ramm[addr];
    end
end
endmodule

```

```

//4 选 1 多路选择器模块
module mux41(d0,d1,d2,Asel,D70);

input [7:0]d0,d1,d2;
input[1:0]Asel ;
output reg [7:0] D70;

```

```

always @(d0 or d1 or d2 or Asel)
if(Asel==2'b00) D70=d0;
else if(Asel==2'b01) D70=d1;
else D70=d2;
endmodule

```

//寄存器 A 模块

```

module a(load,Clear,clk,D,Output,Apos,Aeq0 );
input load,Clear,clk;
input [7:0] D;
output reg [7:0] Output;
output Apos,Aeq0;

```

```

always @(posedge clk)
if(Clear)
    Output<=0;
else begin
    if(load)
        Output<=D;
    end
assign Apos=~Output[7];
assign Aeq0=(Output==0)? 1:0;
endmodule

```

//sub & add 模块

```

module sub(d,q,sub,ot );
input [7:0] d,q;
input sub;
output reg [7:0] ot;

```

```

always @(d or q or sub)
if(sub)
ot=d-q;
else
ot=d+q;
endmodule

```

/*****

将 CU 和 DP 模块连接起来，即可得到微处理器，Verilog 代码如下：

/*****

```

module ec2top(Enter,Clock,Reset,Halt,Output,Input);
input[7:0] Input;
input Enter,Clock,Reset;
output[7:0] Output;
output Halt;

```

```

wire[2:0]IR75;

```

```
wire PCload,Meminst,MemWr,Aload, IRload,JMPmux,Sub,Aeq0,Apos;
wire[1:0] Asel;
```

```
DP dp1(
  .Input(Input),
  .Clock(Clock),
  .Reset(Reset),
  .Output(Output),
  .IRload(IRload),
  .JMPmux(JMPmux),
  .PCload(PCload),
  .Meminst(Meminst),
  .MemWr(MemWr),
  .Asel(Asel),
  .Aload(Aload),
  .IR75(IR75),
  .Aeq0(Aeq0),
  .Apos(Apos),
  .Sub(Sub));
```

```
CU cu1(
  .Enter(Enter),
  .Clock(Clock),
  .Reset(Reset),
  .Halt(Halt),
  .IRload(IRload),
  .JMPmux(JMPmux),
  .PCload(PCload),
  .Meminst(Meminst),
  .MemWr(MemWr),
  .Asel(Asel),
  .Aload(Aload),
  .IR75(IR75),
  .Aeq0(Aeq0),
  .Apos(Apos),
  .Sub(Sub));
```

```
endmodule
```

```
/*****
```

2) 微处理器 Nibbler 的 Verilog 实现

Nibbler 的部分核心 Verilog 代码

Nibbler.v

```
/***** A Verilog implementation of Nibbler *****/
```

```
`default_nettype none
```

```
module Nibbler(
  input wire      clk_in,
```

```

output wire [11:0]    pgm_addr_o,
input wire [7:0]      pgm_data_in,

output wire [11:0]    data_addr_o,
inout wire [3:0]      data_io,
output wire           data_we_o,
output wire           data_oe_o,
output wire           data_ce_o,

output wire [3:0]      port_0_o,
output wire [3:0]      port_1_o,
input wire [3:0]       port_0_in
);

reg [11:0] PC;
reg [7:0] FETCH;
reg [3:0] ACC, port0, port1;
reg NC, NZ;
reg phase, reset;
reg [10:0] uCDATA;
wire inc_pc, load_pc, load_a, load_f, carry_in, cs_ram, we_ram, oe_alu, oe_in, oe_oprnd,
load_o;
wire [3:0] databus, operand, alu_out;
wire alu_n_zero, alu_cn4;

// hardwired microcode 硬连线实现的微码
always @(*)
begin
    casex ({ FETCH[7:4], NC, NZ, phase })
        // +- inc_pc
        // | +- load pc
        // || +- load_a
        // ||| + load_f
        // |||| +c_in
        // ||||| + cs_ram
        // ||||| + we_ram
        // ||||| +- oe_alu
        // ||||| +- oe_in
        // ||||| +- oe_oprnd
        // ||||| +- load_o
        7'bxxxx_x_x_0: uCDATA = 11'b1_1_1_1_1_1_1_0_1_1_1; //Any
        7'b0000_0_x_1: uCDATA = 11'b0_0_1_1_1_1_1_0_1_1_1; //JC
        7'b0000_1_x_1: uCDATA = 11'b1_1_1_1_1_1_1_0_1_1_1; //JC
        7'b0001_0_x_1: uCDATA = 11'b1_1_1_1_1_1_1_0_1_1_1; //JNC
        7'b0001_1_x_1: uCDATA = 11'b0_0_1_1_1_1_1_0_1_1_1; //JNC
        7'b0010_x_x_1: uCDATA = 11'b0_1_1_0_0_1_1_1_1_0_1; //CMPI
        7'b0011_x_x_1: uCDATA = 11'b1_1_1_0_0_0_1_1_1_1_1; //CMPM
        7'b0100_x_x_1: uCDATA = 11'b0_1_0_0_0_1_1_1_1_0_1; //LIT
        7'b0101_x_x_1: uCDATA = 11'b0_1_0_0_0_1_1_1_0_1_1; //IN
        7'b0110_x_x_1: uCDATA = 11'b1_1_0_0_0_0_1_1_1_1_1; //LD
    endcase
end

```

```

7'b0111_x_x_1: uCDATA = 11'b1_1_1_1_0_0_1_1_1; //ST
7'b1000_x_0_1: uCDATA = 11'b0_0_1_1_1_1_0_1_1_1; //JZ
7'b1000_x_1_1: uCDATA = 11'b1_1_1_1_1_1_0_1_1_1; //JZ
7'b1001_x_0_1: uCDATA = 11'b1_1_1_1_1_1_0_1_1_1; //JNZ
7'b1001_x_1_1: uCDATA = 11'b0_0_1_1_1_1_0_1_1_1; //JNZ
7'b1010_x_x_1: uCDATA = 11'b0_1_0_0_1_1_1_1_0_1; //ADDI
7'b1011_x_x_1: uCDATA = 11'b1_1_0_0_1_0_1_1_1_1_1; //ADDM
7'b1100_x_x_1: uCDATA = 11'b0_0_1_1_1_1_1_0_1_1_1; //JMP
7'b1101_x_x_1: uCDATA = 11'b0_1_1_1_1_1_1_0_1_1_0; //OUT
7'b1110_x_x_1: uCDATA = 11'b0_1_0_0_0_1_1_1_1_0_1; //NORI
7'b1111_x_x_1: uCDATA = 11'b1_1_0_0_0_0_1_1_1_1_1; //NORM
default:      uCDATA = 11'b0_1_1_1_0_0_1_1_1_1_1; //None
endcase

end

assign { inc_pc, load_pc, load_a, load_f, carry_in, cs_ram, we_ram, oe_alu, oe_in, oe_oprnd,
load_o } = uCDATA;

assign databus = (oe_oprnd == 1'b0) ? FETCH[3:0]:
                ((cs_ram == 1'b0) && (we_ram == 1'b1)) ? data_io:
                //(oe_alu == 1'b0) ? alu_out:
                (oe_in == 1'b0) ? port_0_in:4'h0;
assign operand = FETCH[3:0];

assign pgm_addr_o = PC;
assign data_addr_o = { operand, pgm_data_in };
assign data_io = ((we_ram == 1'b0) && (cs_ram == 1'b0)) ? alu_out:4'hz;
assign data_we_o = we_ram;
assign data_oe_o = !we_ram;
assign data_ce_o = cs_ram;
assign port_0_o = port0;
assign port_1_o = port1;

Nibbler_ALU alu(
.I(FETCH[7:4]),
.NCARRYIN(NC),
.NZEROIN(NZ),
.PHASE(phase),
.ACCUMULATOR(ACC),
.DATABUS(databus),
.ALUO(alu_out),
.NZERO(alu_n_zero),
.CN4(alu_cn4)
);

always @(posedge clk_in)
begin
    phase <= ~phase;
    if (phase)
        reset <= 1'b1;
    if (inc_pc)

```

```

        PC <= PC + 12'h1;
    if (~load_pc & reset)
        PC <= { operand, pgm_data_in };
    if (phase == 1'b0)
        FETCH <= pgm_data_in;
    if (~load_a)
        ACC <= alu_out;
    if (~load_f)
        NC <= alu_cn4;
    if (~load_f)
        NZ <= alu_n_zero;
    if (~load_o)
        begin
            if (operand[0] == 1'b0)
                port0 <= alu_out;
            if (operand[1] == 1'b0)
                port1 <= alu_out;
        end
    end

end

initial
begin
    reset = 0;
    PC = 0;
    phase = 0;
    FETCH = 0;
    NC = 0;
    NZ = 0;
    ACC = 0;
end
endmodule
/*****/

```

Nibbler_ALU.v

/***** A 4 bit ALU for Nibbler, based on a 74181 *****/

```

module Nibbler_ALU(
    input wire [3:0] I,
    input wire NCARRYIN,
    input wire NZEROIN,
    input wire PHASE,
    input wire [3:0] ACCUMULATOR,
    input wire [3:0] DATABUS,
    output wire [3:0] ALUO,
    output wire NZERO,
    output wire CN4
);

```

// ALU

reg S3, S2, S1, S0, M;

```

always @(*)
begin
    casex ({I, NCARRYIN, NZEROIN, PHASE})
        7'bxxxx_x_x_0: { M, S3, S2, S1, S0 } = 5'b0_0000; //Any
        7'b0000_0_x_1: { M, S3, S2, S1, S0 } = 5'b0_0000; //JC
        7'b0000_1_x_1: { M, S3, S2, S1, S0 } = 5'b0_0000; //JC
        7'b0001_0_x_1: { M, S3, S2, S1, S0 } = 5'b0_0000; //JNC
        7'b0001_1_x_1: { M, S3, S2, S1, S0 } = 5'b0_0000; //JNC
        7'b0010_x_x_1: { M, S3, S2, S1, S0 } = 5'b0_0110; //CMPI
        7'b0011_x_x_1: { M, S3, S2, S1, S0 } = 5'b0_0110; //CMPM
        7'b0100_x_x_1: { M, S3, S2, S1, S0 } = 5'b1_1010; //LIT
        7'b0101_x_x_1: { M, S3, S2, S1, S0 } = 5'b1_1010; //IN
        7'b0110_x_x_1: { M, S3, S2, S1, S0 } = 5'b1_1010; //LD
        7'b0111_x_x_1: { M, S3, S2, S1, S0 } = 5'b0_0000; //ST
        7'b1000_x_0_1: { M, S3, S2, S1, S0 } = 5'b0_0000; //JZ
        7'b1000_x_1_1: { M, S3, S2, S1, S0 } = 5'b0_0000; //JZ
        7'b1001_x_0_1: { M, S3, S2, S1, S0 } = 5'b0_0000; //JNZ
        7'b1001_x_1_1: { M, S3, S2, S1, S0 } = 5'b0_0000; //JNZ
        7'b1010_x_x_1: { M, S3, S2, S1, S0 } = 5'b0_1001; //ADDI
        7'b1011_x_x_1: { M, S3, S2, S1, S0 } = 5'b0_1001; //ADDM
        7'b1100_x_x_1: { M, S3, S2, S1, S0 } = 5'b0_0000; //JMP
        7'b1101_x_x_1: { M, S3, S2, S1, S0 } = 5'b0_0000; //OUT
        7'b1110_x_x_1: { M, S3, S2, S1, S0 } = 5'b1_0001; //NORI
        7'b1111_x_x_1: { M, S3, S2, S1, S0 } = 5'b1_0001; //NORM
        default: { M, S3, S2, S1, S0 } = 5'h0;
    endcase

end

ttl_181 alu0(
    .cn_in(NCARRYIN),
    .m_in(M),
    .a_in(ACCUMULATOR),
    .b_in(DATABUS),
    .s_in({S3, S2, S1, S0}),
    .cn4_o(CN4),
    .f_o(ALUO),
    .x_o(),
    .y_o(),
    .aeqb_o()
);

assign NZERO = ALUO == 4'h0;

endmodule

// A Model 4-Bit ALU '181
module ttl_181(
    input wire          cn_in,
    input wire          m_in,
    input wire [3:0]    a_in,

```

```

input wire [3:0]      b_in,
input wire [3:0]      s_in,
output wire [3:0]     f_o,
output wire           x_o,
output wire           y_o,
output wire           aeqb_o,
output wire           cn4_o
);

```

```

wire b0, b1, b2, b3, a0, a1, a2, a3, s0, s1, s2, s3;
wire f0, f1, f2, f3, cn4, y, x, aeqb;
wire cn, nm;

```

```
//wire [3:0] BH, AL;
```

```

assign cn = cn_in;
assign nm = ~m_in;
assign b0 = b_in[0];
assign b1 = b_in[1];
assign b2 = b_in[2];
assign b3 = b_in[3];
assign a0 = a_in[0];
assign a1 = a_in[1];
assign a2 = a_in[2];
assign a3 = a_in[3];
assign s0 = s_in[0];
assign s1 = s_in[1];
assign s2 = s_in[2];
assign s3 = s_in[3];

```

```
wire b3h, a3l, b2h, a2l, b1h, a1l, b0h, a0l;
```

```

assign b3h = ~((b3 & s3 & a3) | (a3 & s2 & (~b3)));
assign a3l = ~(((~b3) & s1) | (s0 & b3) | a3);
assign b2h = ~((b2 & s3 & a2) | (a2 & s2 & (~b2)));
assign a2l = ~(((~b2) & s1) | (s0 & b2) | a2);
assign b1h = ~((b1 & s3 & a1) | (a1 & s2 & (~b1)));
assign a1l = ~(((~b1) & s1) | (s0 & b1) | a1);
assign b0h = ~((b0 & s3 & a0) | (a0 & s2 & (~b0)));
assign a0l = ~(((~b0) & s1) | (s0 & b0) | a0);

```

```

//assign BH = { b3h, b2h, b1h, b0h };
//assign AL = { a3l, a2l, a1l, a0l };

```

```

assign y = ~(a3l | (b3h & a2l) | (b3h & b2h & a1l) | (b3h & b2h & b1h & a0l));
assign cn4 = ~y | (b3h & b2h & b1h & b0h & cn);
assign x = ~(b3h & b2h & b1h & b0h);

```

```

assign f3 = (b3h ^ a3l) ^ ( ~( cn & b0h & b1h & b2h & nm) | (b1h & b2h & a0l & nm) | (b2h
& a1l & nm) | (a2l & nm) );
assign f2 = (b2h ^ a2l) ^ ( ~( cn & b0h & b1h & nm) | (b1h & a0l & nm) | (a1l & nm) );

```



```

assign f1 = (b1h ^ a1l) ^ ( ~( cn & b0h & nm) | (a0l & nm) );
assign f0 = (b0h ^ a0l) ^ ( ~(nm & cn) );

assign aeqb = f3 & f2 & f1 & f0;

assign f_o = { f3, f2, f1, f0 };
assign y_o = y;
assign x_o = x;
assign aeqb_o = aeqb;
assign cn4_o = cn4;

endmodule

/*****

```

习题

[10.1] 请下列 MIPS 汇编代码转换成机器语言，并将指令写成 16 进制数的格式。

```

add    $t0, $s0, $s1
lw     $t0, 0x20($t7)
addi   $s0, $0, -10

```

[10.2] 请下列 MIPS 汇编代码转换成机器语言。并将指令写成 16 进制数的格式。

```

addi   $s0, $0, 73
sw     $t1, -7($t2)
sub    $t1, $s7, $s2

```

[10.3] 请下列程序是一段机器语言的程序，左边的数字是指令在存储器中的地址，右边的数字是存在该地址中的指令。**\$a0** 是输入，其初值是一个正数 **n**。**\$v0** 是输出。

- 1) 将该机器语言的程序转换成 MIPS 汇编代码。
- 2) 然后反向工程成最可能编译为该汇编代码的高级程序，写出高级 C 语言程序。
- 3) 用文字解释该程序的功能。

0x00400000	0x20080000
0x00400004	0x20090001
0x00400008	0x0089502a
0x0040000c	0x15400003
0x00400010	0x01094020
0x00400014	0x21290002
0x00400018	0x08100002
0x0040001c	0x01001020

[10.4] 请重新设计 EC-1 微处理器使得其满足以下各项改变：

- 修改 OUTPUT 指令使得只有在 OUTPUT 指令执行时输出端才输出值。
- 修改 INPUT 指令使得在执行下个指令之前它将会一直等待一个外部 ENTER 按键信号。
- 加一个额外 INC 指令，该指令使用操作码 000。INC 指令用来使累加器加 1。
- 加一个额外的 LOAD 指令，该指令用操作码 001。LOAD 指令用来将内存中地址为 aaaa 的内容加载到累加器中，其中 aaaa 用该指令的最低四位编码。

[10.5] 请写出 EC-1 微处理的行为级 Verilog 代码。

[10.6] 请重写出题 10.4 中每次修改后的 EC-1 微处理其的行为级 Verilog 代码。

[10.7] 写出并在 EC-2 上运行下列程序：

- 输入两个数，输出这两个数的和。
- 输入两个数，输出两个数中较大的数。
- 输入两个数，输出两个数的乘积。
- 连续输入数字直至遇到 0，输出所有输入数的个数。
- 连续输入数字直至遇到 0，输出所有输入数的和。
- 连续输入数字直至遇到 0，输出所有输入数中的最大值。
- 连续输入数字直至遇到 0，输出所有输入数的最大与次大数。
- 连续输入数字直至遇到 0，升序输出所有输入数。

[10.8] 请重新设计 EC-2 微处理器使得其分别满足以下每项改变：

- 用 LSHIFT 指令替换 EC-2 指令集中的 SUB 指令。LSHIFT 指令用来使累加器中的内容左移一位。移位操作的结果写入累加器。
- 用 OUTPUT 指令替换 EC-2 指令集中的 SUB 指令。OUTPUT 指令用来把累加器的内容输出到输出端口。但在 OUTPUT 指令未执行时，输出端不会输出任何数。
- 加入一个额外的 LSHIFT 指令到 EC-2 指令集中，LSHIFT 的指令如(a)所定义。
- 加入一个额外的 OUTPUT 指令到 EC-2 指令集中，OUTPUT 的指令如(b)所定义。

[10.9] 给出如下指令集，请重新设计 EC2 数据通路以实现这些指令。

Instruction	Encoding	Operation	Comment
-------------	----------	-----------	---------

Data Movement Instructions

LDA A,rrr	0001 0rrr	$A \leftarrow R[rrr]$	Load accumulator from register
STA rrr,A	0010 0rrr	$R[rrr] \leftarrow A$	Load register from accumulator
LDM A,aaaaaa	0011 0000 00 aa aaaa	$A \leftarrow M[aaaaaa]$	Load accumulator from memory
STM aaaaaa,A	0100 0000 00 aa aaaa	$M[aaaaaa] \leftarrow A$	Load memory from accumulator
LDI A,iiiiiii	0101 0000 iiii iiii	$A \leftarrow iiii$	Load accumulator with immediate value (iiii is a signed number)

Jump Instructions

JMP absolute	0110 0000 00 aa aaaa	$PC = aaaaaa$	Absolute unconditional jump
--------------	-------------------------	---------------	-----------------------------

JMPR relative	0110 smmm	IF (smmm != 0) THEN IF (s == 0) THEN PC = PC + mmm ELSE PC = PC – mmm	Relative unconditional jump (smmm is in sign and magnitude format)
JZ absolute	0111 0000 00 aa aaaa	IF (A == 0) THEN PC = aaaaaa	Absolute jump if A is zero
JZR relative	0111 smmm	IF (A == 0 AND smmm != 0) THEN IF (s == 0) THEN PC = PC + mmm ELSE PC = PC – mmm	Relative jump if A is zero (smmm is in sign and magnitude format)
JNZ absolute	1000 0000 00 aaaaaa	IF (A != 0) THEN PC = aaaaaa	Absolute jump if A is not zero
JNZR relative	1000 smmm	IF (A != 0 AND smmm != 0) THEN IF (s == 0) THEN PC = PC + mmm ELSE PC = PC – mmm	Relative jump if A is not zero (smmm is in sign and magnitude format)
JP absolute	1001 0000 00 aa aaaa	IF (A == positive) THEN PC = aaaaaa	Absolute jump if A is positive
JPR relative	1001 smmm	IF (A == positive AND smmm != 0) THEN IF (s == 0) THEN PC = PC + mmm ELSE PC = PC – mmm	Relative jump if A is positive (smmm is in sign and magnitude format)

Arithmetic and Logical Instructions

AND A,rrr	1010 0rrr	$A \leftarrow A \text{ AND } R[\text{rrr}]$	Accumulator AND register
OR A,rrr	1011 0rrr	$A \leftarrow A \text{ OR } R[\text{rrr}]$	Accumulator OR register
ADD A,rrr	1100 0rrr	$A \leftarrow A + R[\text{rrr}]$	Accumulator + register
SUB A,rrr	1101 0rrr	$A \leftarrow A - R[\text{rrr}]$	Accumulator – register
NOT A	1110 0000	$A \leftarrow \text{NOT } A$	Invert accumulator
INC A	1110 0001	$A \leftarrow A + 1$	Increment accumulator
DEC A	1110 0010	$A \leftarrow A - 1$	Decrement accumulator
SHFL A	1110 0011	$A \leftarrow A \ll 1$	Shift accumulator left
SHFR A	1110 0100	$A \leftarrow A \gg 1$	Shift accumulator right
ROTR A	1110 0101	$A \leftarrow \text{Rotate_right}(A)$	Rotate accumulator right

Input/Output and Miscellaneous

In A	1111 0000	$A \leftarrow \text{Input}$	Input to accumulator
Out A	1111 0001	$\text{Output} \leftarrow A$	Output from accumulator
HALT	1111 0010	Halt	Halt execution
NOP	0000 0000	No operation	No operation

Notations:

- 1) A = accumulator
- 2) R = general register
- 3) M = memory
- 4) PC = program counter
- 5) rrr = three bits for specifying the general register number (0 – 7)
- 6) aaaaaa = six bits for specifying the memory address
- 7) iiiiii = an eight-bit signed number
- 8) smmm = four bits for specifying the relative jump displacement in sign and magnitude format. The most significant bit(s) determines whether to jump forward or backward (0 = forward, 1 = backward). The last three bits (mmm) specify the number of locations to increment or decrement from the current PC location.

[10.10] 请分析附录 C 中的 4 位 CPU 的电路图，并画出该 CPU 的体系结构图。

[10.11] 请分析附录 C 中的 4 位 CPU 的电路图，并详细说明该 CPU 支持有哪些指令。

[10.12] 如果将表 10.2 中的 *NORI* 和 *NORM* 指令修改为 *NANDI* 和 *NANDM* 指令，则 Nibbler 相应的微码需要怎么修改，才能实现正确的功能？

[10.13] 如果需要将图 10.26 的 4 位 Nibbler 计算机扩展为 8 位的，请写出你的主要硬件改造思路。

[10.14] 如果需要将 4 位 Nibbler 计算机扩展为 8 位的，请写出其 Verlog 描述中需要修改的部分。

[10.15] 请将 Nibbler 计算机设计和工具文件包 nibbler.zip 中的 guess-the-number.asm 文件，用其中的汇编器 Assembler.exe 编译为 .bin 文件，用仿真器 Simulator.exe 运行，并记录运行的结果。

[10.16] 假设您正根据图 10.26，构建一个 Nibbler 克隆版，需要在数据总线和 ALU 的 B 输入之间可以放置一个像 74HCT7541 的施密特触发缓冲器。请画出修改后的 4 位 Nibbler 计算机的原理图。

[10.17] 请画出[10.16]修改后的 Nibbler 计算机的 PCB 版图。

4-bit arithmetic logic unit

74HC/HCT181

FEATURES

- Full carry look-ahead for high-speed arithmetic operation on long words
- Provides 16 arithmetic operations: add, subtract, compare, double, plus 12 others
- Provides all 16 logic operations of two variables: EXCLUSIVE-OR, compare, AND, NAND, NOR, OR plus 10 other logic operations
- Output capability: standard,
A=B open drain
- I_{CC} category: MSI

GENERAL DESCRIPTION

The 74HC/HCT181 are high-speed Si-gate CMOS devices and are pin compatible with low power Schottky TTL (LSTTL). They are specified in compliance with JEDEC standard no. 7A.

The 74HC/HCT181 are 4-bit high-speed parallel Arithmetic Logic Units (ALU). Controlled by the four function select inputs (S_0 to S_3) and the mode control input (M), they can perform all the 16 possible logic operations or 16 different arithmetic operations on active HIGH or active LOW operands (see function table).

When the mode control input (M) is HIGH, all internal carries are inhibited and the device performs logic operations on the individual bits as listed. When M is LOW, the carries are enabled and the "181" performs arithmetic operations on the two 4-bit words. The "181" incorporates full internal carry look-ahead and provides for either ripple carry between devices using the C_{n+4} output, or for carry look-ahead between packages using the carry propagation ($\bar{P}$) and carry generate ($\bar{G}$) signals. $\bar{P}$ and $\bar{G}$ are not affected by carry in.

When speed requirements are not stringent, it can be used in a simple ripple carry mode by connecting the carry output (C_{n+4}) signal to the carry input (C_n) of the next unit.

For high-speed operation the device is used in conjunction with the "182" carry look-ahead circuit. One carry look-ahead package is required for each group of four "181" devices. Carry look-ahead can be provided at various levels and offers high-speed capability over extremely long word lengths.

The comparator output ($A=B$) of the device goes HIGH when all four function outputs (F_0 to F_3) are HIGH and can be used to indicate logic equivalence over 4 bits when the unit is in the subtract mode. $A=B$ is an open collector output and can be wired-AND with other $A=B$ outputs to give a comparison for more than 4 bits. The open drain output $A=B$ should be used with an external pull-up resistor in order to establish a logic HIGH level. The $A=B$ signal can also be used with the C_{n+4} signal to indicate $A > B$ and $A < B$.

The function table lists the arithmetic operations that are performed without a carry in. An incoming carry adds a one to each operation. Thus, select code LHLH generates A minus B minus 1 (2's complement notation) without a carry in and generates A minus B when a carry is applied.

Because subtraction is actually performed by complementary addition (1's complement), a carry out means borrow; thus, a carry is generated when there is no under-flow and no carry is generated when there is underflow.

As indicated, the "181" can be used with either active LOW inputs producing active LOW outputs or with active HIGH inputs producing active HIGH outputs.

For either case the table lists the operations that are performed to the operands.

ORDERING INFORMATION

TYPE NUMBER	PACKAGE		
	NAME	DESCRIPTION	VERSION
74HC181N3; 74HCT181N3	DIP24	plastic dual in-line package; 24 leads (300 mil)	SOT222-1
74HC181N; 74HCT181N	DIP24	plastic dual in-line package; 24 leads (600 mil)	SOT101-1
74HC181D; 74HCT181D	SO24	plastic small outline package; 24 leads; body width 7.5 mm	SOT137-1

4-bit arithmetic logic unit

74HC/HCT181

QUICK REFERENCE DATA

GND = 0 V; $T_{amb} = 25\text{ }^{\circ}\text{C}$; $t_r = t_f = 6\text{ ns}$

SYMBOL	PARAMETER	CONDITIONS	TYPICAL		UNIT
			HC	HCT	
t_{PHL}/t_{PLH}	propagation delay $\bar{A}_n$ or $\bar{B}_n$ to $A=B$ C_n to C_{n+4}	$C_L = 15\text{ pF}$; $V_{CC} = 5\text{ V}$	28 17	30 21	ns ns
C_i	input capacitance		3.5	3.5	pF
C_{PD}	power dissipation capacitance per L package	notes 1 and 2	90	92	pF

Notes

1. C_{PD} is used to determine the dynamic power dissipation (P_D in μW):

$$P_D = C_{PD} \times V_{CC}^2 \times f_i + \sum (C_L \times V_{CC}^2 \times f_o) \text{ where:}$$

f_i = input frequency in MHz

f_o = output frequency in MHz

$\sum (C_L \times V_{CC}^2 \times f_o)$ = sum of outputs

C_L = output load capacitance in pF

V_{CC} = supply voltage in V

2. For HC the condition is $V_i = \text{GND}$ to V_{CC}
For HCT the condition is $V_i = \text{GND}$ to $V_{CC} - 1.5\text{ V}$

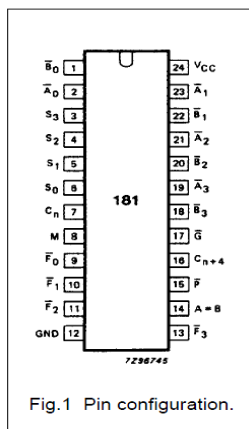


Fig.1 Pin configuration.

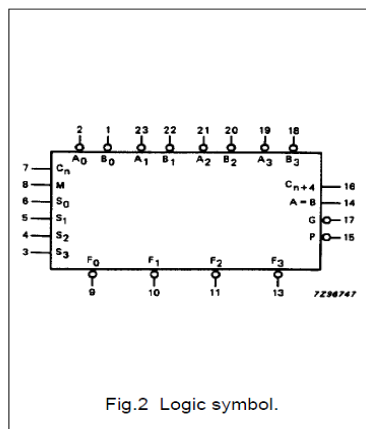


Fig.2 Logic symbol.

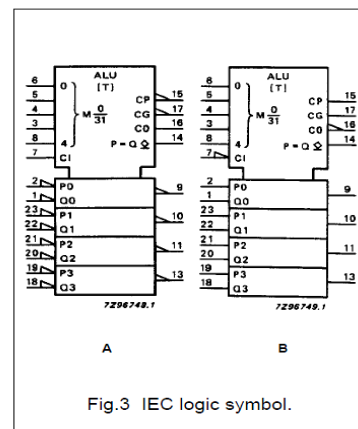


Fig.3 IEC logic symbol.

PIN DESCRIPTION

PIN NO.	SYMBOL	NAME AND FUNCTION
1, 22, 20, 18	$\bar{B}_0$ to $\bar{B}_3$	operand inputs (active LOW)
2, 23, 21, 19	$\bar{A}_0$ to $\bar{A}_3$	operand inputs (active LOW)
6, 5, 4, 3	S_0 to S_3	select inputs
7	C_n	carry input
8	M	mode control input
9, 10, 11, 13	$\bar{F}_0$ to $\bar{F}_3$	function outputs (active LOW)
12	GND	ground (0 V)
14	$A=B$	comparator output
15	$\bar{P}$	carry propagate output (active LOW)
16	C_{n+4}	carry output
17	$\bar{G}$	carry generate output (active LOW)
24	V_{CC}	positive supply voltage

4-bit arithmetic logic unit

74HC/HCT181

FUNCTION TABLES

MODE SELECT INPUTS				ACTIVE HIGH INPUTS AND OUTPUTS	
S ₃	S ₂	S ₁	S ₀	LOGIC (M=H)	ARITHMETIC ⁽²⁾ (M=L; C _n =H)
L	L	L	L	$\overline{A}$	A
L	L	L	H	$A + B$	A + B
L	L	H	L	$\overline{A}B$	$A + \overline{B}$
L	L	H	H	logical 0	minus 1
L	H	L	L	$\overline{A}\overline{B}$	A plus $\overline{A}\overline{B}$
L	H	L	H	$\overline{B}$	(A + B) plus $\overline{A}\overline{B}$
L	H	H	L	$A \oplus B$	A minus B minus 1
L	H	H	H	$\overline{A}\overline{B}$	$\overline{A}\overline{B}$ minus 1
H	L	L	L	$\overline{A} + B$	A plus AB
H	L	L	H	$\overline{A} \oplus \overline{B}$	A plus B
H	L	H	L	B	(A + $\overline{B}$) plus AB
H	L	H	H	AB	AB minus 1
H	H	L	L	logical 1	A plus A ⁽¹⁾
H	H	L	H	$A + \overline{B}$	(A + B) plus A
H	H	H	L	A + B	(A + $\overline{B}$) plus A
H	H	H	H	A	A minus 1

Notes to the function tables

- Each bit is shifted to the next more significant position.
- Arithmetic operations expressed in 2s complement notation.

H = HIGH voltage level

L = LOW voltage level

MODE SELECT INPUTS				ACTIVE LOW INPUTS AND OUTPUTS	
S ₃	S ₂	S ₁	S ₀	LOGIC (M=H)	ARITHMETIC ⁽²⁾ (M=L; C _n =L)
L	L	L	L	$\overline{A}$	A minus 1
L	L	L	H	$\overline{A}\overline{B}$	AB minus 1
L	L	H	L	$\overline{A} + B$	$\overline{A}\overline{B}$ minus 1
L	L	H	H	logical 1	minus 1
L	H	L	L	$\overline{A} + \overline{B}$	A plus (A + $\overline{B}$)
L	H	L	H	$\overline{B}$	AB plus (A + $\overline{B}$)
L	H	H	L	$\overline{A} \oplus \overline{B}$	A minus B minus 1
L	H	H	H	A + $\overline{B}$	A + B
H	L	L	L	$\overline{A}\overline{B}$	A plus (A + B)
H	L	L	H	$A \oplus B$	A plus B
H	L	H	L	B	$\overline{A}\overline{B}$ plus (A + B)
H	L	H	H	A + B	A + B
H	H	L	L	logical 0	A plus A ⁽¹⁾
H	H	L	H	$\overline{A}\overline{B}$	AB plus A
H	H	H	L	AB	$\overline{A}\overline{B}$ plus A
H	H	H	H	A	A

Notes to the function tables

- Each bit is shifted to the next more significant position.
- Arithmetic operations expressed in 2s complement notation.

H = HIGH voltage level

L = LOW voltage level

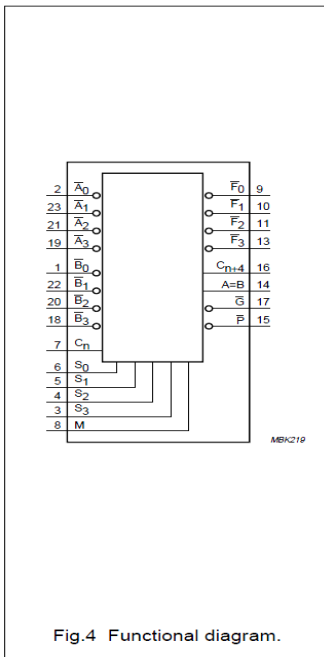


Fig.4 Functional diagram.

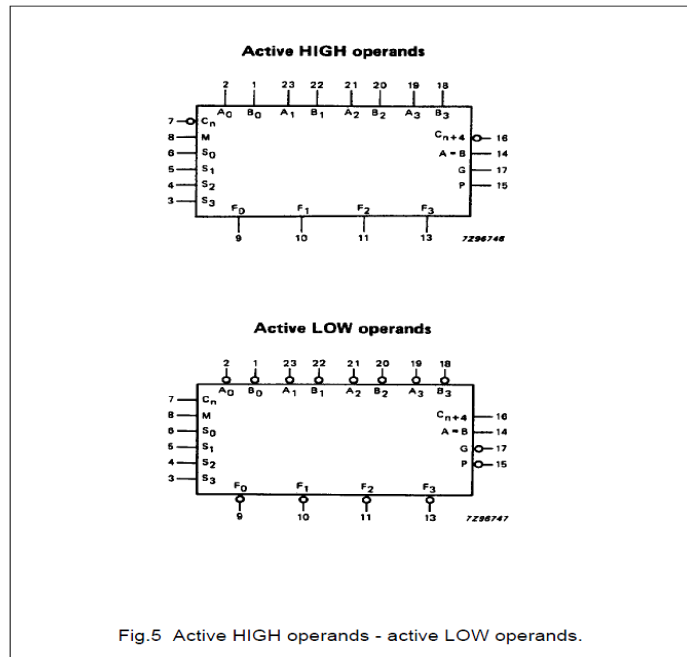


Fig.5 Active HIGH operands - active LOW operands.

4-bit arithmetic logic unit

74HC/HCT181

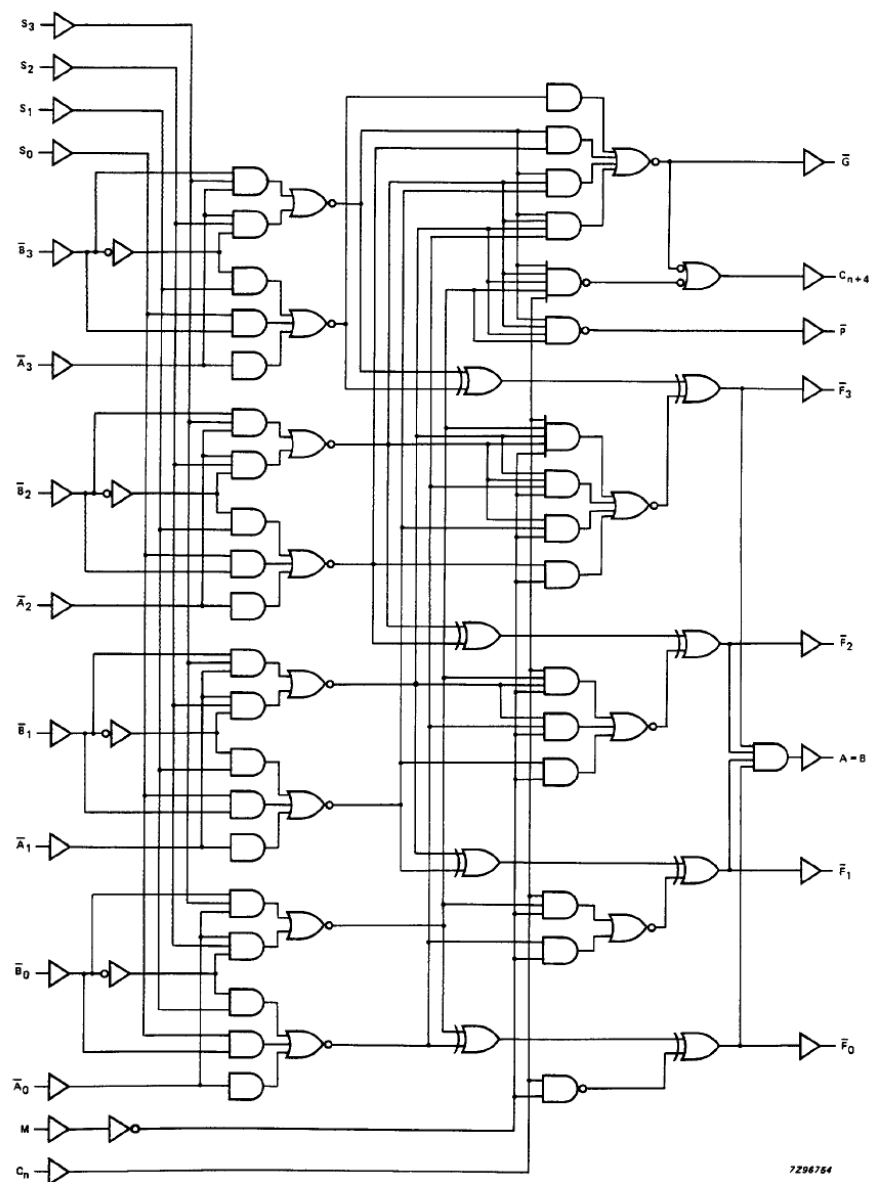


Fig.6 Logic diagram.

4-bit arithmetic logic unit

74HC/HCT181

RATINGS (for A=B output only)

Limiting values in accordance with the Absolute Maximum System (IEC 134)

Voltage are referenced to GND (ground = 0 V)

SYMBOL	PARAMETER	MIN.	MAX.	UNIT	CONDITIONS
V_O	DC output voltage	-0.5	+7.0	V	
$-I_{OK}$	DC output diode current		20	mA	for $V_O < -0.5$ V
$-I_O$	DC output source or sink current		25	mA	for -0.5 V $< V_O$

DC CHARACTERISTICS FOR 74HCFor the DC characteristics see *"74HC/HCT/HCU/HCMOS Logic Family Specifications"*.

Output capability: standard

 I_{CC} category: MSI

Voltages are referenced to GND (ground = 0 V)

SYMBOL	PARAMETER	T _{amb} (°C)							UNIT	TEST CONDITIONS		
		74HC								V _{CC} (V)	V _{IL}	OTHER
		+25			−40 to +85		−40 to +125					
		min.	typ.	max.	min.	max.	min.	max.				
I _{oz}	HIGH level output leakage current			0.5		5.0		10.0	μA	2.0 to 6.0	V _{IL}	note 1 V _O = 0 or 6 V

Note to the DC characteristics

- The maximum operating output voltage ($V_{O(max)}$) is 6.0 V.

Table 1 SUM MODE TESTFunction inputs $S_0 = S_3 = 4.5$ V, $M = S_1 = S_2 = 0$ V

PARAMETER	INPUT UNDER TEST	OTHER INPUT, SAME BIT		OTHER DATA INPUTS		OUTPUT UNDER TEST
		Apply 4.5 V	Apply GND	Apply 4.5 V	Apply GND	
t_{PLH}/t_{PHL}	$\bar{A}_i$	$\bar{B}_i$	none	remaining $\bar{A}$ and $\bar{B}$	C_n	$\bar{F}_i$
t_{PLH}/t_{PHL}	B_i	$\bar{A}_i$	none	remaining $\bar{A}$ and $\bar{B}$	C_n	$\bar{F}_i$
t_{PLH}/t_{PHL}	$\bar{A}_i$	$\bar{B}_i$	none	none	remaining $\bar{A}$ and $\bar{B}$, C_n	$\bar{P}$
t_{PLH}/t_{PHL}	B_i	$\bar{A}_i$	none	none	remaining $\bar{A}$ and $\bar{B}$, C_n	$\bar{P}$
t_{PLH}/t_{PHL}	$\bar{A}_i$	none	$\bar{B}_i$	remaining $\bar{B}$	remaining $\bar{A}$, C_n	$\bar{G}$
t_{PLH}/t_{PHL}	B_i	none	$\bar{A}_i$	remaining $\bar{B}$	remaining $\bar{A}$, C_n	$\bar{G}$
t_{PLH}/t_{PHL}	$\bar{A}_i$	none	$\bar{B}_i$	remaining $\bar{B}$	remaining $\bar{A}$, C_n	C_{n+4}
t_{PLH}/t_{PHL}	B_i	none	$\bar{A}_i$	remaining $\bar{B}$	remaining $\bar{A}$, C_n	C_{n+4}
t_{PLH}/t_{PHL}	C_n	none	none	all $\bar{A}$	all $\bar{B}$	any $\bar{F}$ or C_{n+4}

Table 2 DIFFERENTIAL MODE TESTFunction inputs $S_1 = S_2 = 4.5$ V, $M = S_0 = S_3 = 0$ V

PARAMETER	INPUT UNDER TEST	OTHER INPUT, SAME BIT		OTHER DATA INPUTS		OUTPUT UNDER TEST
		Apply 4.5 V	Apply GND	Apply 4.5 V	Apply GND	
t_{PLH}/t_{PHL}	$\bar{A}_i$	none	$\bar{B}_i$	remaining $\bar{A}$	remaining $\bar{B}$, C_n	$\bar{F}_i$
t_{PLH}/t_{PHL}	B_i	$\bar{A}_i$	none	remaining $\bar{A}$	remaining $\bar{B}$, C_n	$\bar{F}_i$
t_{PLH}/t_{PHL}	$\bar{A}_i$	none	$\bar{B}_i$	none	remaining $\bar{A}$ and $\bar{B}$, C_n	$\bar{P}$
t_{PLH}/t_{PHL}	B_i	$\bar{A}_i$	none	none	remaining $\bar{A}$ and $\bar{B}$, C_n	$\bar{P}$
t_{PLH}/t_{PHL}	$\bar{A}_i$	$\bar{B}_i$	none	none	remaining $\bar{A}$ and $\bar{B}$, C_n	$\bar{G}$
t_{PLH}/t_{PHL}	B_i	none	$\bar{A}_i$	none	remaining $\bar{A}$ and $\bar{B}$, C_n	$\bar{G}$
t_{PLZ}/t_{PZL}	$\bar{A}_i$	none	$\bar{B}_i$	remaining $\bar{A}$	remaining $\bar{B}$, C_n	A=B
t_{PLZ}/t_{PZL}	B_i	$\bar{A}_i$	none	remaining $\bar{A}$	remaining $\bar{B}$, C_n	A=B
t_{PLH}/t_{PHL}	$\bar{A}_i$	$\bar{B}_i$	none	none	remaining $\bar{A}$ and $\bar{B}$, C_n	C_{n+4}
t_{PLH}/t_{PHL}	B_i	none	$\bar{A}_i$	none	remaining $\bar{A}$ and $\bar{B}$, C_n	C_{n+4}
t_{PLH}/t_{PHL}	C_n	none	none	all $\bar{A}$ and $\bar{B}$	none	any $\bar{F}$ or C_{n+4}

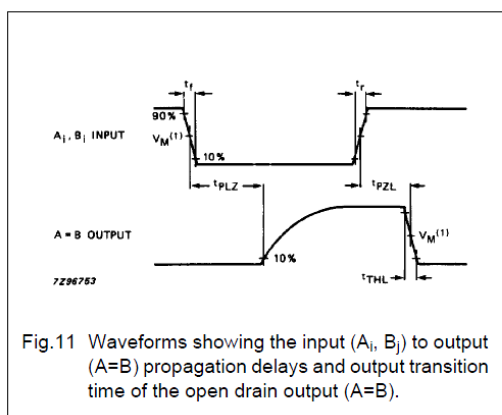
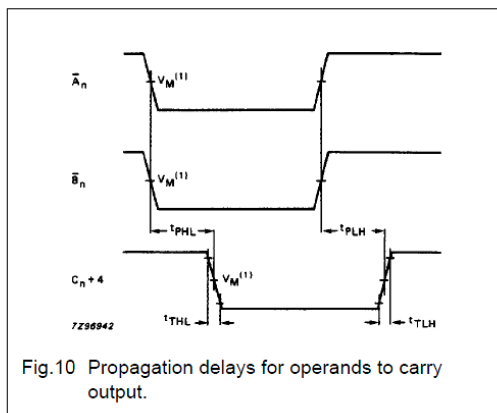
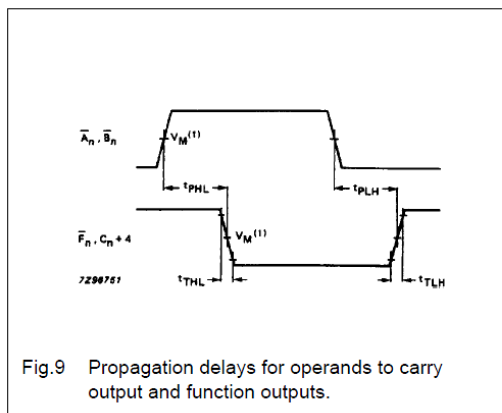
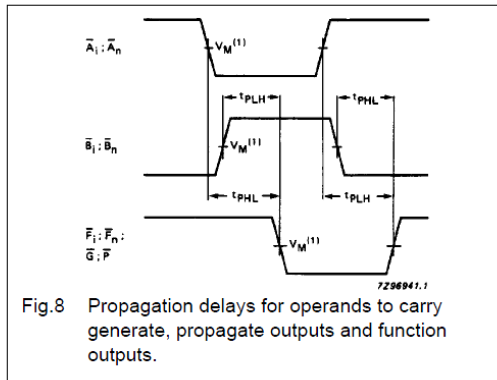
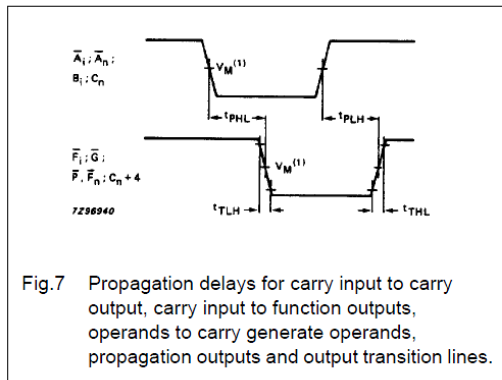
Table 3 LOGIC MODE TESTFunction inputs $M = S_1 = S_2 = 4.5$ V, $S_0 = S_3 = 0$ V

PARAMETER	INPUT UNDER TEST	OTHER INPUT, SAME BIT		OTHER DATA INPUTS		OUTPUT UNDER TEST
		Apply 4.5 V	Apply GND	Apply 4.5 V	Apply GND	
t_{PLH}/t_{PHL}	$\bar{A}_i$	$\bar{B}_i$	none	none	remaining $\bar{A}$ and $\bar{B}$, C_n	$\bar{F}_i$
t_{PLH}/t_{PHL}	B_i	$\bar{A}_i$	none	none	remaining $\bar{A}$ and $\bar{B}$, C_n	$\bar{F}_i$

4-bit arithmetic logic unit

74HC/HCT181

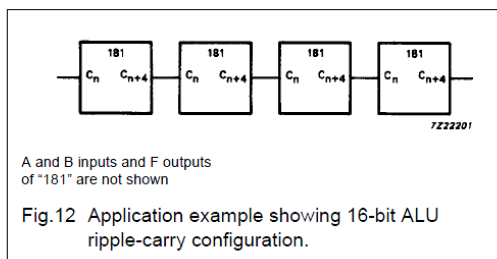
AC WAVEFORMS



Note to AC waveforms

- (1) HC: $V_M = 50\%$; $V_I = \text{GND to } V_{CC}$.
HCT: $V_M = 1.3 \text{ V}$; $V_I = \text{GND to } 3 \text{ V}$.

APPLICATION INFORMATION



附录 B MIPS Instructions

Table B.1 Instructions, sorted by opcode

Opcode	Name	Description	Operation
000000 (0)	R-type	all R-type instructions	see Table B.2
000001 (1) (rt = 0/1)	bltz/bgez	branch less than zero/ branch greater than or equal to zero	if ([rs] < 0) PC = BTA/ if ([rs] ≥ 0) PC = BTA
000010 (2)	j	jump	PC = JTA
000011 (3)	jal	jump and link	\$ra = PC+4, PC = JTA
000100 (4)	beq	branch if equal	if ([rs]==[rt]) PC = BTA
000101 (5)	bne	branch if not equal	if ([rs]!= [rt]) PC = BTA
000110 (6)	blez	branch if less than or equal to zero	if ([rs] ≤ 0) PC = BTA
000111 (7)	bgtz	branch if greater than zero	if ([rs] > 0) PC = BTA
001000 (8)	addi	add immediate	[rt] = [rs] + SignImm
001001 (9)	addiu	add immediate unsigned	[rt] = [rs] + SignImm
001010 (10)	slti	set less than immediate	[rs] < SignImm ? [rt]=1 : [rt]=0
001011 (11)	sltiu	set less than immediate unsigned	[rs] < SignImm ? [rt]=1 : [rt]=0
001100 (12)	andi	and immediate	[rt] = [rs] & ZeroImm
001101 (13)	ori	or immediate	[rt] = [rs] ZeroImm
001110 (14)	xori	xor immediate	[rt] = [rs] ^ ZeroImm
001111 (15)	lui	load upper immediate	[rt] = {Imm, 16'b0}
010000 (16) (rs = 0/4)	mfc0, mtc0	move from/to coprocessor 0	[rt] = [rd]/[rd] = [rt] (rd is in coprocessor 0)
010001 (17)	F-type	fop = 16/17: F-type instructions	see Table B.3
010001 (17) (rt = 0/1)	bc1f/bc1t	fop = 8: branch if fpcond is FALSE/TRUE	if (fpcond == 0) PC = BTA/ if (fpcond == 1) PC = BTA
100000 (32)	lb	load byte	[rt] = SignExt ([Address] _{7:0})
100001 (33)	lh	load halfword	[rt] = SignExt ([Address] _{15:0})
100011 (35)	lw	load word	[rt] = [Address]
100100 (36)	lbu	load byte unsigned	[rt] = ZeroExt ([Address] _{7:0})
100101 (37)	lhu	load halfword unsigned	[rt] = ZeroExt ([Address] _{15:0})
101000 (40)	sb	store byte	[Address] _{7:0} = [rt] _{7:0}

(continued)

Table B.1 Instructions, sorted by opcode—Cont'd

Opcode	Name	Description	Operation
101001 (41)	sh	store halfword	[Address] _{15:0} = [rt] _{15:0}
101011 (43)	sw	store word	[Address] = [rt]
110001 (49)	lwc1	load word to FP coprocessor 1	[ft] = [Address]
111001 (56)	swc1	store word to FP coprocessor 1	[Address] = [ft]

Table B.2 R-type instructions, sorted by funct field

Funct	Name	Description	Operation
000000 (0)	sll	shift left logical	$[rd] = [rt] \ll \text{shamt}$
000010 (2)	srl	shift right logical	$[rd] = [rt] \gg \text{shamt}$
000011 (3)	sra	shift right arithmetic	$[rd] = [rt] \ggg \text{shamt}$
000100 (4)	sllv	shift left logical variable	$[rd] = [rt] \ll [rs]_{4:0}$ assembly: sllv rd, rt, rs
000110 (6)	srlv	shift right logical variable	$[rd] = [rt] \gg [rs]_{4:0}$ assembly: srlv rd, rt, rs
000111 (7)	srav	shift right arithmetic variable	$[rd] = [rt] \ggg [rs]_{4:0}$ assembly: srav rd, rt, rs
001000 (8)	jr	jump register	$PC = [rs]$
001001 (9)	jalr	jump and link register	$\$ra = PC + 4, PC = [rs]$
001100 (12)	syscall	system call	system call exception
001101 (13)	break	break	break exception
010000 (16)	mfhi	move from hi	$[rd] = [hi]$
010001 (17)	mthi	move to hi	$[hi] = [rs]$
010010 (18)	mflo	move from lo	$[rd] = [lo]$
010011 (19)	mtlo	move to lo	$[lo] = [rs]$
011000 (24)	mult	multiply	$\{[hi], [lo]\} = [rs] \times [rt]$
011001 (25)	multu	multiply unsigned	$\{[hi], [lo]\} = [rs] \times [rt]$
011010 (26)	div	divide	$[lo] = [rs]/[rt],$ $[hi] = [rs]\%[rt]$
011011 (27)	divu	divide unsigned	$[lo] = [rs]/[rt],$ $[hi] = [rs]\%[rt]$
100000 (32)	add	add	$[rd] = [rs] + [rt]$
100001 (33)	addu	add unsigned	$[rd] = [rs] + [rt]$
100010 (34)	sub	subtract	$[rd] = [rs] - [rt]$
100011 (35)	subu	subtract unsigned	$[rd] = [rs] - [rt]$
100100 (36)	and	and	$[rd] = [rs] \& [rt]$
100101 (37)	or	or	$[rd] = [rs] \mid [rt]$
100110 (38)	xor	xor	$[rd] = [rs] \wedge [rt]$
100111 (39)	nor	nor	$[rd] = \sim([rs] \mid [rt])$
101010 (42)	slt	set less than	$[rs] < [rt] ? [rd] = 1 : [rd] = 0$
101011 (43)	sltu	set less than unsigned	$[rs] < [rt] ? [rd] = 1 : [rd] = 0$

Table B.3 F-type instructions (fop = 16/17)

Funct	Name	Description	Operation
000000 (0)	add.s/add.d	FP add	$[fd] = [fs] + [ft]$
000001 (1)	sub.s/sub.d	FP subtract	$[fd] = [fs] - [ft]$
000010 (2)	mul.s/mul.d	FP multiply	$[fd] = [fs] * [ft]$
000011 (3)	div.s/div.d	FP divide	$[fd] = [fs]/[ft]$
000101 (5)	abs.s/abs.d	FP absolute value	$[fd] = ([fs] < 0) ? [-fs] : [fs]$
000111 (7)	neg.s/neg.d	FP negation	$[fd] = [-fs]$
111010 (58)	c.seq.s/c.seq.d	FP equality comparison	$\text{fpcond} = ([fs] == [ft])$
111100 (60)	c.lt.s/c.lt.d	FP less than comparison	$\text{fpcond} = ([fs] < [ft])$
111110 (62)	c.le.s/c.le.d	FP less than or equal comparison	$\text{fpcond} = ([fs] \leq [ft])$

附录 C. Simple Hardwired Four-bit CPU Schematic

